CCAO-F Revision Notes: Lessons 01 to 07

Claude Certified Associate – Foundations (exam code CCAO-F)

This document combines the seven lesson revision notes, in order. Each lesson has: where it sits in the exam blueprint, a memory map, topic-by-topic notes, practice questions with answers and explanations, and key takeaways. Lessons 03 to 07 also have an Exam Traps section and links to related points in other lessons.

Domain weights: Output Evaluation 21% (Lesson 03) · Workflow Integration 16% (04) · Governance 15% (06) · Prompting 14% (02) · Product & Model Selection 12% (01) · Configuration 12% (05) · Troubleshooting 10% (07).

Contents

- **01 · Claude Platform & Model Foundations**
  - Exam at a Glance
  - Memory Map
  - 1. The Four Up-Front Decisions
  - 2. What to Expect from Generative AI (5 Behaviours)
  - 3. Entry Points: Chat, Projects, Artifacts, Research
  - 4. Capability Layer: Skills, Code Execution, Memory
  - 5. Choosing Models: Haiku, Sonnet, Opus
  - 6. Context Management
  - Most Likely Questions
  - Key Takeaways
- **02 · Prompting and Task Execution**
  - Where This Lesson Sits in the Exam
  - Memory Map
  - 1. Description: The Backbone Competency
  - 2. Anatomy of an Effective Prompt
  - 3. Task Decomposition for Complex Requests
  - 4. Iterating Prompts to Improve Output
  - 5. Adapting Strategy by Task Type
  - 6. Repairing an Underperforming Prompt (Case Study)
  - Most Likely Questions
  - Key Takeaways
- **03 · Evaluating and Validating Claude Output**
  - Where This Lesson Sits in the Exam
  - Memory Map
  - 1. Discernment and Diligence

- 2. Discernment: Accuracy, Completeness, Fitness
- 3. Hallucinations, Inconsistencies, and Bias
- 4. Fact-Checking and Grounding Techniques
- 5. Diligence: When Human Review Is Non-Negotiable
- 6. Editing and Adapting Output for Your Audience
- 7. Choosing Output Formats
- 8. Putting It Together: A Review Decision Flow
- 9. Links From Lessons 01 and 02 That Are Tested Under This Domain
- 10. Common Exam Traps
- Most Likely Questions
- Key Takeaways

- **04 · Workflow Integration & Solution Design**
  - Where This Lesson Sits in the Exam
  - Memory Map
  - 1. Analyzing Requirements and Use Cases with Claude
  - 2. Research, Planning & Process Optimization
  - 3. Solution Design, Development & Iteration
  - 4. Delegation Mapping: Redesigning Workflows with Claude Inside
  - 5. Communicating Value and Limitations to Stakeholders
  - Exam Traps
  - Most Likely Questions
  - Key Takeaways
- **05 · Configuration & Knowledge Management**
  - Where This Lesson Sits in the Exam
  - Memory Map
  - 1. Configuring Claude Projects
  - 2. Connectors and Uploaded Knowledge
  - 3. System-Level Instructions That Stick
  - 4. Maintaining Configurations
  - Links to Other Lessons (Configuration Points Tested Here)
  - Exam Traps
  - Most Likely Questions
  - Key Takeaways
- **06 · Governance, Risk & Responsible Use**
  - Where This Lesson Sits in the Exam
  - Memory Map
  - 1. Appropriate vs Inappropriate Use Cases
  - 2. Skill Trust and Feature-Level Risk
  - 3. Data Sensitivity, Privacy & Feature Controls

- 4. Organizational Policies and Diligence as a Habit
- 5. Ethical Implications: Bias, Fairness, Transparency
- Links to Other Lessons (Governance Points Tested Here)
- Exam Traps
- Most Likely Questions
- Key Takeaways

- **07 · Troubleshooting & Optimization**
  - Where This Lesson Sits in the Exam
  - Memory Map
  - 1. Diagnosing Underperformance
  - 2. Adjusting Approach from Feedback and Results
  - 3. Optimizing Workflows for Efficiency and Effectiveness
  - Note on the Lesson's Disclaimer
  - Links to Other Lessons (Tested Under Domain 7)
  - Exam Traps
  - Most Likely Questions
  - Key Takeaways

01 · Claude Platform & Model Foundations (Revision Notes)

Exam at a Glance

Item	Detail
Exam / code	Claude Certified Associate – Foundations · CCAO-F
Questions	60 (multiple-choice and multiple-response; each item says how many to pick)
Time	120 minutes (about 2 minutes per question)
Pass mark	Scaled score 720 on a 100–1,000 scale
Scoring type	Criterion-referenced: measured against a fixed standard, not against other candidates
Score report	Pass/fail + scaled score + percent-correct per domain (domain % does not decide pass/fail)
Validity	12 months; free, non-proctored renewal assessment if renewed on time

Domain weights (where marks come from)

#	Domain	Weight	Main lesson
2	Output Evaluation and Validation	**21%**	03
4	Workflow Integration and Solution Design	16%	04
6	Governance, Risk, and Responsible Use	15%	06
1	Prompting and Task Execution	14%	02
3	**Product and Model Selection**	**12%**	**01 (this lesson)**
5	Configuration and Knowledge Management	12%	05
7	Troubleshooting and Optimization	10%	07

Domain 3 objectives this lesson covers

- Select the right product feature (Projects, research mode, chat, artifacts)
- Tell apart the model types (Haiku, Sonnet, Opus)
- Match the model to task needs (cost, speed, quality)
- Understand and manage context limits and memory (when to **restart, summarize, or persist**)

Expect overlap with Domain 2 (review every output) and Domain 5 (Projects, Memory).

Memory Map

```
CLAUDE PLATFORM & MODEL FOUNDATIONS
│
├── 4 DECISIONS BEFORE ANY PROMPT
│     1 Entry point → 2 Capability features → 3 Model → 4 Context management
│     (decides whether work builds on itself or needs constant re-setup)
│
├── 5 GEN-AI BEHAVIOURS (true for every feature)
```

```
│      ├── Responses vary (probabilistic, not deterministic) → build in review
│      ├── Confident tone ≠ accuracy → fabricated stats sound just as sure
│      ├── Context is a budget → long chats get summarized, early detail fades
│      ├── Knowledge has a training cutoff → use connector / web search / Research
│      └── Configured procedures (Skills) still vary → review stays in
│
├── ENTRY POINTS ("WHERE you work")
│      ├── Chat ......... one-off, quick, exploratory, not reused
│      ├── Projects ..... recurring work, stable context
│      │                   = standing instructions + knowledge base + own chat list
│      │                   (chats inside a Project do NOT share context with each other)
│      ├── Artifacts .... deliverable output, separate editable block
│      └── Research ..... deep multi-source synthesis (paid plans)
│                         quick current lookup → web search toggle in Chat
│
├── CAPABILITY LAYER ("WHAT Claude can do")
│      ├── Projects  → carry CONTEXT
│      ├── Skills    → define PROCEDURES (same steps every time)
│      ├── Code Exec → VERIFY COMPUTATION (must be correct, not plausible)
│      └── Memory    → persist CONTINUITY across sessions
│            ├── Settings → Memory: "Search & reference chats" vs "Generate memory from
chats"
│            ├── Project-scoped (automatic, separate per Project)
│            ├── Incognito (ghost icon): out of Memory + history; not org retention
│            └── Curate monthly: review, delete, update; accuracy > volume
│
├── MODELS ("HOW WELL, at what speed/cost")
│      ├── Haiku  → fastest, cheapest; structured, high-volume, low cost of error
│      ├── Sonnet → balanced; DEFAULT start for most knowledge work
│      ├── Opus   → nuanced judgment, multi-step reasoning, ambiguity, high stakes
│      └── (Fable → above Opus; long agentic/coding work; lesson flags as NOT exam-
relevant)
│      Rule: Sonnet first → up to Opus if quality short → down to Haiku if volume/speed
│
└── CONTEXT MANAGEMENT ("HOW LONG it works well")
       ├── Warning signs: ignores earlier instructions, only answers latest turn, accuracy
drops
       ├── RESTART  → new chat; Project instructions + KB carry over, thread does not
       ├── SUMMARIZE→ ask for state summary (decisions, WIP, open questions), paste into
new chat
       ├── PERSIST  → save to Memory or Project knowledge base for all future sessions
       └── Usage limits: short rolling window + weekly limits (separate pool for top tier)
             → segment big tasks, save interim work to KB, restart from summary
```

---

1. The Four Up-Front Decisions

Before writing any prompt, decide in this order:

1. **Entry point** (Chat, Project, Artifact, Research)
2. **Capability features** to switch on (Skills, Code Execution, Memory)
3. **Model** (Haiku, Sonnet, Opus)
4. **Context management** across the session

- These choices set the **quality ceiling** for everything that follows.
- They decide whether sessions **build on earlier work** or need **constant re-setup**.
- Learning "one or two things Claude does well" is fine for simple tasks, but not for recurring work, team projects, or deliverables that must survive review.

2. What to Expect from Generative AI (5 Behaviours)

These apply **no matter which feature** you use.

#	Behaviour	What it means	What you do
1	**Responses vary**	Same question twice gives two different answers. Output is based on probability, not deterministic. Neither is "the one correct answer".	Plan for variation; build **review** into any workflow needing consistency.
2	**Confident tone ≠ accuracy**	Claude is equally fluent when right or wrong. A made-up statistic sounds as sure as a real one.	Never treat confidence as proof. Verify.
3	**Context is a budget**	Each conversation has a working-memory limit. Near the limit, claude.ai **auto-summarizes earlier messages** (paid plans with Code Execution enabled). Summaries compress detail.	Expect early instructions to **lose force** late in a long session.
4	**Knowledge has a training cutoff**	Anything after the cutoff is outside reliable knowledge: recent events, current regulations, live market data.	Connect a current source, turn on **web search** in Chat, or use **Research**.
5	**Configured procedures still vary**	A Skill **reduces** variance but does **not eliminate** it.	Keep review in the workflow, however good the Skill.

Exam traps

- "Claude was confident, so it's correct" is always wrong.
- "A Skill guarantees identical output" is always wrong.
- "Claude knows about last month's regulation change" is unreliable without a current source.

3. Entry Points: Chat, Projects, Artifacts, Research

One interface, four ways of working. The choice decides **efficiency** and whether **today's context carries into tomorrow**.

3.1 Chat

- Default, **unstructured** conversation.
- Saved to history (left sidebar) and **can be continued later**.
- **Memory** and **past-chat search** can bring key context into new chats.
- What Chat lacks vs a Project: **deliberate persistence, standing instructions, curated knowledge base**.
- Use for: one-off questions, quick drafts, exploratory prompting, tasks you won't repeat; work that **starts and ends in that session**.

- **Signal you've outgrown Chat**: you keep opening new chats by **pasting the same background paragraph** you pasted last week.

3.2 Projects

- **Persistent workspaces.** Hold three things:
  1. **Standing instructions**: what Claude should know and do in every conversation in the Project.
  2. **Knowledge base**: documents, policies, reference files **uploaded once**, used without re-uploading.
  3. **Conversation history**: its **own** chat list, separate from your global history.
- Chats in one Project **share instructions + knowledge base**, but **do NOT share context with each other**.
- Solves the biggest productivity drain: **re-explaining the same background every session**.
- Example: RFP response for "Acme Inc". Client's documents → **knowledge base**; how to respond → **standing instruction**.

3.3 Artifacts

- Use when the result is a **deliverable**, not a conversational reply.
- Appears as a **separate, editable block alongside the chat**, not inside the thread.
- Use for: **draft documents, data tables, formatted reports, code**.
- Use **inline responses** for answers you'll act on **within** the conversation.
- Example: a final trip plan (photos, hotels, flights) as a shareable document.

> **NOTE:** Chats can also generate artifacts. But the Artifact entry point starts from the output you want - slides, document, PDF etc. With the Artifact entry point, you provide/pick a template, provide your design guidelines for Claude to generate the Artifact using the same. For example slides will follow your brand guidelines, if you provide them. Chat has no such feature.

3.4 Research

- **Deep, multi-source synthesis**; available on **paid plans**.
- Runs **multi-step searches** across many sources and **synthesizes** them.
- Compare with **web search**:
- Web search is a **per-chat toggle** in the chat input, available on **all plans**.
- On **Team and Enterprise**, an **Owner or Primary Owner** must first enable web search for the workspace in **organization capability settings**; only then can members switch it on.
- Official docs do not say whether the toggle is on or off by default.
- Rule: **quick current-info lookup → web search in Chat**; **deep investigation / synthesis of current info → Research**.

3.5 Selection Logic (memorise)

Task type	Entry point
One-off question or quick task, no reuse	**Chat**

Task type	Entry point
Recurring work, stable context (e.g. RFP responses using client docs)	**Project**
Output is a deliverable someone will open and read (e.g. weekly status report)	**Artifact**
Deep multi-source investigation or synthesis	**Research**
Quick current-information lookup	**Web search in Chat**

---

4. Capability Layer: Skills, Code Execution, Memory

> Entry points decide **where** you work. The capability layer decides **what** Claude can do there.

Three features extend plain text generation: **Skills** (consistent procedures), **Code Execution** (verified computation), **Memory** (continuity across sessions).

4.1 The Four-Layer Model

Layer	Job	Key phrase
Projects	Carry **context**	Background knowledge + standing instructions for the workstream
Skills	Define **procedures**	Step-by-step "how to do this task", consistently every time
Code Execution	**Verify computations**	When the result **must be correct, not merely plausible**
Memory	Persist **continuity**	Relevant facts carry forward without re-entry

- Use layers **alone or combined**, based on the task.

4.2 Memory

- Keeps **work-relevant facts across sessions** so you don't re-enter them.
- Good things to store: **recurring role context, output-format preferences, names of frequent collaborators, standing constraints** across projects.

Memory settings (Chat): Settings → Memory

Toggle	What it does	What it does NOT do
Search and reference chats	Looks through past chats when you refer to earlier work ("the exam prep plan we discussed"); retrieves content from specific old chats **on request**	Does **not** build a profile of you
Generate memory from chats	The **actual Memory feature**: distils details (role, preferences) from chats and stores them for future chats	n/a

Project-scoped Memory

- Memory is kept **separate per Project**: client A context never appears in client B sessions.

- **No separate setting**; it's on by default and distinct for each Project.
- Practice: **separate Projects for separate workstreams**; Memory follows the same boundaries.

Incognito mode

- Ghost icon, top right of the chat window.
- Keeps a session **out of Memory and chat history**.
- Applies to **standalone chats, outside Projects**.
- Use for **sensitive conversations** or exploratory work with **confidential inputs**.
- Does **NOT override your organization's data retention**.

Memory import from other AI platforms

- **Experimental** (as of June 2026).
- Documented for **Free, Pro, Max, Team** plans (**not Enterprise**).
- If auto-import isn't available: **add key facts to Memory manually** in memory settings, **not** into a Project knowledge base.

4.3 Memory Curation

- Stale memory can be **actively misleading** (accurate last quarter, wrong now).
- Review stored memories **at least once a month** (active users).
- **Delete or update** entries that no longer hold.
- Keep only information that **genuinely recurs** across sessions.
- Treat Memory like a **working file**. **Accuracy matters more than volume.**

4.4 Scenario: Monthly Regulatory Tracking Report

Before (months 1–2, in Chat): each session an analyst uploaded regulatory docs, re-pasted portfolio context, re-typed format instructions, and verified every number. Caught 2 errors in month 1 and 1 in month 2, all before release.

After (month 3, capability layer):

- Portfolio context + standing format instructions → **Project**
- Prior reports → **knowledge base**
- Report output format → **Skill**
- Numeric calculations → **Code Execution**
- Verification step → **still ran**

Result: time per session **65 → 30 minutes**; **no errors** in months 3–8.

The four questions she asked (map question to layer)

Question	Layer
Which parts are the same every time?	**Standing instructions + Skill**
Which reference material recurs across sessions?	**Knowledge base**
Which outputs must be computed correctly, not just "sound right"?	**Code Execution**

Question	Layer
Which context should carry across sessions without re-entry?	**Memory**

Lesson: automation **cut time**, but **verification was kept**.

---

5. Choosing Models: Haiku, Sonnet, Opus

> The capability layer decides **what** Claude does. The model decides **how well**, and at what cost in **speed**.

Goal: avoid **over-engineering routine work** and **under-resourcing high-stakes analysis**.

Model	Profile	Use for
Haiku	**Fastest, most efficient** (token use). Good with structured tasks.	**Classification, extraction, formatting, straightforward summarization, high-volume routine work** where speed matters and **cost of an imperfect output is low**. Speed advantage **compounds** across hundreds of items. **Examples:** Tagging incoming BFSI customer support emails by category (loan inquiry, KYC update, complaint) across thousands of messages a day; extracting policy numbers and premium amounts from insurance claim forms.
Sonnet	**Balanced** tier; strong quality across task types.	**Drafting, synthesis, analysis, research assistance, document review. Right starting point for most knowledge work.** **Examples:** Drafting a first-pass credit memo from a borrower's financial statements; reviewing a vendor contract against your standard clauses before it goes to legal.
Opus	**Higher capability** than Sonnet and Haiku; slower.	**Nuanced judgment, complex multi-step reasoning, ambiguous inputs, quality over speed.** Client-facing deliverables, complex document analysis, strategic planning, high-stakes multi-source synthesis. **Examples:** Analyzing a regulatory change such as an RBI circular against your lending products and recommending a compliance plan; synthesizing conflicting data from audit reports, market research, and internal risk models into a board-ready strategy paper.

Movement rules

- Start with **Sonnet**.
- Quality falling short on a complex task → **upgrade to Opus**.
- Speed/volume is the main need and task is structured → **consider Haiku**.

Decision logic (memorise)

Task profile	Model
Routine, structured extraction/classification at volume	**Haiku**
Most professional drafting, synthesis, analysis	**Sonnet**

Task profile	Model
Complex judgment, high-stakes output, ambiguous or multi-layered inputs	**Opus**

Usage and cost

- Opus = better on complex tasks, **slower**. Reserve it for work where the **quality ceiling genuinely matters**.
- On **metered / usage-budgeted plans (including API)**, speed vs capability is also a **per-call cost** trade-off: Opus uses **more usage per call** than Haiku or Sonnet. Efficiency = cost + speed.
- On the **standard claude.ai subscription**, treat **efficiency (speed + usage headroom)** as the practical proxy for cost. Selection logic **does not change**.
- Model picker lineup, default model, and automatic model-switching **vary by plan and change over time**. Decision logic applies whatever tiers you have.

About Fable (know it exists, don't over-study)

- Introduced June 2026, **above Opus** (Mythos-class), priced **above Opus**.
- For **long-horizon** work over hours and many dependent steps: large-scale coding/refactoring, autonomous agentic workflows, multimodal research. Safeguards on biology, cybersecurity, AI research topics may limit some requests.
- "If Opus already meets your quality bar, stay with Opus."
- The lesson states the certification **pins the three-tier Haiku/Sonnet/Opus frame** and marks the fourth tier as **not exam-relevant**. Answer exam questions using **Haiku/Sonnet/Opus**.

---

6. Context Management

> Model and capabilities decide **what** Claude can do. Context management decides **how long** it can do it well.

6.1 The Context Window in Practice

- Every conversation has a **finite working-memory budget** that runs down as it grows (messages + uploaded documents).
- Near the limit, claude.ai **auto-summarizes earlier messages** (**only on paid plans with Code Execution enabled**); full history stays available for reference.
- Effect: detailed instructions given in minute 10 may **not be followed at minute 90**. Not because Claude ignores them, but because **detail compresses in the summary**.

6.2 Warning Signs a Conversation Needs Intervention

1. Claude **stops following instructions** it followed earlier in the same session.
2. Responses address **only the latest exchange**, ignoring earlier decisions.
3. **Accuracy drops** in ways consistent with missing early context.

6.3 Three Responses When Context Degrades (Restart · Summarize · Persist)

Response	How	When
Restart	Start a **new conversation**. **Project instructions + knowledge base carry forward automatically**; the **conversation thread does not**.	Session has **drifted beyond recovery**, or you're starting a **genuinely new task** in the same workstream.
Summarize	Before restarting, ask Claude for a **summary of current state: decisions made, work in progress, open questions**. Paste it at the start of the new chat.	You need **thread continuity** without carrying a **degraded context** forward.
Persist	Save to **Memory** or update the **Project knowledge base**.	Information needed in **all future sessions**. Saving at the right moment beats re-entering repeatedly.

- A well-kept Project with a **current knowledge base** reduces the impact of any single session's context limit.

6.4 Usage Limits

- Work on **more than one time window**:
- a **short rolling session window**, plus
- **weekly limits** on paid plans **across models**, with a **separate weekly pool for the highest tier**.
- Exact windows/allowances **vary by plan and change**; check the Help Center.
- Long sessions on **higher-tier models** may hit a limit before work is done.
- **Plan ahead** rather than work around an interruption:
- **Break large tasks into segments**
- **Save interim progress to the knowledge base**
- **Restart from a summary** rather than stretching one session indefinitely

---

Most Likely Questions

Q1. You get two different answers when you ask Claude the same question twice. What is the best explanation?

- A. The first answer was a hallucination.
- B. Claude's outputs are probability-based, so variation is expected.
- C. Your Memory settings changed between the two questions.
- D. The model was switched automatically.

Answer: B
Why: Responses vary because generation is probabilistic, not deterministic. Neither answer is "the one correct response". A assumes one must be wrong. C and D are possible in theory but are not the general explanation the lesson gives.

Q2. Which statement about Claude's tone is correct?

- A. A confident answer is more likely to be accurate.
- B. Claude adds hedging words when it is unsure, so hedged answers need checking.
- C. Claude writes with the same fluency whether it is right or wrong.
- D. Fabricated statistics usually read less fluently than verified ones.

Answer: C
Why: Confident tone is not a signal of accuracy. A fabricated statistic reads with the same assurance as a real one. A and D contradict this. B implies only hedged answers need checking, which is wrong.

Q3. A marketing lead pastes the same three-paragraph brand background into a new Chat every week before drafting campaign copy. What should they do?

- A. Keep using Chat but switch to Opus.
- B. Move the work into a Project with the background as standing instructions or knowledge.
- C. Use Research mode so Claude can find the brand background online.
- D. Use Incognito mode to avoid cluttering history.

Answer: B
Why: Re-pasting the same background is the signal a workstream has outgrown Chat. A Project sets context once so every chat starts with it. A changes the model, not the re-setup problem. C is for multi-source investigation. D removes history and Memory, making it worse.

Q4. Which THREE items does a Claude Project hold? (Select 3)

- A. Standing instructions
- B. Knowledge base
- C. Its own conversation history
- D. A shared context window across all its chats
- E. A dedicated model that can't be changed

Answer: A, B, C
Why: A Project holds standing instructions, a knowledge base, and its own conversation list. D is a classic trap: chats inside a Project share instructions and knowledge but do NOT share context with each other. E is not described.

Q5. In a Project, you made key decisions in Chat 1. You open Chat 2 in the same Project. What does Claude have access to in Chat 2?

- A. Everything from Chat 1, because chats in a Project share context.
- B. The Project's standing instructions and knowledge base, but not Chat 1's conversation.
- C. Nothing, since each chat starts blank.
- D. Only Chat 1's conversation, not the knowledge base.

Answer: B
Why: Project chats share instructions and the knowledge base but not context with each other. To carry Chat 1's decisions, summarize them or persist them to the knowledge base or Memory.

Q6. You need a formatted weekly status report that your director will open and read. Which entry point fits best?

- A. Chat inline reply
- B. Artifact
- C. Research
- D. Incognito chat

Answer: B
Why: Artifacts are for deliverables the recipient will open and read. They appear as a separate, editable block. Inline replies (A) are for answers you act on within the chat. C is for investigation. D is a privacy mode, not an output format.

Q7. An operations analyst needs a quick check of a currency's current exchange rate before a call. Which is the most efficient choice?

- A. Research mode
- B. Web search toggled on in Chat
- C. Upload last year's report to a Project
- D. Ask Claude without any tools, since it knows recent rates

Answer: B
Why: Quick current-information lookups belong in Chat with web search. Research (A) is for deep multi-source synthesis and is more than needed. C gives stale data. D fails because of the training cutoff.

Q8. A strategy team needs a synthesized view of competitor moves across dozens of current news sources, filings, and analyst notes. Which entry point should they choose?

- A. Chat without web search
- B. Artifact
- C. Research
- D. Project with last quarter's documents only

Answer: C
Why: Research runs multi-step searches across many sources and synthesizes current information. A has no current data. B is an output format, not an investigation method. D lacks current information.

Q9. On a Team plan, a member cannot find the web search toggle in their chats. What is the most likely reason?

- A. Web search only exists in Research mode.
- B. An Owner or Primary Owner has not enabled web search for the workspace.

- C. Web search is available only on Enterprise.
- D. The member must turn on "Generate memory from chats".

Answer: B
Why: On Team and Enterprise plans, an Owner or Primary Owner must enable web search in organization capability settings before members can switch it on. A is wrong: web search is a per-chat toggle. C is wrong: it's on all plans. D is unrelated.

Q10. Match the need to the capability. A finance associate needs loan-portfolio totals that must be exactly right, not just plausible. Which capability fits?

- A. Memory
- B. Skills
- C. Code Execution
- D. Standing instructions

Answer: C
Why: Code Execution verifies computations when the result must be correct, not merely plausible. Memory carries continuity, Skills define procedures, and standing instructions carry context. None of these checks arithmetic.

Q11. Which pairing of layer and purpose is INCORRECT?

- A. Projects: carry context
- B. Skills: define repeatable procedures
- C. Memory: verify computations
- D. Code Execution: verify computations

Answer: C
Why: Memory persists continuity across sessions. It does not verify computations; Code Execution does. A, B, D are correct pairings.

Q12. A team has a well-built Skill that formats their monthly compliance summary. A manager proposes dropping the human review step because "the Skill makes it the same every time". What is the best response?

- A. Agree; Skills produce identical output.
- B. Disagree; Skills reduce variance but don't eliminate it, so review should stay.
- C. Agree, but only if the team uses Opus.
- D. Disagree; Skills only work in Chat, not Projects.

Answer: B
Why: Configured procedures still produce varied output. Review stays in the workflow however well the Skill is built. A is the trap. C: no model removes variation. D is made up.

Q13. In Settings → Memory, what does "Search and reference chats" do when ON?

- A. Builds a profile of your role and preferences.
- B. Lets Claude retrieve content from specific past chats when you refer to earlier work.
- C. Shares your chats with your organization's admins.
- D. Keeps chats out of history.

Answer: B
Why: It retrieves content from old chats on request and does not build a profile. Building a profile (role, preferences) is what "Generate memory from chats" does (A). C and D are not what this setting does; D describes Incognito.

Q14. A consultant works for two competing banks and worries context from Bank A could appear in Bank B work. What is the best setup?

- A. One Project for both clients with a note to keep them separate.
- B. Separate Projects for each client, since Project memory is scoped per Project.
- C. Turn off all Memory and re-paste context each time.
- D. Use Incognito inside a shared Project.

Answer: B
Why: Project-scoped Memory keeps contexts separate, and there is no extra setting to turn on. A relies on an instruction instead of a structural boundary. C throws away the productivity gain. D is wrong because Incognito applies to standalone chats outside Projects.

Q15. Which TWO statements about Incognito mode are true? (Select 2)

- A. It keeps the session out of Memory and chat history.
- B. It overrides your organization's data retention policy.
- C. It applies to standalone chats outside Projects.
- D. It makes sharing regulated personal data acceptable.

Answer: A, C
Why: Incognito keeps a chat out of Memory and history and applies to standalone chats. It does not override org data retention (B). It doesn't change what data policy allows (D).

Q16. A user on an Enterprise plan wants to bring facts from another AI assistant's memory into Claude, but automatic import is not available. What is the documented fallback?

- A. Upload the facts to a Project's knowledge base.
- B. Add the key facts to Memory manually through memory settings.
- C. Paste them into every new chat.
- D. Ask the admin to enable Research.

Answer: B
Why: The documented fallback is adding key facts to Memory manually, not routing them into a Project knowledge base (A). Note that import is documented for Free, Pro, Max, Team, not Enterprise. C is inefficient. D is unrelated.

Q17. A support team must tag 5,000 incoming customer emails into eight fixed categories each day. A wrong tag is easy to fix later. Which model fits best?

- A. Haiku
- B. Sonnet
- C. Opus
- D. Use the most capable model available for every email

Answer: A
Why: Haiku suits structured, high-volume classification where speed matters and the cost of an imperfect output is low. Its speed advantage compounds at volume. C and D over-spend on capability and speed. B works but isn't the best fit for routine volume.

Q18. A project manager is unsure which model to use for drafting a stakeholder update and summarizing meeting notes. What does the lesson recommend?

- A. Start with Haiku and upgrade if needed.
- B. Start with Sonnet; move to Opus only if quality falls short on complex work.
- C. Always use Opus for anything external.
- D. Pick the model with the lowest usage limit.

Answer: B
Why: Sonnet is the right starting point for most knowledge work (drafting, synthesis, analysis). Upgrade to Opus if quality falls short. A suits structured volume tasks. C wastes capacity on routine work. D makes no sense.

Q19. Which tasks are typical Opus candidates? (Select 2)

- A. Board-level strategic plan built from several conflicting reports
- B. Reformatting 300 addresses into one standard layout
- C. Analysis of an ambiguous, multi-party contract for a client deliverable
- D. Extracting invoice numbers from a batch of PDFs

Answer: A, C
Why: Opus fits nuanced judgment, complex multi-step reasoning, ambiguous inputs, and high-stakes client-facing work. B and D are structured, high-volume formatting or extraction: Haiku territory.

Q20. On a metered or usage-budgeted plan (including API access), what changes about model selection?

- A. Nothing; cost is identical across tiers.

- B. The speed vs capability trade-off also becomes a per-call cost trade-off, since Opus consumes more usage per call.
- C. Only Haiku is allowed.
- D. The decision logic reverses: use Opus for volume work.

Answer: B
Why: On metered plans, efficiency means cost as well as speed, and Opus uses more per call. The selection logic itself does not change (D is wrong). A and C are false.

Q21. Ninety minutes into a long session, Claude stops applying a formatting rule you set in the first ten minutes. What is the most likely cause?

- A. Claude is deliberately ignoring the instruction.
- B. Earlier context was summarized as the window filled, so detail was compressed.
- C. The model was downgraded to Haiku.
- D. Memory was turned off.

Answer: B
Why: As the context window fills, claude.ai summarizes earlier messages (paid plans with Code Execution enabled), and detail compresses. The lesson says plainly this is not Claude ignoring you (A). C and D aren't indicated.

Q22. A session has drifted badly, but you need the decisions and open questions from it in the next session. What is the best sequence?

- A. Keep going in the same chat and repeat instructions louder.
- B. Ask Claude for a summary of decisions, work in progress, and open questions; start a new chat and paste it in.
- C. Start a new chat with no context.
- D. Switch to Opus in the same chat.

Answer: B
Why: Summarize then restart keeps thread continuity without dragging a degraded context forward. A keeps the degraded window. C loses continuity. D doesn't fix context degradation.

Q23. Which actions match "Persist" in context management? (Select 2)

- A. Save a stable fact to Memory
- B. Update the Project knowledge base with the latest policy document
- C. Open a new chat
- D. Ask Claude to summarize the current chat

Answer: A, B
Why: Persist means saving information needed in all future sessions to Memory or the Project knowledge base. C is Restart. D is Summarize.

Q24. An analyst running long Opus sessions keeps hitting usage limits mid-task. What is the recommended approach? (Select 3)

- A. Break the large task into segments
- B. Save interim progress to the knowledge base
- C. Restart from a summary rather than extending one session indefinitely
- D. Keep one session running as long as possible to save context

Answer: A, B, C
Why: For intensive tasks, plan ahead: segment, save interim work, restart from a summary. D is the opposite of the guidance and increases the risk of both limits and context degradation.

Q25. It's been four months since a user last reviewed their stored Memory. Their role changed last month. What is the main risk, and the fix?

- A. No risk; more memories always help.
- B. Stale entries can actively mislead; review at least monthly and delete or update outdated entries.
- C. Memory auto-expires after 30 days, so nothing is needed.
- D. Move all memories into a Project knowledge base.

Answer: B
Why: Memory is most useful when curated. Out-of-date entries can mislead, and accuracy matters more than volume. A contradicts this. C is invented. D isn't the recommended fix.

Key Takeaways

1. **Decide before you prompt.** Entry point → capability features → model → context management. These set the quality ceiling.
2. **Five Gen-AI facts never change:** answers vary, confidence ≠ accuracy, context is a budget, knowledge has a cutoff, and even Skills produce varied output. **Review is always part of the workflow.**
3. **Entry points:** Chat = one-off. Project = recurring work with stable context (instructions + knowledge base + own chats; chats don't share context with each other). Artifact = deliverable. Research = deep multi-source synthesis; quick lookups use **web search in Chat**.
4. **Four layers, four jobs:** Projects carry **context**, Skills define **procedures**, Code Execution **verifies computation**, Memory persists **continuity**.
5. **Memory hygiene:** know the two toggles (search past chats vs generate memory), Project memory is automatic and separate per Project, Incognito keeps chats out of Memory and history but doesn't override org retention, and review memory **monthly**.
6. **Models:** Haiku = fast, structured, high volume, low cost of error. **Sonnet = default.** Opus = complex, ambiguous, high stakes. On metered plans, higher tier = higher cost per call.
7. **Context:** watch for drift signs, then **Restart, Summarize, or Persist**. For usage limits, segment work, save interim progress, restart from a summary.

02 · Prompting and Task Execution (Revision Notes)

Where This Lesson Sits in the Exam

Domain	Weight	Objectives covered here
1 · Prompting and Task Execution	**14%**	Create effective prompts · Apply task decomposition · Iterate prompts to improve output · Adapt strategy by task type (analysis, research, drafting, brainstorming)

Overlap: diagnosing weak prompts also shows up in **Domain 7 (Troubleshooting & Optimization, 10%)**, and research/citation advice links to **Domain 2 (Output Evaluation, 21%)**.

Core idea: the same request phrased two ways gives two levels of quality. The model doesn't get smarter between them; **the prompt does**. Prompting is a **learnable communication discipline**, not a knack.

Memory Map

```
PROMPTING AND TASK EXECUTION
│
├── DESCRIPTION (AI Fluency Framework competency)
│     = telling Claude precisely what you want; making every component explicit
│     Claude can't see inbox / org chart / last meeting unless a Connector gives access
│     (and even then only what you allowed) → anything else = CONTEXT GAP
│     Context gaps = #1 reason prompts underperform for new users
│
├── 5 COMPONENTS (mnemonic C·T·R·O·C, "See-Troc")
│     ├── Role ........ who Claude is → vocabulary, depth, assumptions
│     ├── Context ..... what Claude can't know → MOST OFTEN OMITTED
│     ├── Task ........ ONE primary verb, unambiguous (Summarize/Compare/Draft/Identify)
│     ├── Constraints . length, tone, include/exclude/avoid → usable without heavy edits
│     └── Output fmt .. table / bullets / 3-para memo / email → saves an iteration
│     Quick question = task (+ maybe constraint) · Client deliverable = ALL FIVE
│
├── DECOMPOSITION (big multi-stage tasks)
│     one prompt → shallow work on every stage, hidden reasoning
│     split into discrete, ORDERED steps, run in sequence
│     ├── Vendor eval: Criteria → Score → Trade-offs → Recommend
│     ├── Each step = checkable intermediate result; work becomes AUDITABLE
│     ├── Same chat: steps that build on each other
│     ├── New chat: step is independent OR context degrading (link to Lesson 01)
│     └── Parallel case: shared extraction FIRST → VERIFY → then parallel drafts
│
├── ITERATION (diagnose, don't rewrite)
│     ├── Generic/off-base ........ thin CONTEXT → add background
│     ├── Wrong question answered . ambiguous TASK verb → sharpen
│     ├── Wrong length/tone/shape . missing CONSTRAINT/FORMAT → add it
│     ├── Close, one section off .. iterate THAT section only, keep the draft
│     ├── Change ONE component, resend, compare
│     └── STOP when rounds give marginal change → quick manual edit instead
│
└── STRATEGY BY TASK TYPE (control vs latitude dial)
      ├── Analysis ...... TIGHT: criteria, standards, scope · loose: phrasing
      ├── Research ...... TIGHT: question, sources, citations · loose: synthesis approach
      │                   web search (quick) vs Research (deep, paid); verify citations
      ├── Drafting ...... TIGHT: audience, tone, format · loose: word choice (medium
```

```
latitude)
      └── Brainstorming . TIGHT: goal + guardrails only · loose: quantity, direction
                          over-specifying kills divergence; volume first, narrow later
```

---

1. Description: The Backbone Competency

- **Description** is a competency in the **AI Fluency Framework**: telling Claude **precisely** what you want.
- It is the habit of making each prompt component **explicit** instead of assuming Claude will infer it.
- Claude **cannot see** your inbox, org chart, or last week's meeting **unless you connect a source through a Connector**.
- Even with a connector, Claude sees **only what you allowed it to access**.
- Anything that lives **only in your head** or **outside a connected source** is a **context gap**.
- **Context gaps are the single most common reason** a prompt underperforms for new users.

---

2. Anatomy of an Effective Prompt

Most weak prompts are **missing one or more components**. Naming them turns prompting from guesswork into a **checklist** to run before any non-trivial request.

2.1 The Five Components

#	Component	What it is	Why it matters	Key note
1	**Role**	Who Claude should be (financial analyst, editor, policy reviewer)	Sets **vocabulary, depth, assumptions**	
2	**Context**	Background Claude can't know unless told: **audience, situation, prior decisions, source material**	Grounds the output	**Most often omitted**
3	**Task**	The specific action as a clear instruction	Tells Claude what to do	**One primary verb**, unambiguous (Summarize, Compare, Draft, Identify)
4	**Constraints**	Boundaries: **length, tone, what to include, what to leave out, what to avoid**	Keeps output **usable without heavy editing**	
5	**Output format**	Shape of the result: table, bullets, 3-para memo, draft email	Stating it **up front saves an iteration**	

Mnemonic: C·T·R·O·C ("See-Troc") = **C**ontext · **T**ask · **R**ole · **O**utput format · **C**onstraints

2.2 How Many Components?

- **Not every prompt needs all five.**

- Quick question → **task** and maybe a **constraint**.
- **Client deliverable → all five.**
- The skill is **noticing which components a given task requires**.

2.3 Diagnosing a Weak Prompt

Hold a disappointing prompt against the stack; the gap is usually obvious.

Missing / weak	Result
Context	**Generic** output
Ambiguous task verb	**Wrong action**
No constraints	**Wrong length or tone**

The component stack doubles as a **diagnostic checklist** (used again in troubleshooting).

2.4 Worked Example

Weak: "Write a summary of our quarterly operations."

- Result: three plausible paragraphs that could describe almost any company.
- No audience, no figures, no format, no sense of priority.
- Output is **not wrong; it is unusable**, because the prompt specified almost nothing.

Strong (components labelled):

> "You are an operations analyst **(role)**. I am preparing a one-page update for our regional director, who cares about throughput and cost, not process detail **(context and audience)**. Summarize the attached Q3 operations data **(task)**, covering only the three metrics that moved more than 10 percent against target **(constraint)**. Format as a short headline followed by three bullet points, each one sentence **(output format)**."

- Same model, same data. Draft can be refined in **two minutes** instead of rebuilt.
- The difference is **entirely in the specification**.

Habit to build: before any prompt that matters, run the five components in your head. **Thirty seconds of specification saves several rounds of correction.**

---

3. Task Decomposition for Complex Requests

3.1 What and Why

- When a task has **several distinct stages**, one prompt produces **shallow work on every stage**.
- **Decomposition** = splitting a multi-part problem into **discrete, ordered steps**, then **running them in sequence** instead of asking for everything at once.

3.2 Vendor Evaluation Example ("four tasks wearing one sentence")

Single prompt (underperforms): "Evaluate these three vendors and tell me which to pick."

- Claude must **invent criteria, apply them, weigh trade-offs, recommend** in one pass.
- All four done **shallowly**; you **can't see the reasoning**.

Decomposed version:

Step	Action
1 · **Derive criteria**	From the requirements document, derive the criteria that matter and **weight** them
2 · **Score vendors**	Score each vendor against those criteria using supplied materials
3 · **Raise trade-offs**	Highlight where vendors **diverge most**
4 · **Recommend**	Recommend, with reasoning **tied back to the weighted criteria**

Benefits

- Each step gives a **checkable intermediate result**.
- Wrong criteria in Step 1 are caught **before scoring**, not after the recommendation.
- Work becomes **auditable** (matters when someone asks how the recommendation was reached).

3.3 One Conversation or Several?

Situation	Choice
Sequential steps that **build on each other**	**Same conversation**, so each step sees prior results
A step that is **genuinely independent**	**Separate conversation**
Conversation so long that **early context is degrading**	**Separate conversation** (context management from Lesson 01)

3.4 Parallel Case: Three Deliverables, One Foundation

Scenario: turn a dense **20-page policy change** into (1) an internal announcement, (2) a staff FAQ, (3) a short executive briefing.

Step	Action
1	**Extract** the substantive changes and what each means in practice
2	**Confirm** the extraction is complete and accurate
3	Draft the **staff announcement** (general audience) from the confirmed list
4	Draft the **FAQ**, anticipating staff questions
5	Draft the **executive briefing**, compressed to **decisions and impact**

Why this order

- Steps 1–2 build a **verified foundation** all three deliverables use.

- Drafting before the change list is confirmed risks **propagating the same misreading into three documents**.
- Rule: **sequence the shared, high-stakes extraction first; let the parallel drafts follow.**

---

4. Iterating Prompts to Improve Output

A first draft rarely lands perfectly. The skill is **not rewriting the whole prompt**; it is **reading the output to diagnose which component fell short**, then **fixing that one thing**.

4.1 Symptom → Cause → Fix (memorise)

Symptom	Likely cause	Fix
Generic or off-base	**Thin context**	Add background Claude couldn't infer
Answered the wrong question	**Ambiguous task verb**	Sharpen the instruction
Wrong length, tone, or shape	**Missing constraint or format**	Add it
Close, but one section misses	(Draft is mostly right)	**Iterate on that section only**; don't discard the draft

4.2 Targeted Revision, Not Wholesale Rewriting

- Rewriting everything **loses the parts that worked**.
- You also **can't tell which change fixed the problem**.
- Method: **change the one component the output told you to change → resend → compare.**
- Same discipline applies to troubleshooting any underperforming workflow (Domain 7).

4.3 Live Iteration Cycle (Delayed-Deliverable Email)

Round	Prompt	Output	Diagnosis
1	"Write a follow-up email to the client about the delayed deliverable."	Generic, slightly defensive, 3 paragraphs; no new date, no cause	**Thin context** (no reason, no date) + **no tone constraint**
2	Adds: two-day delay on analytics deliverable; cause = data-quality issue now fixed; new date Thursday; tone accountable, not over-apologetic; under 120 words	Tight, accountable, cause + date + confident close	Strong; only **subject line missing**
3	"Good. Add a subject line that signals resolution, not just delay."	Marginal improvement	**Signal to stop**

- Each round changed **exactly** the component the previous output exposed.

- **No round threw away working text.**

4.4 Knowing When to Stop

- Iteration has **converged** when extra rounds give **marginal change, not improvement**.
- Then a **quick manual edit** beats more prompting.
- Goal = **a usable result, not a perfect prompt.** Recognising diminishing returns is part of the skill.

---

5. Adapting Strategy by Task Type

The component stack applies to **every** prompt; the **emphasis** shifts. One fixed style across all task types **costs quality** on every task it doesn't fit.

5.1 The Four Task Types

Task type	Approach	Latitude
Analysis	**Tight constraints, explicit criteria.** Say what to measure, against what standard, how to handle ambiguity.	**Low** latitude, high specification
Research	**Clear scope + source discipline.** Define question, boundaries, whether current sources are needed. **Ask for citations** so claims are checkable. Quick currency → **web search in Chat**; deep multi-source → **Research** (paid plans).	Scoped
Drafting	Specify **audience, tone, format**; leave room for phrasing. You control the shape, Claude fills it.	**Medium** latitude
Brainstorming	**Loose constraints, high latitude.** Give goal + boundaries, ask for **volume and range** before narrowing. **Over-specifying kills divergence.**	**High** latitude

5.2 Quick Reference (memorise)

Task type	Tighten	Loosen
Analysis	Criteria, standards, scope	Phrasing
Research	Question, sources, citations	Synthesis approach
Drafting	Audience, tone, format	Word choice
Brainstorming	Goal and guardrails only	Quantity and direction

5.3 Mini-Demos

- **Analysis:** "Compare these two vendor contracts on payment terms, termination rights, and liability caps. For each, state which is more favorable to us and why, in a three-row table." → tight criteria, defined output.

- **Research:** "Using current sources, summarize how three named competitors positioned their Q2 launches. Cite each source. Flag anything you cannot verify." → scope + citation discipline; Research (or web search) supplies currency.
- **Drafting:** "Draft a 150-word LinkedIn post announcing our new reporting feature, aimed at operations managers, in a confident but not salesy voice." → audience, length, tone fixed; phrasing open.
- **Brainstorming:** "Give me 20 angles for a campaign around faster month-end close. Range widely; do not self-edit yet." → goal + one guardrail; constraints come **after** the range exists.

5.4 Citations Warning (exam favourite)

- Citations are **checkable** when they come from a **grounded source** (web search or Research results).
- Citations from **training memory alone** can look **equally confident** and **must be independently verified**.

Underlying move every time: decide **where you need control** and **where you need range**, then set constraints accordingly. This separates a competent prompter from one who gets the same mediocre output on every task.

---

6. Repairing an Underperforming Prompt (Case Study)

Weak prompt	"Summarize the customer feedback and tell me what to do."
Output	Generic 5-bullet themes ("customers want faster support") + vague advice; not tied to data, no priorities, not actionable
Real goal	PM has **200 survey responses**; needs **top 3 issues by frequency**, each with a **representative quote**, **ranked**, to decide what to fix this quarter
Diagnosis	Missing **every** component: role, context, specific task, constraints, format

Repaired prompt:

> "You are a product analyst **(role)**. Attached are 200 customer survey responses **(context)**. Identify the three most frequently raised issues **(task)**, ranked by how many responses mention each, and for each issue include one representative verbatim quote and the approximate share of responses it appears in **(constraints)**. **Use code execution to count accurately rather than estimating.** Format as a ranked list, most frequent first **(output format)**."

What improved

- Role + context → analysis **grounded in the actual responses**.
- Vague task → **specific, ranked, countable** instruction.
- Constraints (quote, share of responses) → **actionable** output.
- **Code execution** → frequency counts **verified, not guessed** (links to Lesson 01 capability layer).
- Output format → **decision-ready** result.

Most Likely Questions

Q1. Which prompt component is most often omitted by users?

- A. Role
- B. Context
- C. Task
- D. Output format

Answer: B
Why: Context (audience, situation, prior decisions, source material) is named as the component most often left out. Context gaps are the single most common reason prompts underperform for new users. The others matter but are not singled out this way.

Q2. What does the "Role" component mainly set?

- A. The length of the response
- B. The vocabulary, depth, and assumptions Claude brings
- C. The file format of the output
- D. Which model Claude uses

Answer: B
Why: Role (financial analyst, editor, policy reviewer) shapes vocabulary, depth and assumptions. Length is a constraint (A). File shape is output format (C). Role has nothing to do with model choice (D).

Q3. A team member asks, "Can Claude just check what we agreed in last Tuesday's meeting?" No connector is set up and the notes were never shared. What is the issue?

- A. Claude needs a higher-tier model to recall meetings.
- B. This is a context gap; Claude can only see what you provide or allow through a connector.
- C. Claude can infer the agreement from general knowledge.
- D. Turning on Research will find the meeting notes.

Answer: B
Why: Claude can't see your inbox, org chart or meetings unless you provide the content or connect a source, and even then only what you allowed. A and C are wrong because no model can see private, unshared information. D searches public sources, not your private meetings.

Q4. Which statement best describes how many prompt components a request needs?

- A. Every prompt must contain all five components.
- B. Only client deliverables need a task verb.
- C. A quick question may need only a task and perhaps a constraint, while a client deliverable needs all five.
- D. Role is the only required component.

Answer: C
Why: The skill is noticing which components a task requires. A is the trap; not every prompt needs all five. B and D are false.

Q5. Claude answers a different question from the one you intended. Which component most likely failed?

- A. Context was thin
- B. The task verb was ambiguous
- C. Output format was missing
- D. Role was missing

Answer: B
Why: "Answered the wrong question" points to an ambiguous task verb; the fix is to sharpen the instruction. Thin context (A) gives generic output. Missing format or constraints (C) gives the wrong shape, length or tone.

Q6. A draft from Claude is generic and could describe any company. What is the best first fix?

- A. Switch to a more capable model.
- B. Add the background Claude could not infer: audience, figures, situation.
- C. Rewrite the whole prompt from scratch.
- D. Ask Claude to try again with the same prompt.

Answer: B
Why: Generic or off-base output points to thin context. Add the missing background. A doesn't fix missing information. C loses what worked and hides which change helped. D just produces another variation of the same gap.

Q7. A one-page memo is right except the risk section, which is too shallow. What should you do?

- A. Discard the draft and write a fully new prompt.
- B. Iterate on the risk section only and keep the rest of the draft.
- C. Switch to brainstorming mode.
- D. Start a new conversation with the original prompt.

Answer: B
Why: When output is close but misses on one section, iterate on that section only. Don't discard a draft that is mostly right. A and D throw away working text. C is the wrong strategy for the task.

Q8. Why is targeted revision preferred over rewriting the entire prompt? (Select 2)

- A. Rewriting loses the parts that already worked.
- B. Rewriting makes it impossible to tell which change fixed the problem.
- C. Claude penalizes long prompts.
- D. Rewriting always produces lower-quality output.

Answer: A, B
Why: These are the two reasons given. Change one component, resend, compare. C is invented. D is too absolute; the problem is diagnostic, not guaranteed lower quality.

Q9. After three rounds of iteration, each new round makes only tiny changes. What should you do?

- A. Keep iterating until the prompt is perfect.
- B. Recognise convergence and make a quick manual edit instead.
- C. Switch models and start over.
- D. Add all five components again.

Answer: B
Why: Iteration has converged when rounds give marginal change. A quick manual edit beats more prompting. The goal is a usable result, not a perfect prompt.

Q10. A manager prompts: "Evaluate these three vendors and tell me which to pick." What is the main weakness?

- A. The prompt is too short to process.
- B. Claude must invent criteria, apply them, weigh trade-offs and recommend in one pass, so all four are shallow and the reasoning is hidden.
- C. Vendor evaluation needs Research mode.
- D. Claude can't compare more than two vendors.

Answer: B
Why: This is "four tasks wearing one sentence". Decomposition fixes it. A, C, D are not reasons given in the lesson.

Q11. Put the decomposed vendor-evaluation steps in the correct order.

- A. Score vendors → Derive criteria → Recommend → Raise trade-offs
- B. Derive criteria → Score vendors → Raise trade-offs → Recommend
- C. Raise trade-offs → Derive criteria → Score vendors → Recommend
- D. Recommend → Derive criteria → Score vendors → Raise trade-offs

Answer: B
Why: Criteria (weighted, from the requirements doc) come first, then scoring against them, then trade-offs where vendors diverge, then a recommendation tied back to the weighted criteria. Scoring before criteria (A) has no standard to score against.

Q12. Which are benefits of task decomposition? (Select 3)

- A. Each step produces a checkable intermediate result.
- B. Errors in early steps are caught before later steps build on them.
- C. The work becomes auditable.
- D. It removes the need for human review.

Answer: A, B, C
Why: All three are stated benefits. D is false; decomposition makes review easier, it doesn't remove it.

Q13. You are on step 3 of a 4-step analysis where each step uses the previous step's results. Where should step 4 run?

- A. In a new conversation, to keep things tidy.
- B. In the same conversation, so it sees the prior results, unless early context is degrading.
- C. In Incognito mode.
- D. In a separate Project.

Answer: B
Why: Keep sequential, dependent steps in one conversation. Move to a new conversation only when a step is genuinely independent or context is degrading. A breaks the dependency. C and D are irrelevant.

Q14. A communications manager must turn a 20-page policy change into a staff announcement, an FAQ, and an executive briefing. What should happen FIRST?

- A. Draft the executive briefing, since it's the highest priority.
- B. Draft all three deliverables at once to save time.
- C. Extract the substantive changes and confirm the extraction is complete and accurate.
- D. Draft the FAQ, since staff questions drive everything.

Answer: C
Why: Sequence the shared, high-stakes extraction first and verify it. Drafting before the change list is confirmed risks spreading the same misreading into three documents.

Q15. Which task type calls for loose constraints and high latitude?

- A. Analysis
- B. Research
- C. Drafting
- D. Brainstorming

Answer: D
Why: Brainstorming wants volume and range; over-specifying kills divergence. Analysis wants tight criteria (low latitude). Research wants scope and source discipline. Drafting is medium latitude.

Q16. For a drafting task, what should you tighten and what should you leave open?

- A. Tighten word choice; leave audience open.
- B. Tighten audience, tone and format; leave word choice open.
- C. Tighten quantity; leave goal open.
- D. Tighten nothing; drafting needs full latitude.

Answer: B
Why: Drafting is medium latitude: you control the shape (audience, tone, format) and Claude finds the phrasing. A reverses it. C fits nothing. D describes brainstorming taken too far.

Q17. A compliance analyst asks Claude to compare two contracts on payment terms, termination rights and liability caps, stating which is more favourable for each, in a three-row table. Which strategy is this, and why does it work?

- A. Brainstorming; it asks for many options.
- B. Analysis; tight criteria and a defined output leave no room to wander.
- C. Drafting; it fixes tone.
- D. Research; it needs current sources.

Answer: B
Why: Analysis wants explicit criteria, standards and scope, with low creative latitude. The prompt names the criteria and the table format.

Q18. Claude gives you three citations for a claim during a normal chat, with web search and Research both off. What should you do?

- A. Trust them; citations prove the claim.
- B. Verify them independently, since citations from training memory can look equally confident.
- C. Ask Claude to rate how confident it is in the citations.
- D. Reformat them into a reference list.

Answer: B
Why: Citations are checkable when they come from a grounded source (web search or Research results). From training memory alone they can look just as confident and must be independently verified. C relies on self-reported confidence. D doesn't address accuracy.

Q19. Which TWO elements belong in a good research prompt? (Select 2)

- A. A clear question and boundaries, including whether current sources are needed
- B. A request for citations and a flag for anything unverifiable
- C. "Range widely; do not self-edit yet"
- D. A fixed word choice for every sentence

Answer: A, B
Why: Research wants clear scope and source discipline, with citations so claims are checkable. C is a brainstorming instruction. D over-controls phrasing and isn't relevant to research.

Q20. A PM's prompt "Summarize the customer feedback and tell me what to do" produced vague themes. The goal is the top three issues from 200 responses, by frequency, each with a quote. Which addition most directly makes the frequency counts trustworthy?

- A. Asking for a confident tone

- B. Instructing Claude to use code execution to count rather than estimate
- C. Adding a role of "senior executive"
- D. Asking for 20 themes instead of 3

Answer: B
Why: Code execution makes counts verified, not guessed. A doesn't affect accuracy. C changes the perspective but not the counting. D goes against the goal.

Q21. In the repaired customer-feedback prompt, which elements are constraints? (Select 2)

- A. "You are a product analyst"
- B. "Include one representative verbatim quote for each issue"
- C. "Include the approximate share of responses each issue appears in"
- D. "Format as a ranked list, most frequent first"

Answer: B, C
Why: The quote and share requirements are constraints that make the output actionable. A is the role. D is the output format.

Q22. In the delayed-deliverable email example, Round 1 produced a generic, slightly defensive email. What did Round 2 add?

- A. A new role only
- B. Context (cause, new date) and constraints (tone, under 120 words)
- C. A request for 20 alternative emails
- D. A switch to Opus

Answer: B
Why: Round 1 was diagnosed as thin context with no tone constraint. Round 2 added the cause (fixed data-quality issue), new date (Thursday), tone (accountable, not over-apologetic) and length (under 120 words). Each round changed exactly what the previous output exposed.

Q23. A colleague uses the same tightly specified style for every request, including brainstorming campaign ideas, and gets narrow, repetitive ideas. What is the best advice?

- A. Add even more constraints to the brainstorm.
- B. Loosen constraints for brainstorming: give the goal and guardrails, ask for volume and range, then narrow.
- C. Switch to Haiku for brainstorming.
- D. Use decomposition to split each idea into four steps.

Answer: B
Why: One fixed style costs quality on every task it doesn't fit. Over-specifying kills divergence. A makes it worse. C and D don't address the latitude problem.

Q24. In the strong Q3 operations prompt, "covering only the three metrics that moved more than 10 percent against target" is which component?

- A. Role
- B. Context
- C. Constraint
- D. Output format

Answer: C
Why: It sets a boundary on what to include. The role is "operations analyst". The context is the regional director who cares about throughput and cost. The format is a headline plus three one-sentence bullets.

Q25. What is the main lesson of the weak vs strong "quarterly operations summary" example?

- A. The stronger prompt used a more capable model.
- B. The weak output was factually wrong.
- C. With the same model and data, the difference in quality came entirely from the specification.
- D. Longer prompts are always better.

Answer: C
Why: "The model did not get smarter. The prompt did." The weak output was not wrong, just unusable (B is false). A is false; it was the same model. D overgeneralises; the point is the right components, not length.

Key Takeaways

1. **The prompt sets the quality, not the model.** Description (AI Fluency Framework) means making every component explicit.
2. **Five components, C·T·R·O·C:** Context, Task (one clear verb), Role, Output format, Constraints. Context is the most often omitted. Client deliverables need all five; quick questions need fewer.
3. **Claude only knows what you give it** or allow through a connector. Context gaps are the top cause of weak output.
4. **Decompose big tasks** into ordered steps with checkable results (criteria → score → trade-offs → recommend). Keep dependent steps in one chat; split when independent or context degrades. For parallel deliverables, **extract and verify the shared foundation first**.
5. **Iterate by diagnosis:** generic = thin context, wrong question = vague verb, wrong shape/tone/length = missing constraint or format, one section off = fix that section only. Change one thing at a time. **Stop when changes become marginal.**
6. **Match strategy to task type:** Analysis tight, Research scoped with citations, Drafting medium, Brainstorming loose. Verify citations that don't come from web search or Research.
7. **Use code execution for counts** so numbers are verified, not estimated.

03 · Evaluating and Validating Claude Output (Revision Notes)

Where This Lesson Sits in the Exam

Domain	Weight	Objectives covered here
2 · Output Evaluation and Validation	**21% (largest domain)**	Evaluate accuracy and completeness · Identify hallucinations, inconsistencies, bias · Apply fact-checking and validation · Decide when human review is required · Edit, adapt, refine, compare outputs for the audience · Organize information and choose output format (artifacts, inline, structured data)

Why it's the biggest domain: accountability. When your name is on a deliverable, you own every claim in it, whether you wrote it or Claude did.

The core asymmetry: time saved is **small and visible**; the cost of an unverified error is **large and arrives later**, paid by someone whose trust you needed.

Opening case: a consultant pulled 5 statistics for a client market-sizing deck. 4 were sound; 1 growth rate was **fabricated** but plausible. The client's own analyst caught it in the room. Ten minutes saved cost a **credibility hit and a week of rebuilding trust**. Nothing about the figure looked wrong on screen.

The twenty-minute rule: Claude saves you twenty minutes drafting an analysis. One fabricated figure in it, sent to a **client or a regulator**, can cost far more than twenty minutes to undo.

Exam Objective → Where It Is Covered

Domain 2 objective (exam guide)	Sections in these notes
Evaluate outputs for **accuracy and completeness**	2.1 to 2.6
Identify **hallucinations, inconsistencies, and biases**	3.1 to 3.4
Apply **fact-checking and validation** techniques	4.1 to 4.3
Determine when **human review or additional verification** is required	2.3, 5.1 to 5.5, 8
Edit, adapt, refine, and compare outputs for the intended audience	6.1 to 6.3
Organize and curate information; select **output formats** (artifacts, inline, structured data)	7.1 to 7.3

Memory Map

```
EVALUATING & VALIDATING CLAUDE OUTPUT
│
├── TWO AI FLUENCY COMPETENCIES
│     ├── DISCERNMENT = HOW you review (evaluate vs requirements, sources, standards)
│     └── DILIGENCE   = WHY you must (decide when to verify; own the result)
│
├── DISCERNMENT PROTOCOL (same check every time)
│     ├── 3 references: REQUIREMENTS · SOURCE MATERIAL · PROFESSIONAL STANDARDS
│     ├── Stakes calibration: domain-dependent
```

```
│      │      zero-tolerance (legal, financial, compliance) → verify every claim
│      │      low-stakes internal brainstorm → lighter review
│      ├── Completeness ≠ accuracy (review both; missing items are harder to see)
│      └── 3-way triage (with documented reasoning)
│            READY TO USE · NEEDS REVISION · NEEDS HUMAN OVERRIDE
│
├── FAILURE PATTERNS
│      ├── Hallucinations
│      │      ├── Plausible-but-unsupported claim (MOST DANGEROUS)
│      │      ├── Fabricated specifics (stats, dates, names, quotes, citations)
│      │      └── Confident tone masking uncertainty
│      ├── Inconsistency: internal contradictions in long outputs → consistency pass
│      ├── Bias: confirmation bias when prompt implies preferred answer
│      ├── Completeness failure: misses the hardest / most important file
│      └── Capability hallucination: "I've emailed / saved it" → unverified
│
├── FACT-CHECKING & GROUNDING (build into the prompt)
│      ├── Prompt for verifiability
│      │      ├── Permit "I don't know"
│      │      ├── Restrict to provided sources ("Not covered by this document")
│      │      └── Require auditable citations (section + clause)
│      └── Grounding techniques
│            ├── Quote first, then analyze
│            ├── Best-of-N: agree → confidence up; diverge → soft spot
│            └── Validate vs AUTHORITATIVE source (not a 2nd Claude answer)
│                  Claude for Excel → cell-level citations
│
├── DILIGENCE: 4 RISK THRESHOLDS
│     STAKES · REVERSIBILITY · AUDIENCE · REGULATORY EXPOSURE
│     Do-not-ship-without-review list (decided in advance):
│       client deliverables · audit-critical/financially material calcs ·
│       regulated/confidential/sensitive data · public or legal comms
│     Iteration vs escalation: flat improvement curve → human expert
│     Accountability never transfers to the tool
│
├── EDITING FOR AUDIENCE ("Claude drafts; you deliver")
│      ├── 3 passes: CLARITY · TONE · FORMATTING
│      ├── Exec = decision + impact first · Team = detail + method + action
│      │   External = control disclosure + framing · facts stay the same
│      └── Compare several drafts (runs or models) → edit the strongest
│
└── OUTPUT FORMAT = RELIABILITY DECISION
       ├── Inline ...... act on within chat
       ├── Artifact .... documents, code; editable, reusable deliverable
       ├── Structured .. tables/schemas for tools or readers
       ├── Code exec ... numbers that must be right; computed + traceable
       │                 (code can still have a bug → read, verify, re-run)
       └── Curate inputs: DE-DUPLICATE · LABEL & STRUCTURE · PRUNE irrelevant

  ROUTINE: stakes → 3 references → completeness → signatures → verify →
           4 thresholds → verdict + reason → edit for audience
```

---

1. Discernment and Diligence

Competency (AI Fluency Framework)	Meaning	Short form
Discernment	Critically evaluating output against **requirements, sources, and standards**	**How** you review

Competency (AI Fluency Framework)	Meaning	Short form
Diligence	Deciding **when verification is required** and **taking responsibility** for the result	**Why** you must

You must be able to: evaluate systematically, recognise failure patterns, build verification into prompts, know thresholds for mandatory human review, and choose the format that matches the reliability needed.

---

2. Discernment: Accuracy, Completeness, Fitness

Evaluation is not a feeling about whether output looks good. It is a check against **three fixed references**, run **the same way every time** so quality doesn't depend on how rushed you are.

2.1 The Three Evaluation References

Reference	Question	How to check
Requirements	Does it reflect what I asked for?	Re-read your request; confirm **each part** is addressed, not just the easy parts
Source material	Does it match the documents I supplied?	**Trace specific claims back to the source**; don't trust that Claude read carefully
Professional standards	Would this pass in my field?	Watch for a **number without units**, a **recommendation without reasoning**, a **citation you can't locate**. These fail even when fluent

2.2 Stakes Calibration

- Review depth depends on stakes, and stakes are **domain-dependent, not universal**.
- **Zero-tolerance work** (legal analysis, financial figures, compliance reporting): **accuracy outranks speed entirely; verify every claim**.
- **Low-stakes internal brainstorming:** a **lighter review** is fine.
- Risk = applying the **same casual review to both**.
- **Decide the stakes first, then the depth of review.**

2.3 The Three-Way Triage (always with documented reasoning)

Verdict	When
Ready to use	Meets requirements, matches sources, clears professional standards. **Ship it.**
Needs revision	Close, but a **specific gap** remains. **Note the gap and iterate.**
Needs human override	The **stakes, errors, or uncertainty** mean it shouldn't go out on Claude's draft alone. **Escalate to a person.**

2.4 Completeness Is a Separate Review

- **Accuracy:** is what's present correct?
- **Completeness:** is anything **missing**?
- They **fail independently**. An output can be **fully accurate yet omit the one decisive factor**.
- **Missing items are harder to spot** than wrong ones, because nothing on screen draws your eye.

2.5 Protocol Applied: Three Outputs, Three Verdicts

- The protocol is **most useful on outputs that are not obviously good or bad**. Obvious failures catch themselves; the borderline ones are where a fixed check earns its keep.

Output	What the check found	Verdict	Action
Competitor-pricing summary from uploaded PDFs	Requirements met, but **source check fails**: "$40/user" vs PDF's "$40/user, **minimum 10 seats**". Omission changes the comparison	**Needs revision**	**Source-restricted re-prompt**, not a rewrite
Internal process recommendation (3 ways to cut invoice-processing time)	Requirements met, no sources to check, standards cleared, **low stakes** (discussion starter)	**Ready to use**	**Over-verifying wastes the time the tool saved**
Compliance-gap analysis (policy vs regulation)	Looks complete, but **regulation was not uploaded**; Claude used **training recall of a rule that may have changed**; **regulatory stakes**	**Needs human override**	A useful **prompt for a compliance expert**, **not a substitute** for one

Lesson: the verdict never depends on **how polished** the output looks; it depends on what the **three references and the stakes** return.

2.6 Sample Triage Exercise (model answers)

Output	Verdict	Reason
A. Internal brainstorm of campaign ideas, labelled as a starting point; a few generic entries among strong ones	**Ready to use (with light editing)**	Low stakes, internal, labelled as a draft. Trim weak entries and proceed
B. Financial summary whose **total doesn't match the sum of line items**	**Needs revision**	**Internal contradiction**. **Recompute with code execution** and confirm before use
C. Market overview citing **three statistics with no checkable source**	**Needs revision**	Unsourced stats are **unverified claims**. Require **auditable citations** or **validate against authoritative sources**
D. Draft **regulatory filing** summary, accurate on spot-checked points, **for submission to a regulator**	**Needs human override**	**High stakes, irreversible, regulatory audience.** Mandatory expert review **however clean it looks**

Note: in D, spot-checks passing does **not** lower the bar.

Reasoning watch-outs for this exercise (the exam marks the reason, not just the label)

- **Output B:** the core problem is an **internal contradiction** (total ≠ sum of lines). "Code execution wasn't used" is the fix, not the diagnosis. State both: *contradiction found → recompute with code execution → confirm*.
- **Output C:** the fix for unsourced statistics is **auditable citations or validation against authoritative sources**. Adding more context to the prompt does not make a statistic checkable.
- **Output A:** "ready to use" still means **light editing** (trim weak entries). Ready ≠ untouched.

---

3. Hallucinations, Inconsistencies, and Bias

Plausible ≠ verified. Claude is fluent whether right or wrong, so **tone and confidence can't flag errors**. Learn the **signatures** so you don't have to read every line with equal suspicion.

3.1 Hallucination Patterns

Pattern	Description	Key point
Plausible-but-unsupported claim	Sounds reasonable, fits the topic, **no basis in source or fact**	**Most dangerous**: nothing looks wrong
Fabricated specifics	Invented **statistics, dates, names, quotations, citations**	**Specificity reads as authority**, so they persuade
Confident tone masking uncertainty	Claude **rarely hedges in proportion** to its real certainty	A guess and a grounded fact sound **equally assured**

3.2 Inconsistencies and Bias

Pattern	Description	Defence
Internal contradictions	Early claim conflicts with a later one in a **long** output	Run a **consistency pass** over the whole document, not just paragraph by paragraph
Confirmation bias in framing	If your prompt **implies a preferred answer**, Claude may lean toward it	Be wary when output **agrees with you too readily** on a question that should be open

3.3 Two Special Failures

Completeness failure (field pattern)

- Asked to compare a batch of documents and find differences. Output listed several and **read as thorough**.
- It **missed differences in the single most important file**.
- Completeness failures **concentrate where attention is lowest**. A confident summary of the easy files can **mask silence on the hard one**.

Capability hallucination

- Claude **claims an action it can't take**: "I've emailed that to your team", "I've saved the file".

- In claude.ai, Claude works with the **conversation, connected tools, and uploaded files**. It **does not perform external actions it wasn't given a tool for**.
- **Treat any claimed external action as unverified until you confirm it happened.**

3.4 Failure-Pattern Gallery

Example	The tell
"Approximately **63%** of mid-market SaaS firms adopted AI tools in 2025, up from **41%** in 2024." No source.	**The precision is the tell.** Real figures this specific come with a citation; an uncited 63% is a **number-shaped guess**
"Is this clause enforceable in our state?" → confident four-sentence "yes"	Enforceability is **jurisdiction-specific and date-sensitive**; it should hedge and doesn't. **Assurance is not evidence**
10-page analysis: page 2 says market "**~$2 billion**", page 8 projects on "**$2.6 billion**"	Each reads fine alone; only visible if you **hold the whole document in view → consistency pass**

How to read the gallery: none of these looks broken. Fluent generation passes a **casual read** by nature. You catch failures by **knowing the signatures and checking against sources**, not by waiting for something to look wrong.

---

4. Fact-Checking and Grounding Techniques

The strongest verification is built into the prompt, before the output exists. These habits cut hallucinations **at the source** and make what remains **easier to audit**.

4.1 Prompt for Verifiability

Technique	Why	Verbatim prompt
Permit "I don't know"	Without permission, a model **under pressure to answer** is more likely to **invent**	"If the answer is not supported by the documents I provided, say so explicitly rather than estimating. It is acceptable to answer 'the provided materials do not cover this.'"
Restrict to provided sources	Turns **open-ended generation into bounded retrieval**; flags gaps	"Answer using only the attached contract. Do not use general knowledge. For anything the contract does not address, list it under 'Not covered by this document.'"
Require auditable citations	**A citation you can't trace is not a citation**	"For every claim, cite the section and clause number it comes from, in parentheses, so I can verify it against the source."

4.2 Grounding Techniques

Technique	How it works
Quote first, then analyze	For long documents, extract the **supporting quotes before conclusions**. Makes **reasoning and errors visible**. Prompt: "Before you analyze, extract the exact sentences from the document that bear on my question. Then base your analysis only on those quotes."

Technique	How it works
Best-of-N comparison	**Re-run the same request and compare.** Where runs **agree, confidence rises**; where they **diverge, you've found soft spots** needing a human look
Validate against authoritative sources	For claims that matter, check a **trusted external reference**, **not a second Claude response**. In-product aid: **Claude for Excel** gives **cell-level citations** tying figures back to inputs

4.3 Verification Checklist (before relying on any output)

1. Did I **allow uncertainty**?
2. Did I **restrict to sources** where appropriate?
3. Did I **require citations I can audit**?
4. Did I **check high-stakes claims against something authoritative**?

Building these into the prompt is **cheaper than rebuilding trust** afterwards.

---

5. Diligence: When Human Review Is Non-Negotiable

Some outputs must **never go out as a Claude draft alone**, however good they look. Know the thresholds **in advance**, so escalation is decided **by policy**, not in the moment after something went wrong.

5.1 Four Risk Thresholds (memorise)

Threshold	Question	Rule
Stakes	What's the cost if wrong?	High-cost errors need human review **regardless of how confident** the output looks
Reversibility	Can it be undone?	**Irreversible** steps (sent client deliverable, filed report) clear a **higher bar** than a revisable draft
Audience	Who sees it?	**External, executive, regulatory** audiences need more review than internal drafts
Regulatory exposure	Does a rule, contract, or law govern it?	Regulated content carries obligations that **AI assistance does not remove**

5.2 Do-Not-Ship-Without-Review List (fixed, decided in advance)

- **Final client deliverables**
- **Audit-critical or financially material calculations**
- Anything with **regulated, confidential, or highly sensitive data**
- **Public or legal communications** where a misstatement has lasting consequences

5.3 Iteration vs Escalation

- Productive iteration **improves output each round**.

- When rounds **stop improving** → diminishing returns → the move is **a human expert, not another prompt**.
- **More prompting cannot manufacture judgment the situation requires.**

5.4 Owning the Output

- **Accountability does not transfer to the tool.**
- Work made with Claude is **your work** when you ship it; the professional standard is the **same as if unaided**.

5.5 Three Escalation Scenarios

Scenario	Thresholds	Decision
The fast "yes": internal meeting agenda	Low stakes, reversible, internal, no regulation	**Ship, no escalation**
The deceptive "looks fine": board-deck financial summary that reads cleanly	High stakes, executive audience, partly irreversible once presented (**trips 3 thresholds**)	**Human reviewer + recompute figures with code execution**. Clean look is irrelevant
The slow creep: client proposal iterated 5 times; rounds 3–5 changed almost nothing	Diminishing returns + high-stakes external deliverable	**Stop prompting; escalate to a colleague for a fresh read**. Signal is the **flat improvement curve**, not a visible error

---

6. Editing and Adapting Output for Your Audience

"Claude drafts; you deliver." The editing pass is where **your professional standards and audience knowledge** are applied. **An accurate draft is still not finished.**

6.1 Three Editing Passes

Pass	What you do
Clarity	Cut hedging, tighten loose sentences, remove anything not earning its place. Claude tends to **thoroughness**; editing tends to **precision**
Tone	Match **register** to relationship and occasion: peer vs client vs regulator
Formatting	Shape for how it's read: **scannable for executives**, **detailed for a working team**, **clean for external recipients**

6.2 Audience Calibration (one analysis, several deliverables)

Audience	Approach
Executive summary	Leads with **the decision and the impact**
Working team	Keeps **detail and method** (and assigns action)
External communication	**Controls what is disclosed** and **how it's framed**

Facts stay the same; selection, depth, and tone change.

Worked transformation (Q3 processing time +18%):

- **Raw:** one long hedged sentence ("may be attributable to… it is recommended that the team consider whether…"). Serves **neither** audience.
- **Executive:** "Q3 processing time rose 18 percent, driven by volume plus onboarding three new hires. Recommend standardized onboarding docs to limit the effect next time." → **number + decision first; method stripped**.
- **Working team:** "Processing time was up ~18% in Q3. Two drivers: higher volume and three new staff still onboarding. Action: draft onboarding documentation… (owner and timeline to confirm in standup)." → **keeps detail, adds operational next step**.

6.3 Compare Outputs Before You Edit

- When quality matters, **generate more than one draft** (**across runs or across models**) and **pick the strongest base**.
- **Comparing candidates is cheaper than rescuing a weak draft.**
- It also surfaces **framing you might not have prompted for**.

---

7. Choosing Output Formats

Output format is a reliability decision, not just presentation. Choose by purpose and, above all, by **how much the numbers must be trusted**.

7.1 Format by Purpose

Format	Use for
Inline	Conversational replies you **act on within the chat**; quick, contextual, not standalone
Artifacts	**Documents and code**: separate, editable block you **refine and reuse**; home for a **deliverable**
Structured formats	**Data**: tables and defined schemas that **downstream tools or readers consume directly**
Code execution	**Numbers that must be right**: computed, checkable results, plus charts and processed files

7.2 Code Execution: The Verified-Output Path

- **Have Claude compute numbers, not write them.**
- Prose generation → a **plausible-looking figure** ("a guess in the shape of an answer").
- Code execution → **runs the calculation**, returns a **computed, traceable** result.
- **Caveat (exam trap):** determinism attaches to the **executed computation**, but **Claude writes the code, so the logic can still have a bug**. The guarantee is that you can **read, verify, and re-run** it, **not that it's automatically correct**.

Same task, two paths (Q3 revenue + top 3 accounts from a spreadsheet):

Path	Output	Trust
Prose	"About **$4.7 million**; largest accounts Northwind, Contoso, Globex"	Fluent, fast, **unverifiable**. Model can't reliably add a column; a wrong total **propagates into every downstream slide**
Code-executed	**$4,712,380** computed sum, top 3 **ranked by real totals**, bar chart	**Traceable to the rows**. When a figure feeds a decision, **the only path that earns trust**

- Prose isn't "lazy"; it's a different kind of output. Fine for a **low-stakes gut-check**; anything **reported** → code execution.

7.3 Curate Inputs to Shape Output

- **Organized inputs → organized outputs.** Supply clean, labelled material and **state the structure you want back**.
- **Selection rule:** pick the output modality by the **reliability the task requires**.
- Curating means **removing wrong material**, not just adding right material.

Technique	Why
De-duplicate sources	Claude isn't reconciling three near-identical copies
Label and structure inputs	Each input's role is explicit ("this is the approved policy; these are the draft responses")
Prune irrelevant material	**Noise in the input becomes noise in the output**

Clean, well-labelled, minimal inputs beat a large undifferentiated pile.

---

8. Putting It Together: A Review Decision Flow

This combines Discernment (Section 2) and Diligence (Section 5) into one routine.

```
1. SET STAKES FIRST ........ zero-tolerance (legal, financial, compliance) or low-stakes
internal?
2. CHECK 3 REFERENCES ...... requirements (every part) · sources (trace claims) ·
professional standards
3. CHECK COMPLETENESS ...... what's missing? look hardest at the most important / hardest
input
4. SCAN FOR SIGNATURES ..... uncited specifics · confident legal/regulatory claims ·
contradictions
                             (consistency pass) · too-ready agreement · claimed external
actions
5. VERIFY WHAT MATTERS ..... sources / citations / code execution for numbers /
authoritative check
6. APPLY 4 THRESHOLDS ...... stakes · reversibility · audience · regulatory exposure
                             + is it on the do-not-ship list?
7. VERDICT + REASON ........ READY TO USE / NEEDS REVISION / NEEDS HUMAN OVERRIDE
8. IF ITERATING ............ change one thing per round; flat curve → escalate to a human
9. EDIT FOR AUDIENCE ....... clarity · tone · formatting
```

Quick verdict guide

If you see…	Verdict usually is…
Low stakes, internal, meets requirements, nothing to source-check	**Ready to use** (light edit if needed)
A specific, fixable gap: missed source detail, contradiction, unsourced stat, wrong total	**Needs revision** (note the gap, targeted re-prompt or recompute)
High stakes + regulatory/legal/external/irreversible, or missing authoritative source, or iteration has flat-lined	**Needs human override**

---

9. Links From Lessons 01 and 02 That Are Tested Under This Domain

Earlier point	How it shows up in Domain 2
Responses vary (L01)	Plan for variation; Best-of-N uses it to find soft spots
Confident tone ≠ accuracy (L01)	Basis of the "confident tone masking uncertainty" pattern
Training cutoff (L01)	Compliance-gap case: Claude recalled a rule that may have changed
Skills still vary (L01)	Review stays in even for configured workflows
Artifacts vs inline (L01)	Output format choice in Section 7
Code Execution = verified computation (L01)	Code-executed path for numbers
Citations from training memory need checking (L02)	Auditable citations; validate against authoritative sources
Code execution for counts (L02 feedback case)	Numbers that must be right are computed, not written
Iteration converges / diminishing returns (L02)	Here, a flat curve on high-stakes work means **escalate**, not just "edit manually"

---

10. Common Exam Traps

1. **"It looks clean / polished / confident, so it's fine."** Never. Polish and tone are not evidence.
2. **"Ask Claude to rate its own confidence."** Self-reported confidence is not a reliable accuracy signal.
3. **"Ask Claude again / ask another model" as validation of a high-stakes claim.** Use an **authoritative external source**. (Re-running is Best-of-N: it finds soft spots, it doesn't prove facts.)
4. **"Code execution = guaranteed correct."** It's traceable and re-runnable; the code can still have a bug.
5. **"Spot-checks passed, so the regulatory filing can go."** Regulatory + irreversible + high stakes = human override regardless.
6. **"Verify everything to the maximum, always."** Over-verifying low-stakes internal work wastes the time saved. Calibrate to stakes.
7. **"Accurate means complete."** They fail independently.

8. **"Keep prompting until it's right."** When rounds stop improving high-stakes work, escalate to a person.
9. **"Claude said it sent / saved it."** Capability hallucination until you confirm.
10. **"Rewording or reformatting fixes it."** Format changes don't fix correctness (except choosing code execution for numbers, which changes how the number is produced).
11. **"Upload everything for maximum context."** Curate: de-duplicate, label, prune.
12. **"The tool shares responsibility."** Accountability does not transfer to the tool.

Best-of-N vs comparing drafts (easy to confuse)

	Best-of-N comparison (Section 4.2)	**Comparing outputs before editing (Section 6.3)**
Purpose	**Verification**: find where runs disagree	**Quality**: pick the strongest base to edit
Runs across	Same request, re-run	Runs **or models**
Outcome	Agreement raises confidence; divergence = soft spot for human look	Edit the best candidate; may surface new framing

---

Most Likely Questions

60 questions in two sets. Set 1 covers every section; Set 2 goes deeper on calibration, thresholds, the triage exercise and traps.

Set 1: Core Questions (Q1 to Q30)

Q1. An associate asks Claude to summarize a new regulation. The summary cites a specific subsection number confidently. Before sending it to the compliance team, what should they do?

- A. Send it; Claude was confident.
- B. Verify the cited subsection against the official regulation text.
- C. Ask Claude to rate its confidence and send if high.
- D. Reword it to sound more formal.

Answer: B
Why: Fabricated specifics like citation numbers are a hallucination signature. Validate against an authoritative source, especially for a compliance audience. Self-reported confidence (A, C) is not reliable evidence. Rewording (D) doesn't fix correctness.

Q2. What are the three references in the discernment protocol? (Select 3)

- A. The requirements you set
- B. The source material
- C. The professional standards of your field
- D. How polished the output looks
- E. Claude's stated confidence

Answer: A, B, C
Why: Evaluation is a check against requirements, sources and professional standards. Polish (D) and confidence (E) are explicitly not signals of quality.

Q3. Which AI Fluency competency is about deciding when verification is required and taking responsibility for the result?

- A. Description
- B. Discernment
- C. Diligence
- D. Decomposition

Answer: C
Why: Diligence is why you must verify and owning the result. Discernment is how you review (evaluating against references). Description is the prompting competency from Lesson 02. Decomposition (D) is the Lesson 02 technique for splitting complex tasks.

Q4. Claude summarizes three competitors' pricing from uploaded PDFs. One price reads "$40/user" but the PDF says "$40/user, minimum 10 seats." What is the verdict and fix?

- A. Ready to use; the price is correct.
- B. Needs revision; use a source-restricted re-prompt.
- C. Needs human override; escalate to legal.
- D. Needs revision; rewrite the whole prompt.

Answer: B
Why: Requirements were met but the source check failed, and the omission changes the comparison. Note the gap and iterate with a source-restricted re-prompt, not a full rewrite (D). It's not a high-stakes escalation case (C). A ignores a completeness gap that changes the meaning.

Q5. Claude gives three sensible options to reduce internal invoice-processing time, as a discussion starter. No sources were involved. What is the most appropriate verdict?

- A. Ready to use
- B. Needs revision
- C. Needs human override
- D. Verify every sentence against external sources first

Answer: A
Why: Requirements met, no sources to check, standards cleared, low stakes. Over-verifying (D) wastes the time the tool saved. Stakes calibration means lighter review for low-stakes internal work.

Q6. Claude compares your data-handling policy against a regulation and confidently flags four gaps. The regulation itself was not uploaded. What is the verdict?

- A. Ready to use, since four gaps were found.

- B. Needs revision; ask Claude to find more gaps.
- C. Needs human override; Claude relied on training recall of a rule that may have changed, and the stakes are regulatory.
- D. Ready to use if Claude says it's confident.

Answer: C
Why: Without the source, Claude worked from memory of a possibly outdated rule, and regulatory stakes are high. The output is a useful prompt for a compliance expert, not a substitute for one.

Q7. Which statement about accuracy and completeness is correct?

- A. An accurate output is always complete.
- B. They fail independently; an accurate output can still omit the one decisive factor.
- C. Completeness is only relevant for legal documents.
- D. Missing items are easier to spot than wrong ones.

Answer: B
Why: Accuracy asks whether what's present is correct; completeness asks whether anything is missing. Missing items are harder to spot, because nothing on screen draws your eye (D is reversed).

Q8. Which hallucination pattern is described as the most dangerous?

- A. Fabricated citations
- B. Plausible-but-unsupported claims
- C. Internal contradictions
- D. Capability hallucinations

Answer: B
Why: A plausible-but-unsupported claim sounds reasonable and fits the topic, so nothing about it looks wrong. The others are real failures but are not labelled "most dangerous".

Q9. Claude states: "Approximately 63 percent of mid-market SaaS firms adopted AI tools in 2025, up from 41 percent in 2024." No source is given. What is the key warning sign?

- A. The trend direction
- B. The precision of the figures with no citation
- C. The use of the word "approximately"
- D. The topic is too recent

Answer: B
Why: "The precision is the tell." Real figures this specific come with a citation; an uncited 63% is a number-shaped guess. The trend (A) is plausible, which is what makes it persuasive.

Q10. After a session, Claude says "I've emailed the summary to your team." No email tool or connector was set up. How should you treat this?

- A. Trust it; Claude can send emails from claude.ai.
- B. As a capability hallucination; treat it as unverified until you confirm it happened.
- C. As a confirmation bias problem.
- D. Ask Claude to resend it.

Answer: B
Why: Claude can claim actions it can't take. In claude.ai it works with the conversation, connected tools and uploaded files only. Any claimed external action is unverified until confirmed.

Q11. A manager asks: "Our new pricing strategy is clearly better than the old one, right? Explain why." Claude agrees enthusiastically. What risk should you watch for?

- A. Capability hallucination
- B. Confirmation bias in framing
- C. Context window limit
- D. Internal contradiction

Answer: B
Why: When a prompt implies a preferred answer, Claude may lean toward it. Watch for output that agrees too readily on a question that should be open.

Q12. A ten-page market analysis says the market is "roughly $2 billion" on page 2 and builds a projection on "$2.6 billion" on page 8. Which review step would catch this?

- A. A paragraph-by-paragraph read
- B. A consistency pass across the whole document
- C. Asking Claude to sound more confident
- D. Changing the output format to inline

Answer: B
Why: Internal contradictions hide in long outputs because each part reads fine alone. They only show when you hold the whole document in view.

Q13. Claude compares 15 contracts and lists differences thoroughly, but misses changes in the single most important contract. What failure type is this?

- A. Fabricated specific
- B. Completeness failure
- C. Capability hallucination
- D. Confirmation bias

Answer: B
Why: Completeness failures concentrate where attention is lowest. A confident summary of the easy files can mask silence on the hard one.

Q14. Which prompt techniques reduce hallucinations at the source? (Select 3)

- A. Explicitly permit "I don't know"
- B. Restrict answers to the provided sources and list what isn't covered
- C. Require citations with section and clause numbers
- D. Ask Claude to sound authoritative

Answer: A, B, C
Why: These are the three "prompt for verifiability" techniques. Permission to not know reduces invention under pressure. Source restriction turns generation into bounded retrieval. Auditable citations make claims traceable. D increases the risk of confident errors.

Q15. Why does permitting "I don't know" help?

- A. It makes responses shorter.
- B. Without it, a model under pressure to answer is more likely to fill gaps with invented content.
- C. It switches Claude to a more capable model.
- D. It turns on web search.

Answer: B
Why: Giving explicit permission to admit uncertainty reduces the pressure to fabricate. A, C and D are not effects of this instruction.

Q16. You run the same analysis prompt three times. Two conclusions agree; one figure differs across all three runs. What does this tell you?

- A. All three runs are wrong.
- B. Agreement raises confidence; the diverging figure is a soft spot needing a human look.
- C. Use the most common answer and ship it.
- D. The model is broken.

Answer: B
Why: This is Best-of-N comparison. Where runs agree, confidence rises; where they diverge, you've found the spots that need human review. C stops short of checking the soft spot.

Q17. To validate a high-stakes claim, which is the best check?

- A. Ask Claude a second time.
- B. Ask a different Claude model the same question.
- C. Check against a trusted, authoritative external reference.
- D. Check whether the claim is written confidently.

Answer: C
Why: Validate against an authoritative source rather than a second Claude response (A, B). Confidence (D) is not evidence.

Q18. Which are the four risk thresholds for deciding on human review? (Select 4)

- A. Stakes
- B. Reversibility
- C. Audience
- D. Regulatory exposure
- E. Length of the output

Answer: A, B, C, D
Why: These four thresholds decide when human review is non-negotiable. Length (E) is not a threshold.

Q19. Which items belong on the do-not-ship-without-review list? (Select 3)

- A. Final client deliverables
- B. Audit-critical or financially material calculations
- C. Public or legal communications with lasting consequences
- D. An internal meeting agenda

Answer: A, B, C
Why: The fixed list is client deliverables, audit-critical or financially material calculations, regulated/confidential/sensitive data, and public or legal communications. An internal agenda (D) is the "fast yes" case.

Q20. A board-deck financial summary from Claude reads cleanly. What should happen?

- A. Ship it; it reads cleanly.
- B. Send to a human reviewer and recompute the figures with code execution.
- C. Ask Claude to double-check its own numbers in prose.
- D. Convert it to an Artifact and ship.

Answer: B
Why: High stakes, executive audience and partial irreversibility trip three thresholds. The clean appearance is irrelevant. C relies on prose arithmetic. D changes the format, not the reliability.

Q21. You've iterated a client proposal five times. Rounds 3 to 5 changed almost nothing. What is the right move?

- A. Keep iterating with stronger wording.
- B. Switch to Haiku for speed.
- C. Stop prompting and escalate to a colleague for a fresh read.
- D. Ship it, since iteration has converged.

Answer: C
Why: Flat improvement plus a high-stakes external deliverable signals escalation. More prompting can't manufacture the judgment the situation needs. D ignores the high-stakes external audience.

Q22. Who is accountable for a client report drafted with Claude's help?

- A. Anthropic
- B. Claude
- C. The professional who ships it, to the same standard as if unaided
- D. Shared equally between the user and Claude

Answer: C
Why: Accountability does not transfer to the tool. Work made with Claude is your work when you ship it.

Q23. One analysis must become an executive summary and a working-team update. What changes between them? (Select 2)

- A. The underlying facts and figures
- B. Selection and depth of detail
- C. Tone and framing
- D. Whether the numbers are verified

Answer: B, C
Why: Facts stay steady; selection, depth and tone change with the reader. Executives get decision and impact first; the team keeps method and action. Verification (D) applies to both.

Q24. Which description fits the executive version of an analysis?

- A. Keeps the full method and assigns owners and timelines
- B. Leads with the number and the decision, in as little as one sentence
- C. Controls what is disclosed to outsiders
- D. A raw draft with hedging intact

Answer: B
Why: Executives get the decision and impact first. A describes the working-team version. C describes external communications. D serves no audience.

Q25. What are the three editing passes that turn a raw draft into a deliverable?

- A. Clarity, tone, formatting
- B. Accuracy, completeness, sourcing
- C. Role, context, task
- D. Stakes, audience, reversibility

Answer: A
Why: Clarity (cut hedging, tighten), tone (match register) and formatting (shape for how it's read). B is evaluation, C is prompt components, D is risk thresholds.

Q26. When quality matters, what does the lesson recommend before editing?

- A. Commit to the first response to save time.

- B. Generate more than one draft (across runs or models) and edit the strongest.
- C. Always use the longest draft.
- D. Merge every draft into one.

Answer: B
Why: Comparing candidates is cheaper than rescuing a weak draft, and it can surface framing you didn't prompt for.

Q27. You need total Q3 revenue from an uploaded sales spreadsheet for a board slide. Which output path fits?

- A. Inline prose answer
- B. Code execution over the actual file
- C. Brainstorm several estimates
- D. Ask for a confident answer

Answer: B
Why: When numbers must be right and get reported, code execution computes a traceable result. A prose total is a guess in the shape of an answer, and a wrong total propagates into every downstream slide.

Q28. Which statement about code execution is TRUE?

- A. Code-executed results are guaranteed correct.
- B. The computation is deterministic, but Claude writes the code, so the logic can contain a bug; you can read, verify and re-run it.
- C. Code execution is only for producing charts.
- D. Code execution removes the need for human review.

Answer: B
Why: The guarantee is traceability and re-runnability, not automatic correctness. A and D overstate it. C ignores computed results and processed files.

Q29. Match the need to the format. A data table that a downstream tool will ingest should be produced as:

- A. Inline reply
- B. Structured format (table or defined schema)
- C. A brainstorm list
- D. A long narrative Artifact

Answer: B
Why: Structured formats are for data that downstream tools or readers consume directly. Inline is for acting within the chat. Artifacts are for documents and code.

Q30. Before asking Claude to compare draft customer responses against policy, you have three near-identical copies of the policy and several unrelated files. Which input-curation steps help? (Select 3)

- A. De-duplicate the policy copies.
- B. Label which file is the approved policy and which are draft responses.
- C. Prune the unrelated files.
- D. Upload everything so Claude has maximum context.

Answer: A, B, C
Why: De-duplicate, label and structure, and prune irrelevant material. Noise in the input becomes noise in the output. D is the "undifferentiated pile" the lesson warns against.

Set 2: Additional Practice Questions (Q31 to Q60)

Difficulty is marked as [Easy], [Medium] or [Hard].

Q31. [Easy] What does "stakes calibration" mean?

- A. Every output gets the same depth of review.
- B. Review depth is set by the stakes, and the stakes depend on the domain.
- C. Review depth is set by how long the output is.
- D. Only legal documents need review.

Answer: B
Why: Stakes are domain-dependent, not universal. Zero-tolerance work (legal, financial, compliance) gets every claim verified. Low-stakes internal brainstorming gets a lighter review. A is the risk the lesson warns about. C and D are not the rule.

Q32. [Medium] An operations lead uses the same quick skim for internal brainstorms and for monthly figures sent to the finance controller. What is the main problem?

- A. The skim is too slow for brainstorms.
- B. The same casual review is being applied to zero-tolerance work and low-stakes work.
- C. Brainstorms should get the deepest review.
- D. Nothing; one consistent method is best.

Answer: B
Why: You decide the stakes first and then the depth of review. Financial figures are zero-tolerance, where accuracy outranks speed entirely. D confuses a consistent *protocol* (good) with consistent *depth* (bad).

Q33. [Easy] Which items fail professional standards even when they read fluently? (Select 3)

- A. A number without units
- B. A recommendation without reasoning
- C. A citation you cannot locate
- D. A bulleted list

Answer: A, B, C
Why: These are the three examples the lesson gives. Fluency doesn't save them. A bulleted list (D) is a format choice, not a standards failure.

Q34. [Medium] You asked Claude for five things in one request. It handled four well, but the fifth (the hardest) is thin. Which evaluation reference should have caught this?

- A. Source material
- B. Requirements
- C. Professional standards
- D. Stakes

Answer: B
Why: The requirements check means re-reading your own request and confirming each part is addressed, not just the easy parts. Stakes (D) set review depth; they are not one of the three references.

Q35. [Easy] What is the right way to check outputs against the source material you supplied?

- A. Trust that Claude read the documents carefully.
- B. Trace specific claims back to the source.
- C. Check whether the summary is well organized.
- D. Ask Claude whether it used the documents.

Answer: B
Why: Trace specific claims back to the source rather than trusting that Claude read carefully (A). Organization (C) and self-report (D) are not evidence.

Q36. [Medium] A reviewer looks at an output Claude produced and labels it "Needs revision" with no further note. What is missing from the triage step?

- A. Nothing; the label is enough.
- B. Documented reasoning, including the specific gap to fix.
- C. A confidence score from Claude.
- D. A second label.

Answer: B
Why: Triage sorts outputs into three states *with documented reasoning*. For "Needs revision" you note the gap and iterate. Without the gap noted, nobody knows what to fix.

Q37. [Medium] Claude drafts a promotional email that will be sent to 50,000 retail customers. Which risk thresholds does this clearly trip? (Select 3)

- A. Stakes
- B. Reversibility
- C. Audience
- D. Word count

Answer: A, B, C
Why: A mass customer email is high-cost if wrong, can't be unsent, and has an external audience. Word count (D) is not a threshold.

Q38. [Easy] Which are examples of "fabricated specifics"? (Select 4)

- A. Invented statistics
- B. Invented dates
- C. Invented quotations
- D. Invented citations
- E. Section headings

Answer: A, B, C, D
Why: Fabricated specifics are invented statistics, dates, names, quotations or citations. Headings (E) are structure, not factual claims.

Q39. [Medium] Why are fabricated specifics especially persuasive?

- A. Claude marks them in bold.
- B. Specificity reads as authority.
- C. They always appear with a citation.
- D. They are usually close to the real figure.

Answer: B
Why: Precise numbers and names look researched. That is exactly why they persuade, and why precision with no citation is the tell. C is the opposite of the warning sign.

Q40. [Medium] A user asks "Is this clause enforceable in our state?" and Claude answers "yes" in four assured sentences. Why is this a red flag?

- A. The answer is too short.
- B. Enforceability is jurisdiction-specific and date-sensitive, so it should hedge and doesn't.
- C. Claude cannot read contracts.
- D. Legal questions require Research mode.

Answer: B
Why: This is "confident tone masking uncertainty". Claude rarely hedges in proportion to its real certainty, and assurance is not evidence.

Q41. [Medium] Which prompt best turns open-ended generation into bounded retrieval?

- A. "Tell me everything you know about this contract's risks."
- B. "Answer using only the attached contract. Do not use general knowledge. List anything it doesn't address under 'Not covered by this document.'"
- C. "Be as confident as possible."

- D. "Summarize the contract in a friendly tone."

Answer: B
Why: Source restriction plus an explicit "not covered" list bounds the answer to supplied material and flags gaps. A invites general knowledge. C and D don't touch grounding.

Q42. [Medium] A credit analyst must assess a 120-page credit policy against a specific question. Which technique makes both the reasoning and any errors visible?

- A. Ask for a one-line answer.
- B. Quote first, then analyze: extract the exact sentences that bear on the question and base the analysis only on them.
- C. Ask Claude to rate its certainty.
- D. Switch to an inline response.

Answer: B
Why: Quote-first grounding for long documents shows exactly what the conclusion rests on, so a wrong quote or a leap in logic is easy to see.

Q43. [Hard] A colleague says: "I ran the same prompt three times and got the same capital-adequacy figure each time, so it's verified for the regulator." What is the best response?

- A. Agree; three identical runs prove the figure.
- B. Agreement raises confidence, but a claim that matters must be validated against an authoritative source, and a regulatory submission needs human review.
- C. Run it ten more times.
- D. Ask a different model to confirm.

Answer: B
Why: Best-of-N finds soft spots where runs diverge; agreement only raises confidence. High-stakes claims are checked against a trusted external reference, not another Claude response (D). Regulatory content also trips the review thresholds.

Q44. [Easy] Which in-product aid can tie figures back to their inputs with cell-level citations?

- A. Incognito mode
- B. Claude for Excel
- C. Memory
- D. Research mode

Answer: B
Why: The lesson names Claude for Excel as an aid for validation, producing cell-level citations.

Q45. [Medium] Which is NOT part of the verification checklist before relying on an output?

- A. Did I allow uncertainty?

- B. Did I restrict to sources where appropriate?
- C. Did I ask Claude to rate its own confidence?
- D. Did I check high-stakes claims against something authoritative?

Answer: C
Why: The checklist is: allow uncertainty, restrict to sources, require auditable citations, check high-stakes claims against an authoritative source. Self-rated confidence is not a reliable signal.

Q46. [Easy] Under the reversibility threshold, which output must clear the highest bar?

- A. An internal draft you will revise tomorrow
- B. A report filed with an authority
- C. A brainstorm list
- D. A meeting agenda

Answer: B
Why: Irreversible steps, such as a sent client deliverable or a filed report, clear a higher bar than drafts you can still revise.

Q47. [Medium] Why should risk thresholds and the do-not-ship list be decided in advance?

- A. To save typing time.
- B. So escalation is decided by policy, not in the moment after something has gone wrong.
- C. So Claude can enforce them.
- D. Because Claude needs them in Memory.

Answer: B
Why: Diligence means knowing the thresholds ahead of time, so the escalation decision doesn't depend on how rushed or confident you feel at the moment.

Q48. [Medium] In the "slow creep" scenario (client proposal iterated five times), what is the signal to escalate?

- A. A visible factual error
- B. The flat improvement curve across later rounds
- C. Claude refusing to continue
- D. A usage limit warning

Answer: B
Why: Rounds 3 to 5 changed almost nothing. Diminishing returns on a high-stakes external deliverable mean stop prompting and get a colleague's fresh read. No visible error is needed.

Q49. [Hard] After six rounds, Claude's interpretation of an ambiguous clause in a lending agreement keeps shifting slightly, and nothing is getting better. What does Diligence call for?

- A. A seventh round with a stronger role prompt.

- B. A human expert, because more prompting cannot manufacture the judgment the situation requires.
- C. Pick the most recent answer.
- D. Switch to brainstorming mode.

Answer: B
Why: When iteration stops improving the output, the right move is a human expert, not another prompt. The legal stakes reinforce that.

Q50. [Medium] A manager argues that because Claude drafted a regulated disclosure, some of the team's regulatory obligations are reduced. What is correct?

- A. True; AI assistance shares the obligation.
- B. False; regulated content carries obligations that AI assistance does not remove.
- C. True, if the output was reviewed by a second model.
- D. True, for internal disclosures only.

Answer: B
Why: Regulatory exposure is a threshold precisely because the obligations stay with you. Accountability does not transfer to the tool.

Q51. [Easy] Which action belongs to the **clarity** editing pass?

- A. Matching register to a regulator
- B. Cutting hedging and tightening loose sentences
- C. Making the document scannable for an executive
- D. Choosing a table format

Answer: B
Why: Clarity cuts hedging, tightens sentences and removes anything that doesn't earn its place. A is the tone pass. C and D are formatting.

Q52. [Medium] What is distinctive about adapting an analysis for an **external** communication?

- A. It keeps all method detail.
- B. It controls what is disclosed and how it is framed.
- C. It changes the underlying facts.
- D. It assigns owners and timelines.

Answer: B
Why: External versions control disclosure and framing. A and D describe the working-team version. Facts never change (C).

Q53. [Medium] Claude's draft is fully accurate. Why is it still "not finished"?

- A. Accurate drafts must always be rewritten.
- B. The editing pass is where your professional standards and knowledge of the audience get applied.

- C. Claude always adds errors in formatting.
- D. Only Opus drafts are finished.

Answer: B
Why: "Claude drafts; you deliver." An accurate draft still needs clarity, tone and formatting matched to its reader.

Q54. [Medium] The raw Q3 output was one long hedged sentence about an 18% rise in processing time. Why does it serve neither the executive nor the working team?

- A. It contains a wrong number.
- B. It buries the number and decision for executives and lacks a clear action and owner for the team.
- C. It is too short.
- D. It uses a table.

Answer: B
Why: Executives need the number and decision first; the team needs the drivers and an operational next step. The facts were right; the shaping was wrong.

Q55. [Hard] In the sample triage exercise, Output C was a market overview with three statistics that named no checkable source. Which reasoning matches the model answer?

- A. Ready to use, because the statistics sound plausible.
- B. Needs revision: unsourced statistics are unverified claims; require auditable citations or validate against authoritative sources.
- C. Needs revision: add more background context to the prompt.
- D. Needs human override, because all market overviews are regulated.

Answer: B
Why: The gap is verifiability, so the fix is citations or authoritative validation. More context (C) doesn't make a statistic checkable. D overstates the stakes.

Q56. [Hard] In the sample triage exercise, Output B was a financial summary whose total didn't match its line items. Which statement captures the complete model answer? (Select 2)

- A. The diagnosis is an internal contradiction between the total and the line items.
- B. The fix is to recompute with code execution and confirm before use.
- C. The verdict is "Ready to use" because the line items are present.
- D. The verdict is "Needs human override" because all figures need a regulator.

Answer: A, B
Why: The verdict is "Needs revision": the diagnosis is the contradiction, and the fix is recomputing with code execution and confirming. Naming only the fix misses the diagnosis the exam expects you to spot.

Q57. [Medium] A prose answer gives Q3 revenue as "about $4.7 million" and it goes into a board deck. What is the specific danger?

- A. The number is rounded.
- B. The model can't reliably sum a column in prose, and a wrong total propagates into every downstream slide.
- C. The board prefers charts.
- D. Prose answers can't name accounts.

Answer: B
Why: A prose total is the model's best guess at summing a column. Once reported, the error spreads. Code execution gives a computed, traceable figure.

Q58. [Medium] When is a prose (non-computed) number acceptable?

- A. For a board report
- B. For a low-stakes gut-check
- C. For an audit file
- D. Never

Answer: B
Why: The prose answer is not lazy, just a different kind of output. It may be fine for a quick gut-check. Anything that gets reported needs code execution. D is too absolute.

Q59. [Medium] What does "curating inputs" involve? (Select 2)

- A. Removing wrong or irrelevant material, not just adding the right material
- B. Stating the structure you want back
- C. Uploading every related file for maximum context
- D. Keeping duplicate versions so Claude can choose

Answer: A, B
Why: Organized inputs produce organized outputs. Curating includes removing material and stating the output structure. C and D create noise, and noise in the input becomes noise in the output.

Q60. [Hard] Claude prepares a credit-risk summary for a bank's board from an uploaded portfolio file. The figures were written in prose, and one paragraph cites a central bank circular from memory (it wasn't uploaded). Which actions are required? (Select 3)

- A. Recompute the figures with code execution over the uploaded file.
- B. Validate the circular citation against the authoritative text.
- C. Send to a human reviewer before it goes to the board.
- D. Ship it, since Claude used the uploaded file.

Answer: A, B, C
Why: Board audience, high stakes and partial irreversibility trip the thresholds, so it needs human review. Numbers that feed decisions must be computed, not written. A citation from training memory must be checked against the authoritative source. D ignores all three risks.

Key Takeaways

1. **You own what you ship.** Accountability never transfers to Claude. Time saved is small and visible; the cost of an error is large and arrives later.
2. **Discernment (how) + Diligence (why).** Check every output against **requirements, source material, professional standards**, the same way every time, at a depth set by the **stakes**.
3. **Triage into three verdicts:** Ready to use, Needs revision, Needs human override. Polish never decides the verdict.
4. **Accuracy and completeness fail independently.** Missing items are harder to see, and they hide in the hardest or most important file.
5. **Know the failure signatures:** plausible-but-unsupported claims (most dangerous), fabricated specifics (precision is the tell), confident tone, internal contradictions (run a consistency pass), confirmation bias, capability hallucinations ("I've emailed it").
6. **Build verification into the prompt:** permit "I don't know", restrict to sources, require auditable citations, quote first, Best-of-N, and validate against **authoritative** sources, not a second Claude answer.
7. **Four thresholds: Stakes, Reversibility, Audience, Regulatory exposure.** Fixed do-not-ship list: client deliverables, material calculations, sensitive or regulated data, public or legal comms. A flat iteration curve means escalate to a human.
8. **Claude drafts; you deliver.** Edit for clarity, tone and format. Tailor to exec, team or external readers, keeping facts constant. Compare drafts before editing.
9. **Format is a reliability decision.** Inline for in-chat action, Artifacts for documents and code, structured formats for data, **code execution for numbers that must be right** (still read and verify the code). Curate inputs: de-duplicate, label, prune.
10. **Run the same routine every time:** set stakes, check the three references and completeness, scan for failure signatures, verify what matters, apply the four thresholds, give a verdict **with a reason**, then edit for the audience.

04 · Workflow Integration & Solution Design (Revision Notes)

Where This Lesson Sits in the Exam

Domain	Weight	Objectives covered here
4 · Workflow Integration and Solution Design	**16%** (2nd largest)	Apply Claude to analyze requirements and use cases · Use Claude for research, planning, process optimization · Support solution design, development, iteration · Integrate Claude into existing workflows to augment or redesign them · Communicate Claude's value and limitations to stakeholders

Core idea: "I use Claude" (personal productivity habit) is different from **"our workflow uses Claude"** (a repeatable team process where Claude performs specific steps every time). Value **compounds** only when you make that shift **deliberately**; it goes wrong when teams **automate the wrong steps**.

Opening case: two teams, same tool, same contract-review process

	Team 1	Team 2
Approach	**Mapped the work**; Claude drafted the redline; **a lawyer owned every final decision**	Pointed Claude at **the whole process**; let it **approve low-risk clauses unsupervised**
Result	**Review time halved, quality held**	Within a month an approved clause created an **obligation no one caught**; team **pulled the tool entirely**

The difference was **which steps each team chose to delegate**.

Anchor competency (AI Fluency Framework): Delegation = deciding, **for each step**, whether the work is **AI-appropriate, human-retained, or collaborative**. Deliberate delegation turns individual wins into workflow value.

Exam Objective → Where It Is Covered

Domain 4 objective (exam guide)	Sections in these notes
Apply Claude to **analyze requirements and use cases**	1.1 to 1.4
Leverage Claude for **research, planning, and process optimization**	2.1 to 2.4
Support **solution design, development, and iteration**	3.1 to 3.3
Integrate Claude into existing workflows to **augment or redesign** them	4.1 to 4.6
Communicate Claude's **value and limitations** to stakeholders	5.1 to 5.6

Memory Map

```
WORKFLOW INTEGRATION & SOLUTION DESIGN
│
├── "I use Claude" (habit) → "OUR WORKFLOW uses Claude" (repeatable team process)
│     Anchor competency: DELEGATION (AI-appropriate / human-retained / collaborative)
```

```
│
├── 1. ANALYZING REQUIREMENTS
│       ├── Messy inputs → STRUCTURED, TESTABLE requirements (not narrative summary)
│       │     extract · organize · flag ambiguous / missing
│       ├── "We need better reporting" → what report, for whom, how often, what data, what
format
│       ├── RFP example: table = label · RFP section · answered? · ambiguity
│       │     run as a PROJECT: KB = past winning proposals · Skills = formatting
│       │     standing instructions = extraction format · Skills are ACCOUNT-level
│       └── PRESSURE-TEST: ask Claude to challenge its own list → hidden requirements
│             extraction + pressure-test > either alone (= Discernment at requirements
stage)
│
├── 2. RESEARCH, PLANNING, PROCESS OPTIMIZATION
│       ├── Synthesis (Claude excels) vs Calculation (must be verified)
│       ├── Current info: web search (quick) / Research (deep); verify synthesis
│       ├── Numbers → CODE EXECUTION ("guessed utilization = guess; computed = plan")
│       ├── Capacity plan: code exec on 4 quarters → Claude synthesizes recommendation
│       │     + shows assumptions → defensible line by line
│       └── Synthesis steps = most Claude value · Judgment steps (risk appetite,
│             politics, budget, hiring freeze) = STAY HUMAN
│
├── 3. SOLUTION DESIGN & ITERATION
│       ├── Design collaborator, not vending machine
│       ├── Loop: IDEATE → PROTOTYPE → GATHER FEEDBACK → REFINE (repeat)
│       ├── Run inside a PROJECT → stable context, constraints, prior decisions
│       ├── Dashboard artifact: Build → Filter & totals → Color & print (no code written)
│       └── ESCALATE when others depend on it (uptime, security, integration)
│             → Developer / Architect
│
├── 4. DELEGATION MAPPING (core skill)
│       ├── Each step: AI-appropriate / Human-retained / Collaborative
│       ├── 3 criteria: REVERSIBILITY · STAKES · ACCOUNTABILITY (never delegates)
│       ├── Embed features: Skill = repeatable procedure · Code exec = data steps
│       │     Human steps = explicit REVIEW GATES
│       ├── Over-delegation: drafting quality ≠ licence to delegate the decision
│       └── Mapping errors: HALO delegation · COLLAPSING collaborative into automate
│             · MAPPING THE TOOL not the work
│
└── 5. COMMUNICATING TO STAKEHOLDERS
      ├── Value AND limits; overstating loses trust at first visible miss
      ├── "Claude drafts first-pass redline, lawyer reviews" [good] vs "Claude handles
contract review" [bad]
      ├── Calibrate to AI literacy: Legal lead (detail + failure modes) ·
      │     Executive (outcome + oversight) · Risk function (assurance)
      ├── Document human oversight: explicit gates RAISE trust
      └── Red-flag phrases: "fully automated" · "Claude handles X" ·
            "basically as good as a person"  → fix: what tool does + human checkpoint
```

---

1. Analyzing Requirements and Use Cases with Claude

Most real work starts from **messy inputs**: a long document, a thread of half-formed emails, a verbal ask. Requirements must be **extracted, structured, and pressure-tested** before you build.

1.1 What Claude Does Here

- Turns **raw inputs into testable requirements** others can act on.
- Takes unstructured material and returns **structure**:

- **pulls** requirements out of a document,
- **organizes** them,
- **flags** what is **ambiguous or missing**.
- Rule: upload raw inputs and ask for a **structured analysis, not a narrative summary**.

1.2 Translating Business Needs into Task Definitions

- "We need better reporting" is **not actionable**.
- Claude helps convert it into **specific task definitions**:
- **what** report, **for whom**, **how often**, **from what data**, **in what format**.
- Each becomes a requirement you can **build against and check completion on**.

1.3 Worked Example: RFP Response Workflow

- **Inputs:** a **40-page RFP** + a **scattered internal email thread**.
- **Recurring task:** turn them into a **structured list of answerable requirements**.
- **Prompt asks for, per requirement:** a **short label**, the **exact RFP section**, **whether the thread already has an answer**, and any **ambiguity needing clarification**, returned **as a table**.
- **Output:** a requirements table the whole team works from; each row **traced to its RFP section**, marked **answered or open**, ambiguities flagged for a **clarifying question to the client**.

Why this is a Project, not a one-off Chat

Element	Where it lives
Past winning proposals	Project **knowledge base**
Extraction format	Project **standing instructions**
Formatting steps	**Skills** (live at the **account level** and apply **everywhere**, this Project included)

- **No technical build required.**

1.4 Pressure-Testing the Requirements

- **Extraction is the first pass; pressure-testing makes it trustworthy.**
- Ask Claude to **challenge its own list**: "Which are ambiguous as written? Which could be interpreted two ways by our proposal team? Which imply a requirement the RFP states only indirectly?"
- Surfaces **hidden requirements**: buried in a **subordinate clause** or **implied by an evaluation criterion**. Missing these **costs teams the bid**.
- **Structured list + pressure-test pass is far stronger than either alone.**
- This is the **Discernment** habit (Lesson 03) applied at the **requirements stage**.

---

2. Research, Planning & Process Optimization

Planning mixes two things Claude handles differently: **synthesis** (it **excels**) and **calculation** (must be **verified**). The strongest planning workflows **pair Claude's synthesis with code-executed analysis**, so the plan rests on numbers that were **computed, not generated**.

2.1 Research and Synthesis

- Claude synthesizes across sources to build a plan: **gather considerations, structure options, lay out trade-offs**.
- Current info after the training cutoff: **web search in chat** for quick lookups; **Research** for deeper up-to-date inputs.
- Synthesis is useful **and is where unverified claims can enter**, so Lesson 03 verification applies **throughout**.

2.2 Code Execution for Verified Analysis

- When a plan depends on numbers, **have Claude compute them**.
- Upload the dataset; use code execution to **run calculations, produce trend charts, process files**.
- "A staffing plan built on a **guessed utilization rate is a guess**; one built on a **code-executed analysis of actual timesheet data is a plan**."

2.3 Worked Example: Capacity Plan

- Operations lead planning **next quarter's headcount**.
- Workflow:
  1. Upload **last four quarters** of ticket-volume data.
  2. **Code execution** computes **quarterly volume growth** and **average tickets resolved per analyst**.
  3. Claude **synthesizes a staffing recommendation** from the verified figures to hold current resolution time, and **shows the assumptions**.
- **The recommendation is only as trustworthy as the figures under it.** Code-executed figures mean the plan can be **defended line by line**.
- **Identifying which steps of a plan are built on a number** is how you decide where AI insight impacts the answer.

2.4 Where AI Insight Changes the Plan

Step type	Who	Why
Synthesis steps (many considerations to weigh and structure)	**Claude adds the most**	Turning data into a structured, defensible recommendation
Judgment steps (risk appetite, political reality)	**Stay human**	Need context and authority Claude doesn't have

- A **quick scan for synthesis-heavy steps** tells you where to apply Claude and where to leave the call to a person.
- Capacity plan: Claude's leverage = four quarters of data → defensible recommendation. **The decision to hire** (against **budget and hiring-freeze realities Claude cannot see**) **stays with the operations lead**.

---

3. Solution Design, Development & Iteration

Claude is a design collaborator, not a vending machine. Value shows up in an **explicit loop**.

3.1 The Iteration Loop

Ideate → Prototype → Gather feedback → Refine (repeat until the solution holds)

Stage	Purpose
Ideate	Produces **options**
Prototype	Makes one **concrete**
Gather feedback	**Exposes what is wrong**
Refine	**Fixes it**

- Run the loop **inside a Project** so **context, constraints, and prior decisions stay stable**; each iteration **builds on the last instead of restarting**.
- Result: **a solution, not a pile of one-off drafts**.

3.2 Worked Example: Internal Process Tool (three iterations, no code written)

A business analytics team needed a small internal tool to **track and visualize** a set of metrics. Instead of commissioning a build, they had Claude produce it as a **web artifact** and **iterated by asking**.

Cycle	Request
1 · Build	"Build a simple dashboard artifact that shows these five metrics from the attached data, with a chart for each." → working artifact
2 · Filter & totals	"Add a filter by region and a summary row at the top with the totals." → refine by **describing the change, not coding it**
3 · Color & print	"Month-over-month deltas color-coded, green for improvement; layout should print cleanly."

Three cycles, **no code written by the team**.

3.3 Knowing When to Escalate

- The artifact worked because it served **a small team's internal need**.
- When a solution becomes **a system others depend on**, with **uptime, security, or integration requirements**, it has **outgrown Associate scope → Developer or Architect** expertise.
- **Escalation signal = dependency**: the moment people **rely on it as infrastructure**, it's no longer a prompt-and-iterate exercise.

---

4. Delegation Mapping: Redesigning Workflows with Claude Inside

The core skill of this module. Before redesigning a workflow around Claude, **map it step by step** and decide who owns each step: **AI, a human, or both**. Getting it right decides whether the workflow **compounds value or quietly accumulates risk**.

4.1 Three Classifications, Three Criteria

Classify each step as: AI-appropriate · Human-retained · Collaborative

Criterion	Question	Rule
Reversibility	Can it be undone if Claude gets it wrong?	Reversible → more delegation; **irreversible → human involvement**
Stakes	Cost of an error at this step?	High-cost steps stay **human-owned or human-reviewed**
Accountability	Who is answerable for the outcome?	**Accountability does not delegate, even when the drafting does**

4.2 Building the Redesign

- Embed the right feature at each AI step:
- **Skill → repeatable procedure** steps
- **Code execution → data** steps
- **Consistency from a configured Skill beats "heroic prompting"** that relies on remembering the right wording every time.
- **Human-retained steps become explicit review gates, not afterthoughts.**

4.3 Worked Map 1: Contract Review (a common first-win workflow)

Step	Delegation	Why
Extract clauses	**AI-appropriate**	Reversible, low stakes, mechanical
Flag departures from the company playbook	**AI-appropriate**	Reversible; **a Skill carries the playbook rules**
Draft the redline and rationale	**Collaborative**	AI drafts, **human judges each edit**
Approve or reject each change	**Human-retained**	High stakes; accountability doesn't delegate
Compute financial exposure of a penalty clause	**AI-appropriate (code execution)**	Numeric; **computed, not estimated**
Sign and send	**Human-retained**	**Irreversible, external, legally binding**

4.4 Worked Map 2: Onboarding Documents

Step	Delegation	Why
Pull new-hire details from the HRIS export	**AI-appropriate (code execution)**	Mechanical, reversible, **must be exact**
Draft the offer letter from the approved template	**AI-appropriate**	Reversible draft; **a Skill carries the template**
Personalize the welcome note	**Collaborative**	AI drafts; **hiring manager adds the human voice**

Step	Delegation	Why
Confirm compensation figures match the approved req	**Human-retained**	High stakes; accountability doesn't delegate
Send the signed offer	**Human-retained**	Irreversible, legally binding

- The AI does **real work** (including the **redline draft**), not just a summary.
- **The human owns the decisions and the irreversible steps. That split is the redesign.**

4.5 Recognizing Over-Delegation

- **Over-delegation** = giving AI **more than the risk profile justifies** (e.g., letting Claude **approve clauses** or **send the contract** because it drafted them well).
- **Drafting quality is not a licence to delegate the decision.**
- If the map gives an **irreversible or high-accountability** step to AI → over-delegation (Team 2's mistake).

4.6 Common Mapping Errors (memorise all three)

Error	What happens	Fix
Halo delegation	A step goes to AI **because the previous step went well**	**Judge each step on its own**
Collapsing collaborative into automate	"AI drafts, human reviews" quietly becomes "AI drafts" because the **review gate is never staffed**	**A collaborative step with no real reviewer is an automated step**; staff the gate
Mapping the tool, not the work	Team maps around **features they like** (e.g., a Skill they built)	**Map the work first, then adjust the features**

---

5. Communicating Value and Limitations to Stakeholders

Integrating Claude means describing it to people **who didn't build it**: a **manager, a client, a risk function**. **Credibility comes from accurate claims**: communicate **limits as clearly as value**. **Overstating capability loses trust on the first visible miss.**

5.1 Describe Capability Accurately

- State what Claude **can reliably do** for this use case and what it **cannot, without inflation or false modesty**.
- [good] "Claude drafts the first pass redline, which a lawyer reviews." (accurate, credible)
- [bad] "Claude handles contract review." (overstates; invites the question your first error will answer badly)

5.2 Same Workflow, Three Audiences (contract review)

Audience	Focus	Message
Legal lead	**High literacy**: features + **failure modes**	"Claude extracts clauses, flags playbook departures, drafts the redline. It does **not approve** changes; that gate stays with you. **Known failure mode:** it can **miss obligations implied indirectly**, so flags are **a prompt for your read, not a substitute**."
Practice executive	**Outcome focused**	"Review time is **down about half** at the **same approval standard**. Every change is still approved by a lawyer before it leaves the building."
Client risk function	**Assurance focused**	"AI assists drafting; a **qualified human reviews and approves every term**. **No contract is sent without human sign-off.**"

Same workflow, same human gate. What changes is **the detail each audience needs to trust it**.

5.3 Calibrate to the Audience

- Match the message to the audience's **AI literacy**.
- **Technical stakeholder:** feature detail and **failure modes**.
- **Executive: outcome, oversight in place, risk posture**.
- The expectation you set should **match the capability boundary**, so **no one is surprised later**.
- This is the **Description** competency (Lesson 02) **applied outward**: precise specification of what the tool can and can't do, aimed at **stakeholders instead of Claude**.

5.4 Document the Human Oversight

- **Name the review gates** that stay in place: "Every output destined for a client passes human review."
- That control **makes the workflow defensible**.
- Stakeholders trust an AI workflow **more, not less**, when human checkpoints are **explicit**.

5.5 Good vs Bad Messaging

Overstated [bad]	Accurate [good]
"Our new AI system reviews contracts automatically." Sets an expectation the workflow doesn't meet and **hides the human gate**.	"Claude drafts the redline and flags playbook departures; our legal lead reviews and approves every change before anything is sent. Review time is down about half, with the same approval standard." **Value and limits in one breath.**

5.6 Phrases That Quietly Overstate

Phrase	Problem
"Fully automated"	Almost never true; **first visible error exposes it**
"Claude handles X"	**Collapses the human gate** out of the sentence
"It's basically as good as a person at Y"	Sets a standard the tool will **eventually miss publicly**

Fix every time: state what the tool does, then identify the human checkpoint.

Exam Traps

1. **"Claude drafted it well, so let it approve / send it."** Over-delegation. Drafting quality never licenses delegating the decision or an irreversible step.
2. **"The last step went well, so automate the next one too."** Halo delegation. Each step is judged on its own reversibility, stakes and accountability.
3. **"AI drafts, human reviews" with nobody actually reviewing.** That's automation in disguise. A collaborative step needs a staffed review gate.
4. **"We built a great Skill, so design the workflow around it."** Mapping the tool, not the work. Map the work first.
5. **"Accountability moves to whoever (or whatever) drafted it."** Accountability does not delegate, even when drafting does.
6. **Asking for a narrative summary of requirements.** Ask for a **structured analysis** (table, section references, answered/open, ambiguities).
7. **Stopping at extraction.** The stronger answer adds a **pressure-test pass** for ambiguous and implied requirements.
8. **Letting Claude estimate the numbers a plan depends on.** Use **code execution**; a guessed rate is a guess.
9. **Handing the final hire/budget decision to Claude.** Synthesis is Claude's strength; **judgment steps** (risk appetite, politics, budget, hiring freeze) stay human.
10. **Keeping a widely-used artifact in prompt-and-iterate mode.** Once others depend on it (uptime, security, integration), **escalate to Developer/Architect**.
11. **Running design iterations as separate one-off chats.** Use a **Project** so context and prior decisions stay stable.
12. **"Fully automated" / "Claude handles X" / "as good as a person".** All overstate. Pair the capability with the human checkpoint.
13. **Hiding the human gate to sound impressive.** Explicit oversight **increases** stakeholder trust.
14. **One message for all audiences.** Same facts and same gate; detail calibrated to AI literacy (legal lead vs executive vs risk function).
15. **Skills belong to one Project.** Skills live at the **account level** and apply everywhere; standing instructions and knowledge base live in the Project.

16. **Mixing up the two criteria sets.** Delegation mapping (this lesson) uses **Reversibility, Stakes, Accountability**. Human-review thresholds for outputs (Lesson 03) use **Stakes, Reversibility, Audience, Regulatory exposure**. Read the question to see which one it is asking about.
17. **Treating synthesis as automatically safe.** Synthesis is where Claude adds most value **and** where unverified claims can enter; Lesson 03 verification still applies.

Most Likely Questions

Q1. [Easy] Which AI Fluency Framework competency anchors workflow integration?

- A. Description
- B. Discernment
- C. Delegation
- D. Diligence

Answer: C
Why: Delegation means deciding for each step whether work is AI-appropriate, human-retained or collaborative. Description is about prompting (and here, about stakeholder messaging applied outward). Discernment and Diligence are about evaluating output.

Q2. [Medium] Two teams used Claude for contract review. Team 2 let Claude approve low-risk clauses unsupervised and later missed a harmful obligation. What was the root cause?

- A. They used the wrong model.
- B. They delegated the wrong steps: a decision step went to AI.
- C. They didn't use Artifacts.
- D. They used Projects instead of Chat.

Answer: B
Why: Same product, same process. The difference was which steps were delegated. Approval is high-stakes and accountability can't be delegated; that is over-delegation.

Q3. [Easy] A manager says, "We need better reporting." What is the best way to use Claude first?

- A. Ask Claude to build a dashboard immediately.
- B. Have Claude convert the need into specific task definitions: what report, for whom, how often, from what data, in what format.
- C. Ask Claude for a narrative essay on reporting best practice.
- D. Escalate to a Developer.

Answer: B
Why: A vague business need isn't actionable. Turning it into task definitions creates requirements you can build against and check completion on.

Q4. [Medium] A bank's proposal team has a 40-page RFP and a messy email thread. Which request gets the most useful output?

- A. "Summarize the RFP in a few paragraphs."
- B. "Extract every distinct requirement; for each give a label, the exact RFP section, whether our thread has an answer, and any ambiguity. Return as a table."
- C. "Write our RFP response."
- D. "Tell me if we should bid."

Answer: B
Why: Ask for structured analysis, not a narrative summary. The table traces each requirement to its source, shows answered/open status and flags ambiguities for clarification.

Q5. [Medium] After extracting RFP requirements, what step makes the list trustworthy?

- A. Reformat it as bullets.
- B. Ask Claude to pressure-test its own list for ambiguous, double-meaning and indirectly implied requirements.
- C. Send it straight to the client.
- D. Switch to Haiku.

Answer: B
Why: Extraction is the first pass; pressure-testing surfaces hidden requirements buried in subordinate clauses or implied by evaluation criteria. The combination is stronger than either alone.

Q6. [Hard] The RFP workflow runs as a Project. Where does each element live? (Select 3)

- A. Past winning proposals: Project knowledge base
- B. Extraction format: Project standing instructions
- C. Formatting steps: Skills, which live at the account level and apply everywhere
- D. Skills: only inside this one Project

Answer: A, B, C
Why: Knowledge base and standing instructions are Project configuration. Skills live at the account level and apply everywhere, including this Project, so D is wrong.

Q7. [Medium] An operations lead wants a headcount plan. Which workflow is best?

- A. Ask Claude to estimate a utilization rate and recommend headcount.
- B. Use code execution on four quarters of ticket data to compute growth and per-analyst throughput, then have Claude synthesize a recommendation and show assumptions.
- C. Use Research to find industry averages and hire accordingly.
- D. Let Claude make the hiring decision.

Answer: B
Why: Pair Claude's synthesis with code-executed analysis so the plan rests on computed numbers and can be defended line by line. A is a guess. D hands a judgment step to AI.

Q8. [Medium] In a planning workflow, which steps should stay human? (Select 2)

- A. Weighing the organization's risk appetite
- B. Decisions shaped by budget limits and a hiring freeze Claude can't see
- C. Structuring many considerations into options and trade-offs
- D. Computing quarterly volume growth

Answer: A, B
Why: Judgment steps (risk appetite, political reality, constraints Claude can't see) stay human. Synthesis (C) is where Claude adds the most. Computation (D) goes to code execution.

Q9. [Easy] What is the correct order of the design iteration loop?

- A. Prototype → Ideate → Refine → Gather feedback
- B. Ideate → Prototype → Gather feedback → Refine
- C. Gather feedback → Ideate → Prototype → Refine
- D. Refine → Prototype → Ideate → Gather feedback

Answer: B
Why: Ideation gives options, a prototype makes one concrete, feedback exposes what's wrong, refinement fixes it, and the loop repeats.

Q10. [Medium] Why run design iterations inside a Project?

- A. Projects make Claude write code faster.
- B. Context, constraints and prior decisions stay stable, so each iteration builds on the last instead of restarting.
- C. Projects remove the need for feedback.
- D. Projects automatically escalate to Developers.

Answer: B
Why: Stable context turns iterations into a solution rather than a pile of one-off drafts.

Q11. [Hard] A team's dashboard artifact, built in three prompt cycles, is now used daily by 400 staff, and IT asks for single sign-on and a live database link. What should the associate do?

- A. Keep iterating with prompts to add SSO.
- B. Escalate to Developer or Architect expertise, since people now rely on it as infrastructure with security and integration requirements.
- C. Rebuild it as a new artifact.
- D. Switch to Opus and continue.

Answer: B
Why: Dependency is the escalation signal. Uptime, security and integration needs put it beyond Associate scope.

Q12. [Easy] Which three criteria decide whether a workflow step is AI-appropriate, human-retained or collaborative?

- A. Speed, cost, quality
- B. Reversibility, stakes, accountability
- C. Role, context, task
- D. Audience, tone, format

Answer: B
Why: These are the delegation-mapping criteria. A is model selection. C is prompt components. D is editing.

Q13. [Medium] In the contract-review map, which steps are human-retained? (Select 2)

- A. Extract clauses
- B. Approve or reject each change
- C. Compute financial exposure of a penalty clause
- D. Sign and send

Answer: B, D
Why: Approval is high stakes and accountability doesn't delegate. Sign-and-send is irreversible, external and legally binding. Extraction is AI-appropriate; exposure calculation is AI-appropriate via code execution.

Q14. [Medium] In the onboarding map, "Personalize the welcome note" is classified as:

- A. AI-appropriate
- B. Human-retained
- C. Collaborative
- D. Code execution

Answer: C
Why: AI drafts and the hiring manager adds the human voice. Pulling HRIS details is AI-appropriate via code execution. Confirming compensation and sending the offer are human-retained.

Q15. [Medium] Which features should be embedded at AI steps in a redesigned workflow? (Select 2)

- A. A Skill for repeatable procedure steps
- B. Code execution for data steps
- C. Incognito mode for every step
- D. Research mode for clause extraction

Answer: A, B
Why: Skills give consistent procedures (better than "heroic prompting"), and code execution handles data steps. C and D don't fit.

Q16. [Hard] Claude extracted clauses and flagged playbook departures perfectly for months, so the team now lets it approve the changes too. Which mapping error(s) apply? (Select 2)

- A. Halo delegation
- B. Over-delegation
- C. Mapping the tool, not the work
- D. Collapsing collaborative into automate

Answer: A, B
Why: A step was handed over because earlier steps went well (halo), and an approval step with high accountability went to AI (over-delegation). C and D describe different errors.

Q17. [Hard] A workflow document says "AI drafts the credit memo, analyst reviews", but analysts have stopped reviewing because of workload. What has happened?

- A. Nothing; the design is still collaborative.
- B. The collaborative step has collapsed into automation; a step with no real reviewer is an automated step.
- C. Halo delegation.
- D. The Skill has failed.

Answer: B
Why: "AI drafts, human reviews" quietly becomes "AI drafts" when the review gate isn't staffed. The paper design no longer matches reality.

Q18. [Medium] A team built a strong Skill and redesigned their whole workflow around what it can do. What is the error and fix?

- A. Halo delegation; judge each step on its own.
- B. Mapping the tool, not the work; map the work first, then adjust features.
- C. Over-delegation; remove the Skill.
- D. No error; Skills should drive design.

Answer: B
Why: Mapping around a favoured feature rather than the actual workflow steps is the error. Map the work, then fit features to it.

Q19. [Medium] Which statement to a client risk function is most appropriate?

- A. "Our AI reviews contracts automatically."
- B. "AI assists drafting; a qualified human reviews and approves every term. No contract is sent without human sign-off."
- C. "Claude handles contract review end to end."
- D. "It's basically as good as a lawyer."

Answer: B
Why: The risk function is assurance-focused, so it needs the human gate stated clearly. A, C and D all overstate and hide the human checkpoint.

Q20. [Hard] Which messages fit the right audience? (Select 2)

- A. To a legal lead: "It can miss obligations implied indirectly, so flags are a prompt for your read, not a substitute."
- B. To a practice executive: "Review time is down about half at the same approval standard, and every change is still lawyer-approved."
- C. To a practice executive: a detailed list of every failure mode and feature setting.
- D. To a client risk function: "Fully automated, no human needed."

Answer: A, B
Why: High-literacy legal leads want features and failure modes. Executives want outcome, oversight and risk posture. C gives an executive the wrong level of detail. D overstates and removes the gate assurance-focused readers need.

Key Takeaways

1. **Shift from "I use Claude" to "our workflow uses Claude" deliberately.** Value compounds only when you choose the right steps to delegate.
2. **Delegation is the anchor competency.** Classify every step as AI-appropriate, human-retained or collaborative, using **reversibility, stakes and accountability**. Accountability never delegates.
3. **Requirements:** turn messy inputs into a **structured, testable** table (not a narrative), then **pressure-test** it for ambiguous and implied requirements. Run recurring work in a **Project** (knowledge base + standing instructions; Skills are account-level).
4. **Planning:** Claude excels at **synthesis**; numbers come from **code execution**. Judgment steps (risk appetite, politics, budget, hiring freeze) stay human.
5. **Design:** use the **Ideate → Prototype → Gather feedback → Refine** loop inside a Project. Escalate to **Developer/Architect** once others depend on the solution (uptime, security, integration).
6. **Redesign:** Skills for procedures, code execution for data, human steps as explicit review gates. Avoid **over-delegation, halo delegation, collapsing collaborative into automate, and mapping the tool instead of the work**.
7. **Stakeholders:** state value **and** limits, calibrate detail to AI literacy, and **name the human checkpoints**. Avoid "fully automated", "Claude handles X" and "as good as a person".

05 · Configuration & Knowledge Management (Revision Notes)

Where This Lesson Sits in the Exam

Domain	Weight	Objectives covered here
5 · Configuration and Knowledge Management	**12 %**	Configure Claude Projects with instructions and knowledge sources · Manage uploaded knowledge and connectors (e.g., Google Drive, Gmail) · Create effective system-level instructions · Inform, maintain, and update configurations, knowledge sources, and instructions

Exam Objective → Where It Is Covered

Domain 5 objective (exam guide)	Sections in these notes
Configure Projects with instructions and knowledge sources	1.1 to 1.6
Manage **uploaded knowledge and connectors** (Google Drive, Gmail)	2.1 to 2.4
Create effective **system-level instructions**	3.1 to 3.4
Maintain and update configurations, knowledge, instructions	4.1 to 4.5

Core Ideas

- **Using vs operating Claude.** *Using* = typing a good prompt today. *Operating* = building an environment where the right **context, instructions, and procedures are already in place**, so every chat starts from a **configured baseline, not a blank slate**.
- **Configuration is leverage:** set it up **once**, benefit on **every** conversation after.
- **Turns individual skill into team capability.** With a curated Project, **two people asking the same question get the same quality of answer**. Without it, everyone **re-invents context daily** and answers **drift**.
- **Second half of the discipline = maintenance.** Configurations **age**: last quarter's instruction, superseded documents in the knowledge base, an out-of-date Skill all **quietly degrade output**. Build deliberately; **review on a cadence**.

Memory Map

```
CONFIGURATION & KNOWLEDGE MANAGEMENT
│
├── USING (good prompt today) vs OPERATING (configured baseline every chat)
│      configuration = leverage · individual skill → team capability · must be MAINTAINED
│
├── 1. FOUR CONFIGURATION MECHANISMS ("right slot for each need")
│      ├── Standing instructions → BEHAVIOUR (tone, format defaults, verification habits)
│      ├── Knowledge base ........ FACTS & REFERENCE (docs, policies; no re-upload)
│      ├── Skills ................ PROCEDURES; ACCOUNT-level under CUSTOMIZE; reusable in
any Project
```

```
│     └── Scoped Memory ......... CONTINUITY; what the Project has SETTLED; isolated per
Project
│     Test: "always cite sources" = instruction · "brand hex codes" = knowledge ·
│           "format into report template" = Skill
│     Most common mistake: procedure in instructions / behaviour rule in KB
│     Memory vs KB: stable reference fact → KB · evolving record of decisions → Memory
│     PAIRING RULE: needs often span 2 slots (instruction + KB docs; Skill + brand guide)
│     One Project per client; share on Team/Enterprise under permissions
│
├── 2. CONNECTORS & UPLOADED KNOWLEDGE
│     ├── Connector = reach an external system YOU AUTHORIZE (Drive search, find email)
│     ├── Keep access DELIBERATE, not everything by default
│     ├── Boundaries: mail may SEARCH/READ but NOT SEND → confusing failure, not clear
error
│     ├── Pitfall 1: add-connector path → PUBLIC DIRECTORY, not org-vetted → check with
admin
│     ├── Pitfall 2: boundary hit looks like a BUG → fix misrouted to wrong team (field-
observed)
│     └── Uploaded knowledge: CURRENT · RELEVANT · NO DUPLICATES (3 policy versions →
wrong cite)
│
├── 3. SYSTEM-LEVEL INSTRUCTIONS THAT STICK
│     ├── Write guardrails ONCE (cite sources; "I don't know" not guessing) → inherited
by all chats
│     ├── Anticipate use cases (format, register up front → first draft near final)
│     ├── PRECISION or it SILENTLY fails ("be professional" [bad] → formal register,
│     │     define acronyms, paragraphs < 4 sentences [good])
│     └── TEST: would two different people read it the same way?
│
└── 4. MAINTAINING CONFIGURATIONS (living assets; stale = quiet degradation, no error)
      ├── Review cadence: MONTHLY for active Projects (instructions current? KB free of
      │     superseded docs? right Skills enabled?) · late signal = quality slips for no
visible reason
      ├── Skills versioning: Anthropic-built + org-provisioned = AUTO-update ·
      │     custom-uploaded = change ONLY when you RE-UPLOAD · off-format every run =
maintenance problem
      ├── Memory lifecycle: review · edit/delete stale · EXPORT backup before major
change ·
      │     FULL RESET when outdated context misleads · accuracy > volume
      ├── Configuration is ADDITIVE: missing fact → add to instructions or Memory
explicitly
      └── Audit example: renamed metric in instruction + 2 template versions + departed
            stakeholder in Memory → fix = MAINTENANCE, not a new prompt
```

---

1. Configuring Claude Projects

A Project has **several configuration slots**. The skill is **putting each piece of a recurring need into the right slot**. Choosing correctly is what makes a Project run smoothly.

1.1 The Four Configuration Mechanisms

Mechanism	Holds	Rule of thumb	Key facts
Standing instructions	How Claude should **behave** across all chats in the Project: **tone, format defaults, verification habits**	**Behaviour, not facts**	Inherited by every conversation in the Project
Knowledge base	**Documents, policies, reference files** Claude draws on **without re-uploading**	**Facts and reference, not behaviour**	Keep current, relevant, de-duplicated
Skills	**Repeatable procedures** followed consistently for a task type	**Procedure, not one-off instructions**	Live at the **account level under Customize**, **not inside any one Project**; **reusable across any Project**
Scoped Memory	**Continuity** within the Project	**What the Project has already settled**	**Kept separate from other Projects**, so context doesn't bleed between workstreams

One-line summary: Instructions govern **behaviour**, knowledge base holds **facts**, Skills carry **procedures**, scoped Memory keeps **continuity**.

1.2 Choosing the Right Mechanism: "Instruction, knowledge, or Skill?"

Need	Example	Slot
Rule about **behaviour**	"Always cite sources"	**Standing instruction**
Fact Claude needs	"Our brand palette is these hex codes"	**Knowledge base**
Multi-step procedure	"Format findings into our standard report template"	**Skill** (built once at account level under Customize, reused across Projects)

- **Most common configuration mistake:** putting a **procedure into the instructions**, or a **behaviour rule into the knowledge base**. It makes the Project **harder to maintain**.

1.3 Worked Example: Client Account Workspace (one Project per client)

Slot	Client A content
Standing instructions	"Write in a formal register. Always cite the source document for any factual claim. Flag anything you are unsure of rather than guessing."
Knowledge base	Client's **brand guide**, current **statement of work**, **last three status reports**
Skill (account-level, reused here)	Firm's **status-report formatter**; lives under **Customize**, available to any Project, so every weekly update has the same structure

Slot	Client A content
Scoped Memory	Client's **stakeholder names and standing preferences**, which never appear in other clients' Projects

- On **Team or Enterprise**, the Project can be **shared with the engagement team under permissions**, so everyone works from the **same configured baseline**.
- **Project-scoped Memory guarantees** Client A's context doesn't appear in a Client B conversation. This separation makes **one-Project-per-client safe**.

1.4 Scoped Memory as a First-Class Mechanism

- Easy to treat as an afterthought, but in a Project it is **a configuration slot like any other**.
- How the slots divide the work: instructions say **how Claude should behave**; the knowledge base says **what is true**; Skills carry **reusable procedures** (account level); scoped Memory holds **what the Project has already settled**.
- Holds what the Project has **already settled**: **stakeholder names, standing preferences, decisions from earlier conversations** you don't want to restate.
- **"Scoped" is the key word**: sectioned off from your other Projects, so context **never appears in another** client or workstream.
- That **isolation makes Memory safe** for **sensitive or client-specific continuity**; Claude remembers what it needs **without revealing information to the wrong audience**.

Memory vs knowledge base: decide by type

Type of information	Goes in
Stable reference fact	**Knowledge base**
Evolving record of Project decisions	**Memory**

1.5 The Pairing Rule (when a need spans two mechanisms)

- The cleanest configurations **rarely map a need to a single slot**.
- **Most common pattern: a behaviour rule + the facts it acts on.**
- "Always cite the source document" = **standing instruction**; the documents it cites = **knowledge base**.
- **Neither works alone:** the instruction has **nothing to cite** without the documents; the documents get used **inconsistently** without the instruction.
- **Skills pair the same way:** a status-report **Skill** carries the procedure; the **brand guide** it formats against sits in **knowledge**.
- When configuring, ask: **which slot does this need belong in, and does it need two slots wired together?**

1.6 Quick Slot-Sorting Practice

Item	Slot
"Say 'I don't know' rather than guessing"	Standing instruction

Item	Slot
Current statement of work	Knowledge base
Firm's weekly status-report procedure	Skill
"The client's CFO prefers figures in a one-page appendix" (settled preference)	Scoped Memory
"We agreed last week to drop the Q2 comparison" (evolving decision)	Scoped Memory
Approved product price list	Knowledge base

---

2. Connectors and Uploaded Knowledge

Connectors extend Claude's reach into the data you already work in, **like Google Drive and Gmail**. They are **powerful and have boundaries**. Manage them as **curated sources** and know **exactly what each can and cannot do**.

2.1 Connecting External Sources

- A connector lets Claude reach an **external system you authorize**, e.g. **search your Drive** for a document or **find a relevant email**.
- **You manage what is accessible.** Keep that set **deliberate**, **not connecting everything by default**.

2.2 Capability Boundaries

- Each connector has a **defined boundary**; knowing it **prevents wasted time**.
- Example: a **mail connector may let Claude search and read messages, but not send them**.
- Expecting an action a connector can't perform gives a **confusing failure, not a clear error**.
- **Learn each connector's boundaries before you build a workflow on it.**

2.3 Two Field-Observed Pitfalls

Pitfall	What happens	Fix
Wrong path to add a connector	The obvious add-connector path can route to a **public directory** instead of your **organization's vetted connectors**	**Confirm the right path with your admin**, especially on **Team or Enterprise**, so you connect the **approved source, not a look-alike**
Boundary confusion misroutes fixes	A connector hitting its boundary can **look like a bug** (reported from **field use, not documented product behaviour**); reports go to the **wrong team** and the fix **stalls**	**Know where each connector's capability ends** so the issue isn't mislabelled

2.4 Keeping Uploaded Knowledge Current

- Uploaded knowledge needs the **same care as a connected source**: **current, relevant, free of duplicates**.
- **Three versions of the same policy invite Claude to cite the wrong one.**

- Curate it **like a shared drive**: **remove deprecated versions as you add new ones**.

---

3. System-Level Instructions That Stick

Teams keep asking: *can we bake the guardrails in once instead of re-typing them every conversation?* **Persistent (standing) instructions are exactly that mechanism.** Write **verification behaviours, format defaults, and tone once**; **every conversation in the Project inherits them**.

3.1 Write the Guardrails Once

- **Highest-value instructions = the ones you'd otherwise re-type constantly.**
- Example verification instruction: "Cite the source document for every factual claim, and say 'I don't know' rather than guessing when the documents do not cover something."
- That discipline then applies to **every conversation without anyone remembering to ask**.

3.2 Anticipate the Use Cases

- Good standing instructions embed **format, tone, and guardrail guidance ahead of need**.
- If the Project produces **client deliverables**, specify **preferred format and register up front**, so the **first draft lands closer to final** instead of needing the **same corrections each time**.

3.3 Precision, or It Silently Fails

- **Vague instructions don't announce failure; they just quietly don't work.**
- "Be professional" gives Claude **almost nothing** to act on.
- "Use a formal register, define any acronym on first use, and keep paragraphs under four sentences" is **precise enough to change the output**.
- **Test of an instruction: would two different people read it the same way?**

3.4 Before and After

	Instruction	Result
Vague	"Make the reports good and accurate."	Quality **varies chat to chat**; nothing concrete changed
Precise	"For every figure in a report, state its source. If a figure is not in the provided data, mark it 'unverified' rather than including it. Lead each report with a one-sentence headline."	Verification behaviour is **consistent**; the headline **appears every time**

---

4. Maintaining Configurations

Configurations are **living assets**. **Instructions, knowledge, Skills, and Memory all drift toward stale**, and stale configuration **degrades output quietly, with no error to alert you**. **Scheduled maintenance** catches decay **before it reaches a deliverable**.

4.1 Review Cadence

- Set a **recurring review for each active Project**. Ask:
  1. Do the **standing instructions still match the current process**?
  2. Is the **knowledge base free of superseded documents**?
  3. Are the **right Skills enabled**?
- **A monthly pass for active Projects catches most drift.**
- **Signal you waited too long: output quality slipping for no visible reason**.

4.2 Skills Versioning

Skill type	How it updates
Anthropic-built Skills	**Update automatically**
Organization-provisioned Skills	**Update automatically**
Your own custom-uploaded Skills	**Change only when you re-upload them**

- Watch for a **misconfigured or out-of-date Skill** degrading output.
- A Skill that **silently produces a slightly off format every run** is a **maintenance problem, not a prompting problem**.

4.3 Memory Lifecycle

- Treat Memory **like a working file**.
- **Review periodically**; **edit or delete stale entries**.
- **Export it as a backup before a major change.**
- When Memory has accumulated **enough outdated context to mislead**, a **full reset** is the right call.
- **Accuracy of what's stored matters more than volume.**

4.4 Configuration Is Additive

- If Claude **hasn't automatically captured** a fact that matters (a **constraint, a preference, a piece of background**), **add it explicitly** to **standing instructions or Memory** rather than **re-supplying it each session**.

4.5 Worked Example: Auditing a Degraded Setup

Symptom: a recurring report Project starts producing **subtly wrong output**.

Cause found by the maintenance checklist	Slot	Fix
Standing instruction references a **metric renamed last quarter**	Instructions	**Update the instruction**
Knowledge base holds **two versions of the template**	Knowledge base	**Remove the old template**
Memory entry records a **stakeholder who has left**	Memory	**Delete the stale entry**

- **The fix is maintenance, not a new prompt.** Output returns to standard **without changing how anyone prompts**.

Links to Other Lessons (Configuration Points Tested Here)

Earlier point	Lesson	Why it matters for Domain 5
A Project holds **standing instructions, knowledge base, own conversation history**; chats in a Project **don't share context with each other**	01	Basis of Project configuration
Memory settings: **"Search and reference chats"** (retrieves past chats on request, no profile) vs **"Generate memory from chats"** (the actual Memory feature)	01	Know which toggle does what
Project Memory needs no separate setting; it's on by default and distinct per Project	01	Supports one-Project-per-client
Incognito keeps standalone chats out of Memory and history; **doesn't override org data retention**	01	Sensitive work outside Projects
Memory import from other AI platforms: experimental; Free, Pro, Max, Team (not Enterprise); fallback = **add key facts to Memory manually, not to a Project knowledge base**	01	Slot choice for imported facts
Memory review **at least monthly** for active users	01	Same cadence as Project reviews
Skills **reduce but don't eliminate** variance; review stays in	01	A good Skill doesn't remove review
RFP Project: past proposals = knowledge base, extraction format = standing instructions, formatting = **account-level Skills**	04	Worked slot example

---

Exam Traps

1. **Putting a multi-step procedure in standing instructions.** Procedures belong in a **Skill**. Behaviour rules go in instructions, facts in the knowledge base.
2. **Putting a behaviour rule ("always cite sources") in the knowledge base.** It's a **standing instruction**. This and trap 1 are named as the **most common configuration mistake**.
3. **"Skills are configured inside each Project."** Skills live at the **account level under Customize** and are **reusable across any Project**.
4. **Choosing a single slot when the need spans two.** "Cite the source" needs the **instruction + the documents in the knowledge base**; a report Skill needs the **brand guide in knowledge**. Look for the **pairing** answer.
5. **Stable facts in Memory, or evolving decisions in the knowledge base.** Stable reference fact → **knowledge base**; evolving record of decisions → **Memory**.
6. **"One shared Project for all clients is fine if we tell Claude to keep them apart."** Use **one Project per client**; **project-scoped Memory** is what guarantees separation.
7. **"Connect everything so Claude has maximum access."** Keep connector access **deliberate**; you manage what's accessible.

8. **"The mail connector can send the reply."** Not necessarily: a mail connector may **search and read but not send**. Learn boundaries before building on them.
9. **Treating a connector boundary as a product bug.** It can look like a bug; misreporting sends the fix to the **wrong team**.
10. **Using the obvious add-connector path on Team/Enterprise without checking.** It may lead to a **public directory**; confirm the **org-vetted path with your admin**.
11. **Keeping old policy versions "just in case".** Duplicates invite Claude to cite the wrong one; **remove deprecated versions** as you add new ones.
12. **"Be professional" / "make it good and accurate" as instructions.** Too vague; they **fail silently**. Use precise, testable wording that **two people would read the same way**.
13. **Fixing a degraded Project by writing better prompts.** If the cause is stale instructions, duplicate templates or outdated Memory, the fix is **maintenance**.
14. **"All Skills update themselves."** Only **Anthropic-built and org-provisioned** Skills auto-update; **custom-uploaded Skills change only when re-uploaded**.
15. **Waiting for an error message to signal stale config.** There is **no error**; the signal is **quality slipping for no visible reason**. Review **monthly**.
16. **Deleting Memory before a major change without a backup. Export first**; **full reset** only when outdated context would mislead.
17. **Re-pasting a missing constraint each session.** Configuration is **additive**: add it to instructions or Memory once.

Most Likely Questions

Q1. [Easy] What is the difference between "using" and "operating" Claude?

- A. Using means paid plans; operating means free plans.
- B. Using means writing a good prompt today; operating means building an environment where context, instructions and procedures are already in place.
- C. Operating means writing code against the API.
- D. There is no difference.

Answer: B
Why: Operating builds a configured baseline so every conversation starts ready, which is leverage. C describes developer work, which is outside this credential.

Q2. [Easy] Match the need to the slot: "Always cite sources."

- A. Knowledge base
- B. Standing instruction
- C. Skill
- D. Scoped Memory

Answer: B
Why: It's a rule about behaviour. Standing instructions govern behaviour, not facts.

Q3. [Easy] Match the need to the slot: "Our brand palette is these hex codes."

- A. Standing instruction
- B. Knowledge base
- C. Skill
- D. Connector

Answer: B
Why: It's a fact Claude needs. The knowledge base holds facts and reference, not behaviour.

Q4. [Medium] Where do Skills live, and what does that mean?

- A. Inside each Project; they must be rebuilt for every Project.
- B. At the account level under Customize; a Skill you build is reusable across any Project that needs it.
- C. In the knowledge base as documents.
- D. In Memory, as remembered procedures.

Answer: B
Why: Skills are not inside any one Project. Build once at the account level and reuse anywhere.

Q5. [Medium] What is described as the most common configuration mistake? (Select 2)

- A. Putting a procedure into the standing instructions
- B. Putting a behaviour rule into the knowledge base
- C. Creating one Project per client
- D. Enabling project-scoped Memory

Answer: A, B
Why: Mixing up slots makes the Project harder to maintain. C and D are recommended practices.

Q6. [Medium] Which belongs in scoped Memory rather than the knowledge base?

- A. The client's signed statement of work
- B. The company's brand guide
- C. A decision made last week to drop a section from future reports
- D. The approved price list

Answer: C
Why: Stable reference facts go in knowledge. An evolving record of Project decisions goes in Memory.

Q7. [Hard] A team adds "Always cite the source document for factual claims" as a standing instruction, but never uploads the documents. What is wrong?

- A. Nothing; the instruction is enough.
- B. The pairing rule is broken: the instruction has nothing to cite without the documents in the knowledge base.

- C. The instruction should be a Skill.
- D. Citing should be stored in Memory.

Answer: B
Why: Many needs span two slots. A behaviour rule needs the facts it acts on, and documents without the rule are used inconsistently.

Q8. [Medium] A consultant works for two competing banks. What setup prevents Bank A context from appearing in Bank B work?

- A. One shared Project with a standing instruction to keep them separate
- B. One Project per client, relying on project-scoped Memory
- C. Turning Memory off entirely
- D. Putting both clients' documents in one knowledge base

Answer: B
Why: Project-scoped Memory guarantees separation, which makes one-Project-per-client safe. A relies on an instruction instead of structural isolation.

Q9. [Medium] On a Team plan, how can an engagement team all work from the same configured baseline?

- A. Each member copies the instructions into their own chats.
- B. Share the Project with the team under permissions.
- C. Put the instructions in each person's Memory.
- D. Email the knowledge base files to everyone.

Answer: B
Why: On Team or Enterprise, a Project can be shared with the team under permissions, so everyone uses the same instructions, knowledge and configuration.

Q10. [Medium] An analyst expects Claude to send replies through the Gmail connector, but nothing happens and the failure is confusing. What is the most likely explanation?

- A. The model is too small.
- B. The connector may allow search and read but not send; this is a capability boundary.
- C. Gmail is not supported.
- D. Memory is full.

Answer: B
Why: Each connector has a defined boundary, and hitting it produces a confusing failure rather than a clear error. Learn the boundaries before building workflows on them.

Q11. [Hard] On an Enterprise plan, a user clicks the obvious add-connector option and connects a tool with a familiar name. What risk does the lesson highlight?

- A. Connectors slow Claude down.

- B. The path may route to a public directory rather than the organization's vetted connectors, so the user may connect a look-alike.
- C. Enterprise plans can't use connectors.
- D. The connector will delete files.

Answer: B
Why: Confirm the right path with your admin, especially on Team or Enterprise, so you connect the approved source.

Q12. [Medium] A connector keeps "failing", and the team files bug reports with IT, which go nowhere. What does the lesson suggest is happening?

- A. A genuine product defect
- B. Boundary confusion: the connector hit its capability limit, which looks like a bug, so the fix was misrouted
- C. A Memory conflict
- D. Usage limits

Answer: B
Why: This is a field-observed pattern. Knowing where each connector's capability ends stops the issue being mislabelled and sent to the wrong team.

Q13. [Easy] A knowledge base holds three versions of the same credit policy. What is the risk?

- A. Claude will refuse to answer.
- B. Claude may cite the wrong version.
- C. The Project will be deleted.
- D. None; more versions give more context.

Answer: B
Why: Keep uploaded knowledge current, relevant and free of duplicates. Remove deprecated versions as you add new ones.

Q14. [Medium] Which standing instruction is precise enough to change output reliably?

- A. "Be professional."
- B. "Make the reports good and accurate."
- C. "Use a formal register, define any acronym on first use, and keep paragraphs under four sentences."
- D. "Do your best."

Answer: C
Why: Vague instructions fail silently. The test is whether two different people would read it the same way, and only C passes.

Q15. [Medium] A reporting Project's standing instructions say: "For every figure, state its source. If a figure is not in the provided data, mark it 'unverified' rather than including it. Lead each report with a one-sentence headline." What does this achieve?

- A. It guarantees every figure is correct.
- B. It makes the verification behaviour and headline consistent in every conversation.
- C. It replaces the need for a knowledge base.
- D. It turns on code execution.

Answer: B
Why: Precise guardrails written once are inherited by every chat. They improve consistency but don't guarantee correctness (A), and they still need the data in the Project (C).

Q16. [Medium] How often should active Projects be reviewed, and what is the sign you waited too long?

- A. Yearly; an error message appears.
- B. Monthly; output quality slips for no visible reason.
- C. Never; configurations don't age.
- D. Daily; Claude asks for a review.

Answer: B
Why: A monthly pass catches most drift. Stale configuration degrades quietly, with no error to alert you.

Q17. [Hard] Which Skills update automatically? (Select 2)

- A. Anthropic-built Skills
- B. Organization-provisioned Skills
- C. Your own custom-uploaded Skills
- D. Skills stored in the knowledge base

Answer: A, B
Why: Custom-uploaded Skills change only when you re-upload them. Skills aren't stored in the knowledge base (D).

Q18. [Hard] A weekly report Skill produces a slightly wrong format every run. Prompt tweaks don't help. What is the correct diagnosis?

- A. A prompting problem; write longer prompts.
- B. A maintenance problem: a misconfigured or out-of-date Skill.
- C. A model problem; switch to Opus.
- D. A connector problem.

Answer: B
Why: A Skill that silently produces an off format every run is a maintenance problem, not a prompting problem. Fix or re-upload the Skill.

Q19. [Hard] A recurring report Project produces subtly wrong output. The audit finds a renamed metric still referenced in the instructions, two template versions in the knowledge base, and a Memory entry for a stakeholder who left. What is the fix? (Select 3)

- A. Update the standing instruction to the new metric name
- B. Remove the old template from the knowledge base
- C. Delete the stale Memory entry
- D. Ask users to write more detailed prompts

Answer: A, B, C
Why: The fix is maintenance, not a new prompt. Output returns to standard without changing how anyone prompts.

Q20. [Hard] Before restructuring a long-running Project, a manager notices its Memory is full of outdated context that keeps misleading Claude. What is the best sequence?

- A. Delete everything immediately.
- B. Export Memory as a backup, then do a full reset and add back what's still accurate.
- C. Ignore it; more memory is better.
- D. Move all Memory into the knowledge base.

Answer: B
Why: Export as a backup before a major change; a full reset is right when outdated context misleads. Accuracy matters more than volume. A skips the backup. D mixes up the slots.

Key Takeaways

1. **Operate, don't just use.** Configuration gives every conversation a ready baseline, turns individual skill into team capability, and needs **regular maintenance**.
2. **Four slots:** Instructions = **behaviour**. Knowledge base = **facts**. Skills = **procedures** (account level under Customize, reusable). Scoped Memory = **continuity** (settled decisions and preferences, isolated per Project).
3. **Ask "instruction, knowledge, or Skill?"** Misplacing procedures or behaviour rules is the most common mistake. Stable facts go in the knowledge base; evolving decisions go in Memory. Many needs **pair two slots**.
4. **One Project per client**, shared under permissions on Team/Enterprise. Project-scoped Memory keeps clients apart.
5. **Connectors:** authorize deliberately, learn each one's boundaries (mail may read but not send), add through the **org-vetted** path, and don't mistake a boundary for a bug.
6. **Uploaded knowledge:** current, relevant, no duplicates. Remove old versions as you add new ones.
7. **Standing instructions:** write guardrails once, anticipate use cases, and be **precise**. Vague instructions fail silently. The test: would two people read it the same way?

8. **Maintain monthly.** Only Anthropic-built and org-provisioned Skills auto-update; custom ones need re-uploading. Curate Memory (export before big changes, reset when misleading). Add missing facts explicitly. When output degrades, **fix the configuration, not the prompt**.

06 · Governance, Risk & Responsible Use (Revision Notes)

Where This Lesson Sits in the Exam

Domain	Weight	Objectives covered here
6 · Governance, Risk, and Responsible Use	**15%** (3rd largest)	Identify appropriate and inappropriate use cases · Apply data sensitivity, regulatory, and privacy considerations · Follow organizational AI policies and governance standards · Understand the ethical implications of AI usage

Exam Objective → Where It Is Covered

Domain 6 objective (exam guide)	Sections in these notes
Identify **appropriate and inappropriate use cases**	1.1 to 1.5
Apply **data sensitivity, regulatory, and privacy** considerations	2 (Skill/feature risk), 3.1 to 3.6
Follow **organizational AI policies** and governance standards	4.1 to 4.4
Understand the **ethical implications** of AI usage	5.1 to 5.5

Core Ideas

- **One inappropriate use case can freeze an entire organization's AI program**: sensitive data uploaded to the wrong place, an untrusted Skill with broad access, a quiet policy violation at the wrong moment. A freeze costs **every team** the productivity it had gained.
- **Governance keeps adoption moving safely.** It is exercised **by practitioners, one decision at a time**, not by a policy binder on a shelf.
- **Governance is a practitioner skill.** Policy sets the boundary; **you decide in the moment** whether this use case, Skill, or upload is appropriate.
- **Two AI Fluency competencies:**
- **Diligence** = ownership and verification.
- **Delegation** = the criteria for judging whether a use case is appropriate at all.

Memory Map

```
GOVERNANCE, RISK & RESPONSIBLE USE
│
├── WHY: one bad use case can FREEZE the whole AI program
│     governance = practitioner skill, one decision at a time
│     DILIGENCE (ownership, verification) + DELEGATION (is the use case appropriate?)
│
├── 1. USE-CASE SCREENING (structured, not gut feel)
│     ├── 4 criteria: REVERSIBILITY · CONSEQUENCE OF ERROR ·
│     │     NEED FOR HUMAN CREATIVITY/EMPATHY · ACCOUNTABILITY
│     ├── Criteria INTERACT → name the LOAD-BEARING one
│     │     (the one that, if changed, moves the classification)
│     ├── 3 classes: FULLY APPROPRIATE · APPROPRIATE WITH HUMAN REVIEW · INAPPROPRIATE
│     ├── "With human review" → DEFINE THE GATE: WHO (accountable role) ·
│
```

```
│       │     WHAT (specific risk) · WHEN (before use). Can't state it → not ready
│       └── Portfolio: FAQ = fully · resume shortlist = review + bias safeguards ·
│             final medical/legal determination = inappropriate ·
│             billing-complaint replies = review, load-bearing = ACCOUNTABILITY
│
├── 2. SKILL TRUST & FEATURE RISK (a Skill is software)
│       ├── Check: SOURCE · REACH (inherits session access; audit bundle) · APPROPRIATENESS
│       ├── Internal ≠ vetted (sister-department software)
│       ├── Outcomes: ENABLE · ESCALATE (admin/security) · DECLINE
│       └── Any feature (connector, tool, integration): who provides · what it accesses ·
│             proportional? → LEAST PRIVILEGE, revisit when job changes
│
├── 3. DATA SENSITIVITY & FEATURE CONTROLS
│       ├── CLASSIFY BEFORE UPLOAD; unsure → ask; between tiers → treat as MORE sensitive
│       │     GREEN safe · YELLOW review first · RED keep out unless approved path
│       ├── Redaction: when task doesn't need identifiers ("Customer 1, 2")
│       │     Failures: PARTIAL redaction (account no., rare title, date) ·
│       │     redaction that BREAKS THE TASK → approved path or keep out
│       ├── Controls: code-exec SANDBOX · MEMORY persistence · INCOGNITO
│       │     (out of history + Memory, NOT out of org retention/exports) ·
│       │     ORG MEMORY: Team = none · Enterprise = Owners/Primary Owners
│       ├── Match: Green → none · Yellow (no persistence) → Incognito ·
│       │     Red → approved entry point FIRST
│       └── INCOGNITO DOES NOT MAKE RED DATA SAFE ("allowed here?" comes first)
│
├── 4. POLICY & DILIGENCE AS A HABIT
│       ├── Followed only when watched ≠ governance; risk lives in the policy/practice gap
│       ├── Apply on ROUTINE low-visibility decisions (drift accumulates there)
│       ├── AUDIT usage vs policy → close gaps · STAY CURRENT (policy + features change)
│       └── Mini audit: unreleased spec to non-approved entry point · unvetted Skill ·
│             skipped review gate → all DRIFT, not malice → habit-level fixes
│
└── 5. ETHICS: BIAS · FAIRNESS · TRANSPARENCY (part of routine review)
      ├── Bias from prompt, framing, language patterns; highest risk in people-facing
work
      ├── Disclosure: depends on setting/audience/policy; unsure → DISCLOSE
      ├── Ambiguous cases: WHO affected · WHAT could go wrong · FAIR outcome ·
      │     DISCLOSURE needed → document reasoning
      ├── Escalate to governance/ethics function: large population · significant harm ·
      │     outside team's standing → bring REASONING, not a verdict
      └── Performance-review summaries → appropriate with human review + fairness check
```

---

1. Appropriate vs Inappropriate Use Cases

Deciding whether a use case is appropriate is a **structured evaluation, not a gut feeling**. The same **Delegation** criteria that map workflow steps also **screen whole use cases**. Running them every time turns **vague unease into a defensible call**.

1.1 Delegation Criteria for Screening (four)

Criterion	Question to ask
Reversibility	Can a wrong output be **caught and undone before it causes harm**? **Irreversible consequences raise the bar sharply.**
Consequence of error	What is the cost if wrong? Higher consequence → **more human control**, or **rules the use case out**

Criterion	Question to ask
Need for human creativity or empathy	Does it need **judgment, relationship, or care** AI cannot supply? Some work **stays human regardless of capability**
Accountability	Who is answerable, and **can that accountability be exercised over an AI-produced result**?

1.2 The Criteria Interact: Name the Load-Bearing One

- The four criteria are **not a checklist where any single failure ends the discussion**. They **interact**.
- A task can be **low-consequence and reversible and still need human review**. **Accountability cannot be transferred to a model.**
- Example: a **condolence note to a client** is low-stakes and fully reversible, but the **relationship element** means **a person should own it**.
- A task can be **high-consequence but still appropriate with the right gate**.
- Example: a **financial summary feeding a decision** is consequential, but if a **named reviewer signs off before it's used**, **accountability and reversibility are restored**.
- **Practical rule:** run **all four**, then ask which one is **load-bearing** for this use case.
- **Load-bearing criterion** = the one that, **if changed, would move the use case between classifications**.
- **Naming it makes your classification defensible** to a risk or compliance reviewer.

1.3 Three Classifications (always with documented rationale)

Classification	When	What to do
Fully appropriate	Reversible, low consequence, no special human element	**Delegate with normal review**
Appropriate with human review	Useful for AI assistance, but **stakes or accountability need a human gate**	**Define the gate explicitly**
Inappropriate	Consequence, irreversibility, or human-element requirement means AI should not do it	**Articulate why** and **name the human role that must own it**

1.4 The Gate Is the Classification

- "Appropriate with human review" is the classification **people most often get wrong**: they **stop at the label and never specify the gate**.
- The gate is **not** "someone will check it."
- **A defined gate states three things:**

Element	Meaning
Who reviews	The **role with the accountability**, not whoever is free
What they verify	The **specific risk** the review exists to catch (factual accuracy, fairness, policy compliance)

Element	Meaning
When it happens	**Before the output is used**, not after

- Defined gate: "A **manager** reviews the shortlist **for adverse-impact patterns before any candidate is contacted**."
- Not a gate: "We will keep a human in the loop."
- **If you can't state the gate in who/what/when form, the use case is not ready to run.**

1.5 Worked Example: A Use-Case Portfolio

Use case	Classification	Reasoning
Draft internal FAQ from **approved policy docs**	**Fully appropriate**	Reversible, low stakes, **factual grounding available**
Summarize candidate resumes to a **shortlist**	**Appropriate with human review, only with bias safeguards**	Consequential, **fairness-sensitive**; accountability rests with the **hiring manager**, who **owns the selection**
Generate a **final medical or legal determination**	**Inappropriate**	**Irreversible consequence** + **professional accountability that can't transfer to a tool**; the human professional makes and owns it
Draft **customer-facing responses to billing complaints** (the hard one)	**Appropriate with human review**; **load-bearing criterion = accountability** (the company owns what it tells a customer about their money)	Reversible **if reviewed before sending**; moderately consequential (erodes trust, **can have regulatory weight** in some industries); partly relationship-sensitive

- Billing-complaint **gate**: **a support agent** reads each drafted response **for accuracy against the actual account record** and **adjusts tone before it sends**.
- The interesting governance work is **naming the gate for the cases in the middle**, not the obvious yes/no ones.
- **The deliverable is articulating why** a use case fails and **what human role must be retained**. "It feels risky" doesn't travel; "**irreversible consequence plus non-transferable accountability**" does.

---

2. Skill Trust and Feature-Level Risk

A Skill is software. It can access **what you give Claude access to during a session** and can **take actions through code execution**. A Skill from an **untrusted source is a real risk**. The answer is a **source-and-permissions check before enabling it**.

2.1 Why Untrusted Skills Are a Risk

- A Skill runs procedures and can touch **data and tools available in the session**; one from an unknown source could **mishandle data or take unintended actions**.
- "Skills are a black box" is a common worry. The answer is **neither blind trust nor blanket bans**, but a **repeatable trust evaluation**.

2.2 Trust Evaluation Before Enabling (three checks)

Check	Question	Key points
Source	Who published it?	**Anthropic-provided and internally approved** Skills = **lower-risk starting point**; **unknown third-party** = **more scrutiny**
Reach	What could it reach in the session where it will run, and is that **proportional to the task**?	**Skills don't request permission**; a Skill **inherits whatever access the session already has**. **Audit the bundle** (instructions, dependencies, bundled files) before enabling. **A formatting Skill whose instructions roam far beyond formatting is a red flag**
Appropriateness	Is it the right tool at all, or **more capability than the job needs**?	

2.3 Internal Does Not Mean Vetted

- Source isn't a simple trusted/untrusted switch. **Hardest case: a Skill built by another team in your own organization.**
- "Internal" feels safe but **isn't vetted**: the team may have given it **broad access for their own convenience** or built it **against an older policy**.
- Treat it like **software from a sister department**: **confirm with the publisher what it accesses and why**, and check its access **still matches current policy**, before enabling it on your data.
- **The trust question: "Do I know what it does, and does its access match the job?"**

2.4 Three Outcomes from a Trust Check

Failing the check doesn't always mean "never use it". It means **don't enable it on your authority alone**.

Outcome	When
Enable	Source, access (permissions), and appropriateness **all clear**
Escalate	Useful, but **source unknown** or **access looks broad** → route to **admin or security function**
Decline	Access **clearly disproportionate** or **source can't be established**, and **no review would change that**

The judgment = identifying which of the three applies.

2.5 Worked Example: Vetting Two Skills

Skill	Assessment	Call
Anthropic-provided document formatter	Trusted source; access matches the task (document handling); appropriate	**Enable**

Skill	Assessment	Call
Third-party "analytics booster", unknown publisher, wanting **broad data access**	Unknown source; **disproportionate access** for stated purpose	**Don't enable without your organization's review**; treat like **unvetted installed software**

- **General rule:** evaluate a Skill's source and access **like any software you'd install on a work machine**.

2.6 Beyond Skills: Feature-Level Risk and Least Privilege

- Skills are the **sharpest** case, but the same habit applies to **any capability that can read or act on your data**: a **connector, a tool, an integration**.
- Ask the same three questions: **who provides it**, **what does it access**, **is that access proportional**?
- **Principle of least privilege:** grant the **narrowest access** that gets the job done, and **revisit it when the job changes**.

---

3. Data Sensitivity, Privacy & Feature Controls

Before any data enters any feature, know its sensitivity. Classification prevents leakage; feature controls (**Incognito, Memory management, sandbox awareness**) are how you act on it when persistence or processing isn't appropriate.

3.1 Classify Before You Upload

- Sort data into **safe to use**, **needs review first**, and **stays out entirely**.
- Public and internal low-sensitivity material is generally fine; **regulated, confidential, or personal data** needs a **deliberate check against policy** first.
- **Unsure? The safe default is to ask before uploading.**

Tier	Meaning	Examples
Green: Safe to use	Use freely	**Published material**, **anonymized or aggregated data**, internal documents **already cleared for wide sharing**
Yellow: Review first	Check policy before use	Internal docs **not meant to leave the company**, anything with **names or contact details**, **draft material tied to an unannounced deal or product**
Red: Keep out unless an approved path exists	Don't upload without a confirmed approved path	**Regulated data (health, financial, government)**, **credentials and secrets**, anything under a **confidentiality obligation to a third party**

- **Unsure between two tiers? Treat it as the more sensitive one** until confirmed otherwise.

3.2 Redaction and Anonymization

- When the task **doesn't need the sensitive specifics, remove them**.
- Redacting **names, account numbers, identifiers** before upload gives the **analytical value without exposing the data**.

- This habit prevents the **most common form of accidental leakage**.
- Works **when the task doesn't depend on the identifiers**: trends across a customer list don't need names; "Customer 1, Customer 2" loses nothing.

Two redaction failure modes

Failure	What goes wrong	Fix
Partial redaction	Removing the name but leaving an **account number, rare job title, or specific date** can still identify someone, **especially in a small population**	**Strip every field that could lead to identification**
Redaction that breaks the task	The work **genuinely needs** the sensitive specifics	Redaction isn't the answer: **confirm an approved path** or **keep it out**. Redaction is for when **sensitivity and necessity don't overlap**; it **doesn't make any data safe**

3.3 Feature-Specific Controls

Control	What it does	Practitioner note
Code execution sandbox	Code runs in a **sandboxed environment**; **uploaded files are processed there**	**Review what you upload** before running analysis
Memory persistence	Carries information **across sessions**	For sensitive work, that persistence may be **exactly what you don't want**
Incognito mode	Keeps the session **out of chat history and Memory**	**Still follows org data-retention policies** and **can appear in organizational data exports**. **Memory exclusion and data retention are separate controls.** Use for sensitive conversations or confidential inputs that shouldn't surface in history or Memory
Org-level Memory controls	**Team plans: no org-level controls** for memory. **Enterprise: Owners and Primary Owners** hold org-wide Memory controls, **including disabling memory for the organization**	Know **your plan** and **who controls the setting**; verify current behaviour in the Claude Help Center memory article (support.claude.com)

3.4 Classify First, Then Pick the Matching Control

Tier	Control
Green	**No special control**

Tier	Control
Yellow that shouldn't persist (confidential, not regulated)	**Incognito**: analysis now, not in Memory or chat history (**org retention still applies**)
Red	**Approved entry point confirmed before anything is uploaded, regardless of Memory settings**

- **Common error: using Incognito and assuming it makes sensitive data safe.**
- **Incognito does not make Red data safe.** It controls **whether something is remembered**, not **whether the data was allowed here in the first place**. For regulated data, **"is it allowed here?" comes first**.

3.5 Awareness Across Entry Points

- Different claude.ai entry points **may handle data retention differently** depending on **how your organization configured Claude**.
- You don't need the technical detail; you need the **habit of asking before uploading** when unsure about an entry point.
- **Plain rule: know your data's sensitivity first, then match the feature and its controls to it.**

3.6 Worked Example: Four Data Decisions

Data	Tier	Decision
Anonymized survey data for trend analysis	**Green**	Safe to upload; **use code execution for verified counts**; no identifiers
Confidential M&A document for summarization	**Yellow**	**Review against policy first** + **Incognito** (not in Memory/ history; **still subject to org retention**). If policy prohibits the entry point, **it stays out**
Spreadsheet of customer PII for cleanup	(contains personal identifiers)	**Redact or anonymize identifiers before upload**, **or keep it out** and **confirm the approved path with your admin**
Patient records for a healthcare workflow summary	**Red** (the case Incognito doesn't solve)	First question: **is this entry point approved for protected health information at all?** If no approved, compliant path is confirmed → **stop and escalate to your admin**

- For regulated data, **"is this allowed here" is settled before "how do I handle it here."**

---

4. Organizational Policies and Diligence as a Habit

A policy followed only when someone is watching is not governance. The **gap between what the policy says and what people do is where risk lives**. Diligence = applying the framework **consistently** and **auditing real usage** to close gaps.

4.1 Apply Governance Consistently

- Compliance is a **sustained habit, not a one-time acknowledgment**.

- Apply the framework on **routine, low-visibility decisions**, not only the obvious high-stakes ones, because **routine decisions are where drift accumulates unnoticed**.

4.2 Audit Usage Against Policy

- **Periodically compare** what your team does (or plans to do) with Claude against what policy requires.
- Divergence = a **Diligence gap** to close, e.g.:
- a **data type uploaded that shouldn't be**,
- a **review step skipped**,
- a **Skill enabled without vetting**.

4.3 Stay Current

- **Policies and capabilities both evolve.** A practice compliant **last quarter** may not be after a **policy update or new feature**.
- Staying current keeps your **judgment in step with the tools and the rules**.

4.4 Worked Example: A Mini Usage Audit

A team lead reviews **a month** of Claude use and finds three gaps:

Gap found	Habit-level fix
Marketer uploaded an **unreleased product spec** to a **non-approved entry point**	**Reminder on approved entry points**
A **Skill was enabled without a source check**	**Skill-vetting step added to Project setup**
A recurring **client report skipped its required human-review gate twice** under deadline pressure	**Non-negotiable review gate on client deliverables**

- **None was malicious; all were drift.**
- The audit **converted invisible risk into three closeable actions**.

---

5. Ethical Implications: Bias, Fairness, Transparency

Ethical risk doesn't announce itself; it hides in ordinary outputs: a summary that **quietly favours one group**, a recommendation built on a **biased framing**, an AI-assisted document **presented as fully human-authored**. Evaluate bias, fairness and transparency **in routine review, not as a separate ethics exercise**.

5.1 Recognize Bias and Fairness Risk

- Bias can come from **the prompt, the framing, or underlying patterns in how language is generated**.
- In **people-facing work** (**hiring, evaluation, communications to specific groups**) check whether the output **treats people fairly** and whether a **framing has tilted the result**.
- **Risk is highest where stakes for individuals are highest.**

5.2 Transparency and Disclosure

- Know **when to disclose AI assistance**. Some **contexts and organizational policies require it**; others treat it as routine tooling.
- The obligation **depends on the setting and the audience**.
- **Responsible default when unsure: disclose rather than conceal.**

5.3 Reasoning Through Ambiguous Cases (four questions)

1. **Who is affected?**
2. **What could go wrong?**
3. **What does the fair outcome look like?**
4. **What disclosure does the situation call for?**

- **Reason it through and document the reasoning**: the professional standard when **no policy gives a direct answer**.

5.4 When Reasoning Alone Is Not Enough (escalate)

Escalate to your organization's **AI governance or ethics function** rather than deciding alone if:

- the **affected population is large**,
- the **potential harm is significant**, or
- the question touches areas your **team doesn't have standing to resolve**.
- **Escalate with documented reasoning, not a verdict.** It shows you **applied the framework**, **identified where it ran out**, and **flagged the gap** for the right reviewer.

5.5 Worked Example: Performance-Review Summaries

A manager uses Claude to draft performance-review summaries **from their own notes**.

Question	Answer
Who is affected?	**The employees**
What could go wrong?	Generated phrasing introduces an **unfair or inconsistent tone across reviews**
Fair outcome?	Manager **verifies each summary reflects the actual notes** and **applies a consistent standard**
Disclosure?	The setting **may call for disclosing** that AI assisted the drafting

- **Conclusion: appropriate with human review and a fairness check**, not a blanket yes or no.
- **The reasoning, not the verdict alone, makes the decision defensible.**

Links to Other Lessons (Governance Points Tested Here)

Earlier point	Lesson	Why it matters for Domain 6
Incognito applies to **standalone chats, outside Projects**, and doesn't override org data retention	01	Incognito can't be used as a control inside a Project
Project-scoped Memory keeps client contexts apart	01, 05	Prevents revealing one client's information to another
On **Team/Enterprise**, an **Owner or Primary Owner** must enable **web search** for the workspace	01	Admin-controlled capabilities
Do-not-ship-without-review list includes **regulated, confidential, or highly sensitive data** and **public or legal communications**	03	Overlaps with Red-tier data and "appropriate with human review"
Confirmation bias: prompts that imply a preferred answer tilt the output	03	A source of bias in framing
Delegation mapping (reversibility, stakes, accountability); over-delegation; staffed review gates	04	Same thinking, applied here to whole use cases
Add connectors through the **org-vetted path** (the obvious path may lead to a public directory); keep connector access **deliberate**	05	Feature-level risk and least privilege

---

Exam Traps

1. **"Incognito makes sensitive or regulated data safe."** No. Incognito only keeps a chat out of history and Memory. It doesn't confirm the data was **allowed** there, and chats still follow **org retention** and can appear in **org data exports**.
2. **"Tell Claude not to retain it" as a privacy control.** An instruction to the model doesn't satisfy a policy control. Redact/anonymize, or use an approved path.
3. **"Skip the task entirely" when anonymization would work.** Unnecessary. If the task doesn't need identifiers, **redact** and proceed.
4. **Partial redaction.** Removing names but leaving account numbers, rare job titles or specific dates can still identify people, especially in small groups.
5. **Redacting data the task actually needs.** Then redaction breaks the task; use an approved path or keep the data out.
6. **Unsure between Yellow and Red → pick the lighter tier.** Wrong. Treat it as the **more sensitive** tier until confirmed.
7. **"Appropriate with human review" with no defined gate.** "We'll keep a human in the loop" is not a gate. State **who (accountable role), what (specific risk), when (before use)**.
8. **Treating the four screening criteria as a pass/fail checklist.** They interact; name the **load-bearing** criterion.
9. **"Low stakes + reversible = fully appropriate, always."** Not if a human element applies (e.g., condolence note) or accountability can't transfer.

10. **"High consequence = always inappropriate."** A defined human gate (named reviewer signs off before use) can make it appropriate with review.
11. **"Internal Skill = safe."** Internal doesn't mean vetted. Confirm with the publisher what it accesses and that it matches **current** policy.
12. **"Skills ask for permissions, so I'll see what they access."** Skills **don't request permission**; they **inherit whatever access the session has**. Audit the bundle.
13. **Choosing only "enable" or "ban".** There's a third outcome: **escalate** to admin/security when useful but the source is unknown or access looks broad.
14. **Granting broad access "just in case".** Apply **least privilege** and revisit when the job changes.
15. **"Team plan admins can switch off Memory for everyone."** Team plans have **no** org-level Memory controls; on **Enterprise**, **Owners/Primary Owners** can disable Memory org-wide.
16. **Asking "how do I handle it?" before "is it allowed here?" for regulated data.** Permission comes first; if no approved path, stop and **escalate to your admin**.
17. **Bringing a verdict to the ethics function.** Bring **documented reasoning** that shows where the framework ran out.
18. **Unsure about disclosure → stay quiet.** Default is to **disclose**.
19. **Mixing criteria sets across lessons.** Use-case screening here has **four**: reversibility, **consequence of error**, **need for human creativity or empathy**, accountability. Workflow step mapping (Lesson 04) has **three**: reversibility, stakes, accountability. Output review thresholds (Lesson 03) have **four**: stakes, reversibility, audience, regulatory exposure.
20. **Governance only on big decisions.** Drift accumulates in **routine, low-visibility** decisions; audit usage regularly.

Most Likely Questions

Q1. [Easy] A project manager wants to upload a spreadsheet with customer names and account numbers so Claude can analyze trends. Policy restricts sharing regulated personal data. What is the most appropriate action?

- A. Upload as-is, since the analysis is internal.
- B. Remove or anonymize the personal identifiers before uploading, consistent with policy.
- C. Upload but tell Claude not to retain the file.
- D. Skip the analysis entirely.

Answer: B
Why: Trend analysis doesn't need identifiers, so redaction keeps the value without exposing data. A violates policy. C relies on an instruction to the model, which doesn't satisfy the policy control. D is unnecessary when anonymization enables the task.

Q2. [Easy] Which are the four delegation criteria for screening a use case? (Select 4)

- A. Reversibility
- B. Consequence of error

- C. Need for human creativity or empathy
- D. Accountability
- E. Output length

Answer: A, B, C, D
Why: These four screen whole use cases. Length (E) is irrelevant.

Q3. [Medium] What is the "load-bearing" criterion in a use-case classification?

- A. The criterion that scores highest
- B. The criterion that, if changed, would move the use case between classifications
- C. Always accountability
- D. The first criterion you check

Answer: B
Why: Run all four criteria, then name the one that actually decides the classification. That makes the call defensible to risk or compliance reviewers. Accountability is load-bearing in the billing example, but not always (C).

Q4. [Medium] Drafting a condolence note to a client is low-stakes and fully reversible. How should it be classified?

- A. Fully appropriate; delegate entirely.
- B. Inappropriate; AI must never touch it.
- C. A person should own it because of the relationship element, even though stakes are low and it's reversible.
- D. Escalate to the ethics function.

Answer: C
Why: The criteria interact. The need for human empathy and relationship is load-bearing here, so a person should own it. A ignores the human element. B and D overreact.

Q5. [Medium] Which statement is a properly defined human-review gate?

- A. "We will keep a human in the loop."
- B. "Someone will check it."
- C. "A manager reviews the shortlist for adverse-impact patterns before any candidate is contacted."
- D. "The team will look at outputs when they have time."

Answer: C
Why: A defined gate names who (the accountable role), what (the specific risk: adverse impact) and when (before use). The others give none of these.

Q6. [Medium] Which use case is inappropriate for AI to perform?

- A. Drafting an internal FAQ from approved policy documents

- B. Summarizing resumes into a shortlist with bias safeguards and manager review
- C. Generating a final medical or legal determination
- D. Drafting billing-complaint responses that an agent reviews before sending

Answer: C
Why: Final medical or legal determinations have irreversible consequences and professional accountability that can't transfer to a tool. A is fully appropriate. B and D are appropriate with human review.

Q7. [Hard] A bank wants Claude to draft replies to customer billing complaints. Which classification, load-bearing criterion and gate are correct?

- A. Fully appropriate; reversibility; no gate needed
- B. Appropriate with human review; accountability; a support agent checks each reply against the actual account record and adjusts tone before sending
- C. Inappropriate; empathy; humans must write every reply
- D. Appropriate with human review; output length; a manager spot-checks weekly

Answer: B
Why: The company owns what it tells customers about their money, so accountability is load-bearing. The gate must be who/what/when, before sending. D's gate is after the fact and uses an irrelevant criterion.

Q8. [Medium] What are the three checks before enabling a Skill?

- A. Speed, cost, quality
- B. Source, reach, appropriateness
- C. Role, context, task
- D. Stakes, audience, reversibility

Answer: B
Why: Who published it, what it could reach in the session and whether that's proportional (audit the bundle), and whether it's the right tool for the job.

Q9. [Hard] Which statements about Skill access are correct? (Select 2)

- A. Skills request specific permissions before running.
- B. A Skill inherits whatever access the session it runs in already has.
- C. You should audit the bundle's instructions, dependencies and files before enabling.
- D. Skills can't take actions, only read text.

Answer: B, C
Why: Skills don't request permission; they inherit session access, so ask what they could reach and audit the bundle. D is false: Skills can act through code execution.

Q10. [Medium] A Skill built by another department in your company would be useful, but you don't know what it accesses. What should you do?

- A. Enable it; internal Skills are vetted.
- B. Confirm with the publisher what it accesses and why, and that its access matches current policy, before enabling it on your data.
- C. Decline it permanently.
- D. Enable it in Incognito.

Answer: B
Why: Internal doesn't mean vetted. It may have broad access for the builders' convenience or be built against an older policy. Treat it like sister-department software.

Q11. [Medium] A useful third-party Skill has an unknown publisher and looks like it would reach far more data than its stated purpose needs. What is the best outcome of your trust check?

- A. Enable it.
- B. Escalate (or don't enable) pending your organization's review.
- C. Enable it but avoid sensitive files.
- D. Rewrite its instructions yourself.

Answer: B
Why: An unknown source plus disproportionate access means you don't enable it on your own authority. Escalate to admin or security, or decline if no review would change the picture.

Q12. [Easy] What does the principle of least privilege mean for connectors, tools and integrations?

- A. Give the broadest access so nothing breaks.
- B. Grant the narrowest access that gets the job done, and revisit it when the job changes.
- C. Only admins may use connectors.
- D. Disable all features.

Answer: B
Why: Ask who provides it, what it accesses and whether that access is proportional, then grant the minimum.

Q13. [Medium] Classify: a draft press release for a product that hasn't been announced yet.

- A. Green
- B. Yellow
- C. Red
- D. No classification needed

Answer: B
Why: Draft material tied to an unannounced deal or product is Yellow: review first. Red is regulated data, credentials and secrets, or third-party confidentiality obligations.

Q14. [Medium] An analyst can't decide whether a dataset is Yellow or Red. What should they do?

- A. Treat it as Yellow to keep working.
- B. Treat it as Red until they can confirm otherwise.
- C. Upload it in Incognito.
- D. Split the file in half.

Answer: B
Why: When unsure between two tiers, treat data as the more sensitive one. The safe default is to ask before uploading.

Q15. [Hard] An employee removes customer names from a file but leaves account numbers, rare job titles and exact dates, then uploads it. What is the problem?

- A. None; names were removed.
- B. Partial redaction: the remaining fields can still identify people, especially in a small population.
- C. The redaction broke the task.
- D. The file should have been put in Memory.

Answer: B
Why: Strip every field that could lead to identification, not just names.

Q16. [Hard] A team wants to summarize patient records for a healthcare workflow and plans to use Incognito "to be safe". What is the correct action?

- A. Proceed in Incognito; the data won't be remembered.
- B. First confirm whether this entry point is approved for protected health information; if no approved, compliant path exists, stop and escalate to the admin.
- C. Redact patient names and proceed.
- D. Turn off Memory and proceed.

Answer: B
Why: This is Red, regulated data. Incognito controls whether data is remembered, not whether it was allowed there. "Is it allowed here" comes before "how do I handle it".

Q17. [Medium] Which statements about Incognito are true? (Select 2)

- A. It keeps the chat out of history and Memory.
- B. It exempts the chat from organizational data-retention policies.
- C. Incognito chats can appear in organizational data exports.
- D. It makes regulated data safe to upload.

Answer: A, C
Why: Memory exclusion and data retention are separate controls. Incognito still follows org retention, can appear in exports, and doesn't make Red data safe.

Q18. [Medium] Who can disable Memory for an entire organization?

- A. Any member on a Team plan
- B. Team plan admins
- C. Owners and Primary Owners on Enterprise
- D. Nobody; Memory can't be disabled

Answer: C
Why: Team plans have no org-level Memory controls. On Enterprise, Owners and Primary Owners hold org-wide Memory controls, including disabling it.

Q19. [Hard] A monthly usage audit finds: an unreleased spec uploaded to a non-approved entry point, a Skill enabled without a source check, and a client report that skipped its review gate twice under deadline pressure. Which responses fit? (Select 3)

- A. A reminder on approved entry points
- B. A Skill-vetting step added to Project setup
- C. A non-negotiable review gate on client deliverables
- D. Disciplinary action, since the gaps were malicious

Answer: A, B, C
Why: None of the gaps was malicious; all were drift. The fixes are habit-level, and the audit turned invisible risk into closeable actions.

Q20. [Hard] A manager uses Claude to draft performance-review summaries from their own notes and isn't sure about the ethics. Which approach matches the lesson? (Select 2)

- A. Reason through who is affected, what could go wrong, the fair outcome and the disclosure needed, and document it.
- B. Classify it as appropriate with human review plus a fairness check: verify each summary matches the notes and applies a consistent standard.
- C. Ban AI for HR work entirely.
- D. Let Claude finalize the summaries to ensure consistency.

Answer: A, B
Why: Structured, documented reasoning gives the defensible conclusion: appropriate with human review and a fairness check. Disclosure may be required, and when unsure the default is to disclose. C is a blanket no; D removes the human gate.

Key Takeaways

1. **Governance is a practitioner skill.** One bad use case can freeze the whole AI program. Diligence gives ownership; Delegation gives the criteria.
2. **Screen use cases with four criteria:** reversibility, consequence of error, need for human creativity or empathy, accountability. They interact, so **name the load-bearing one**.

3. **Three classifications:** fully appropriate, appropriate with human review, inappropriate. **"With review" needs a defined gate: who, what, when (before use).** "It feels risky" doesn't travel; documented reasons do.
4. **A Skill is software.** Check **source, reach (it inherits session access; audit the bundle), appropriateness**, then **enable, escalate or decline**. Internal doesn't mean vetted. Apply **least privilege** to every connector, tool and integration.
5. **Classify data before upload:** Green (safe), Yellow (review first), Red (keep out unless approved path). When unsure, go **more sensitive** and ask.
6. **Redact when the task doesn't need identifiers**, and redact completely. If the task needs them, use an approved path or keep the data out.
7. **Controls:** sandbox (review uploads), Memory (persistence), Incognito (out of history and Memory, **not** out of retention or exports), org Memory controls (Enterprise Owners/Primary Owners only). **Incognito does not make Red data safe.**
8. **Policy as habit:** apply it on routine decisions, audit usage against it, stay current. Gaps are usually drift, fixed with habits.
9. **Ethics in routine review:** watch for bias in people-facing work, disclose when unsure, reason through ambiguous cases (who, what could go wrong, fair outcome, disclosure), and escalate large or high-harm questions **with your reasoning**.

07 · Troubleshooting & Optimization (Revision Notes)

Where This Lesson Sits in the Exam

Domain	Weight	Objectives covered here
7 · Troubleshooting and Optimization	**10%** (smallest domain)	Identify, diagnose, and resolve issues with underperforming prompts or poor outputs · Adjust approach based on feedback and results · Optimize workflows for efficiency and effectiveness

Exam Objective → Where It Is Covered

Domain 7 objective (exam guide)	Sections in these notes
Identify, diagnose, resolve underperforming prompts or poor outputs	1.1 to 1.5
Adjust approach based on feedback and results	2.1 to 2.4
Optimize workflows for efficiency and effectiveness	3.1 to 3.6

Core idea: when output disappoints, most people either **give up** ("Claude can't do this") or **thrash** (change random things until something works). **Diagnostic skill replaces both with a sequence.** Underperformance has **findable root causes**; the sequence usually turns a dead end into a **five-minute fix**.

This lesson pulls together earlier lessons: the **five prompt components** (02), **context management** (01), **feature and model choice** (01, 03), and **configuration maintenance** (05).

Memory Map

```
TROUBLESHOOTING & OPTIMIZATION
│
├── BAD DEFAULTS: GIVE UP or THRASH → replace with a DIAGNOSTIC SEQUENCE
│
├── 1. FAILURE PATTERNS (read the SYMPTOM TIMING)
│     ├── Under-specification ..... wrong from FIRST RESPONSE → add context/constraints/
format
│     │                            (most common, cheapest)
│     ├── Context overload ........ DEGRADED OVER TIME in a good session → restart /
summary
│     ├── Wrong feature or model .. SPECIFIC, REPEATABLE ERROR TYPE
│     │                            subtly-off numbers → code exec · shallow depth →
model tier
│     ├── Stale configuration ..... "IT USED TO WORK" → maintenance (Lesson 05 checklist)
│     └── Expectation mismatch .... task was never a fit → RESHAPE the task (range +
assumptions)
│
├── DIAGNOSTIC SEQUENCE (cheapest fix first; don't skip to step 5)
│     1 Prompt vs 5 components → 2 Conversation length → 3 Feature & model →
│     4 Configuration current? → 5 Only then: is the task a fit?
│     Common inversion: "use the most capable model" / "impossible" first = expensive
│     Ends in: cheap specific fix (usually) or reasoned "reshape the task" (sometimes)
│
├── 2. ADJUSTING FROM FEEDBACK
```

```
│      ├── Every bad output = diagnostic data about the SETUP
│      ├── Feedback loops in recurring work: same weekly correction = setup missing
something
│      ├── REACTION ("too generic") → INSTRUCTION ("add the audience")
│      │     Ask: what must be present to be right, and which LEVER controls it?
│      │     (prompt · context loaded · configuration) · can't name lever = still a
reaction
│      └── CAPTURE what worked: phrasing → standing instruction · multi-step fix → Skill
│            Test: will this correction be needed again (by me or anyone)? → configuration
│            Memory = per-user, best-effort · configuration = deliberate, shared, reliable
│            Captured (marketer) vs Lost (analyst + test accounts)
│
└── 3. OPTIMIZING WORKFLOWS (deliberate: instrument → find friction → promote)
       ├── Friction signals: REPETITION → saved context/instruction ·
       │     CORRECTION → config change · VARIANCE → shared Skill/KB
       │     Find it: run one cycle, write down EVERY manual step = backlog
       ├── CONSOLIDATE steps · PROMOTE patterns into Projects and Skills
       ├── RULE → instructions · REFERENCE → knowledge base · PROCEDURE → Skill
       ├── Promote with caution: run a few cycles with old approach available, then commit
       ├── MEASURE: time per cycle · revision rounds · consistency · quality
       │     pick the metric that matches WHY it matters; stop when good enough
       └── Audit: 45 → 25 min, consistent format, one fewer revision round
```

---

1. Diagnosing Underperformance

1.1 Common Failure Patterns and Root Causes

Pattern	What's happening	Fix
Under-specification	Prompt left out **context, constraints, or format**. **Most common cause and cheapest to fix**	**Add what was missing**
Context overload	Long conversation near the context limit; **earlier context summarized** to make room; detail compresses, **early instructions lose force**	**Restart or summarize, not a better prompt**
Wrong feature or model	Calculation asked for **in prose instead of code execution**, or **deep analysis from a speed-tier model**	**Use the right tool, not more prompting**
Stale configuration	A standing instruction, knowledge source, or Skill has **drifted out of date**, degrading output **silently**	**Maintenance**

1.2 Read the Symptom Timing (memorise)

The patterns produce **similar-looking bad output**, so the skill is reading **when and how** the problem shows up.

Symptom timing / pattern	Pattern	Why
Wrong from the very first response	**Under-specification**	The prompt **never carried what it needed**
Started fine, degraded as the conversation grew	**Context overload**	Early context compressed

Symptom timing / pattern	Pattern	Why
Specific, repeatable error types	**Wrong feature or model**	**Subtly-off numbers** → needs **code execution**; **shallow analysis** on a task needing depth → **wrong model tier**
"It used to work" (good last month, not now)	**Stale configuration**	Something it depends on **drifted**

Shorthand: **first-response-wrong · degraded-over-time · specific-error-type · used-to-work**. Each points to a **different fix**.

1.3 Isolating the Cause

Before blaming the tool, locate the failure. Is it:

- the **prompt** (specification),
- the **context** (window full, or **wrong material loaded**),
- the **feature choice** (wrong **entry point, model**, or **no code execution**), or
- an **expectation mismatch** (the task was **never one Claude could do well**)?

Naming which one **points straight to the fix**.

> **How the lists fit together:** the lesson names **four failure patterns** (under-specification, context overload, wrong feature or model, stale configuration), but this "isolate the cause" list swaps stale configuration for **expectation mismatch**, and the gallery shows **five** cases. Learn all five: the four fixable patterns (diagnostic steps 1 to 4) plus expectation mismatch (step 5).

1.4 The Diagnostic Sequence (run in order)

Step	Check	Relative cost
1	**Re-read the prompt against the five components**: anything under-specified?	**Seconds**; fixes the **most common** failure
2	**Check conversation length**: context overloaded, needs a **restart or summary**?	A little more effort
3	**Check feature and model**: a calculation needing **code execution**, or a complex task on a **speed-tier model**?	More
4	**Check configuration**: are instructions, knowledge, and Skills **current**?	More
5	**Only then** question whether the task **fits at all**	**Most expensive conclusion**: it **ends the attempt**

- **Cheapest-fix-first on purpose.**
- **Most people invert this**: they jump to **"switch to the most capable model"** or **"this task is impossible"** first, the expensive moves that **usually weren't needed**.
- **Discipline = resisting the urge to skip to step 5.**

- Run end to end, the sequence takes **a minute or two** and ends in one of two places:
- a **specific, cheap fix** (most of the time), or
- a **confident, reasoned "this task needs reshaping"** (occasionally).
- **Both are wins**: each replaces the two bad defaults with **a decision you can explain**.
- **Habit:** reach for the sequence **automatically** the moment output disappoints, **before forming an opinion about whose fault it is**.

1.5 Worked Example: Failure Gallery (five failures, five diagnoses)

Complaint	Diagnosis	Fix
"The summary keeps **missing key points**."	**Under-specification**: prompt never said which points matter	**Name the criteria for "key"**
"It **stopped following my format halfway through**."	**Context overload** in a long session	**Restart from a summary**, or **persist the format to a standing instruction**
"The **numbers are subtly wrong**."	**Wrong feature**	**Move the calculation to code execution**
"It **worked last month, now it's off**."	**Stale configuration**	Run the **Lesson 05 maintenance checklist**
"I asked it to **predict next quarter's exact sales** and the number was wrong."	**Expectation mismatch** (the **step-5** case)	**Reshape the task**: ask for a **range with stated assumptions**, or a **model of the drivers you can adjust**

- For expectation mismatch, **no prompt, restart, feature, or configuration change fixes it**: the task asks for something the tool **can't do** (predict a future number **with precision**).
- **Recognizing a genuine mismatch is as much a skill as fixing a fixable failure**; it stops you burning time tuning a prompt for an output that **was never available**.

---

2. Adjusting Approach from Feedback and Results

Every disappointing output is diagnostic data. Fixing the same problem by hand each time it recurs lets that data **evaporate**. The skill: **translate critique into a specific adjustment, then capture the fix so it persists**.

2.1 Build Feedback Loops into Recurring Work

- Treat each round's output as **a signal about the setup**.
- A recurring report needing **the same manual correction every week** is telling you the **prompt, context, or configuration is missing something**.
- Habit: after each round, ask **what the output revealed about the system that produced it**.

2.2 Translate Critique into a Specific Adjustment

- "This isn't quite right" **improves nothing**. Convert it to the **precise change**: **which component, which context, which configuration setting**.

- The adjustment must be **specific enough to act on and to check**.

Reaction vs instruction

Reaction (how it feels)	Instruction (what to change)
"Too generic"	"Add the audience to the context" / "Name the audience and the one action I want them to take"
"Wrong tone"	"Add a tone constraint to the standing instruction"
"Missed the point"	"State the single question the output must answer up front"

- **How to make the jump:** ask **"What specifically would have to be present for this to be right, and which part of the setup controls that?"**
- Each translation points to a **specific lever**: **the prompt, the context loaded, or a configuration setting**. That lever is where the fix goes.
- **If you can't name the lever, the critique is still a reaction, and the next attempt will be a guess.**

2.3 Capture What Worked

- **Don't leave a working fix in a one-off conversation** where it will be lost. **Promote it**:
- a **phrasing** that reliably works → **standing instruction**
- a **multi-step fix** → **Skill**
- Capturing turns **a lucky round into a permanent improvement**.
- **Expensive failure mode: finding the fix, then losing it.** An uncaptured fix is **rediscovered** next week **and by whoever else runs the task**; each rediscovery costs the same, **multiplied by every person and every cycle**.
- Capture **once** (standing instruction in the Project, or Skill) → **repeated cost becomes one-time cost**.
- **Test for promotion: "Will this same correction be needed again, by me or someone else?"** If yes → **configuration**.
- **Memory vs configuration:** Claude's **Memory may pick up patterns you repeat**, but it is **per-user and best-effort**. **Configuration is the deliberate, shared, reliable home** for a fix.

2.4 Worked Example: Captured vs Lost

	Captured	Lost
Who	**Marketer**	**Analyst**
Recurring problem	Every campaign-brief draft **omits the target segment** and **buries the call to action**	Monthly report must **exclude internal test accounts**
What they did	Added **two standing instructions** to the briefs Project: "Always state the target segment in the first line" and "Place the call to action in its own closing section"	Found the right instruction in month one but **kept it in conversation**, retyping it monthly

	Captured	Lost
Outcome	**Next draft arrives correct**; recurring weekly correction removed	Six months later still typing it; a **covering colleague didn't know**, and **a report shipped with test data**
Lesson	One round of feedback **captured into configuration**	**The fix existed the whole time; the failure was leaving it uncaptured**

---

3. Optimizing Workflows for Efficiency and Effectiveness

Optimization is deliberate, not accidental. You **instrument the workflow, find the friction, promote the fix into configuration**. Done well it **compounds**: each redundancy removed and pattern promoted makes the workflow **faster and more consistent for everyone**.

3.1 Find Redundancy and Friction

- Look for steps that **repeat unnecessarily**: **same context pasted** every session, **same correction** every round, **same manual reformatting** at the end. Configuration can remove each.
- Friction is **easy to live with and hard to see**, because you **absorb it one session at a time**.
- **How to find it:** watch **one full cycle** and note **every step you do by hand that you also did last time**. Still can't see it? Run it once with the **explicit job of writing down every manual step. That list is your optimization backlog.**

Three friction signals and fixes (memorise)

Signal	Looks like	Fix
Repetition	You paste or type the same thing every run	**Saved context or a standing instruction**
Correction	You fix the same flaw in every output	**A configuration change** so the flaw stops appearing
Variance	Different people running the same task get **different results**	**A shared Skill or knowledge base** so everyone runs the same setup

3.2 Consolidate and Promote

Two moves do most of the work:

1. **Consolidate** steps that can run **together** instead of as separate prompts.
2. **Promote** repeated patterns into **Projects and Skills**:
   - context you paste every time → **knowledge base or standing instructions**
   - a procedure you run every time → **Skill**

Promotion turns a personal workaround into a team-wide efficiency.

3.3 The Rule · Reference · Procedure Test (where a fix belongs)

Fix is a…	Example	Home
Rule that should always apply in a Project	"Always include the target segment"	**Standing instruction**
Reference every run needs	"Our brand voice guide", "the current product list"	**Project knowledge base**
Procedure with steps	"Generate the weekly report in this exact format and order"	**Skill**

- **Wrong home = optimization that doesn't stick:**
- a **procedure pasted as a one-line instruction loses its steps**;
- **reference material crammed into an instruction bloats every prompt**.

3.4 Promote with Caution

- **Not every change is an improvement.**
- When you move a fix into configuration, **run the workflow a few cycles with the old approach still available** before fully relying on the new setup.
- If the optimized version is **worse or less predictable**, catch it **before it has shipped several times**.
- Compounding works both ways: real **compounding benefit**, but also **compounding cost of a bad change** baked into a workflow everyone uses.
- **Prove the gain on a few runs, then commit.**

3.5 Measure the Improvement

- **Optimization you can't measure is hard to justify or sustain.**
- Track concrete gains: **time saved per cycle**, **revision cycles reduced**, **consistency across people**. E.g., **40 → 25 minutes**, or **three revision rounds → one**.
- **Time is the easiest metric but not always the one that matters most.** Other real gains:
- **Consistency**: same format, same issues caught, **whoever runs it**.
- **Quality**: **fewer errors reach the final product**.
- **Pick the metric that matches why the workflow matters:**

Workflow	Optimize for
Customer-facing report	**Consistency and accuracy** more than raw speed
Internal draft	**Time**

- Choosing the metric **up front** also tells you **when to stop**: optimization has **diminishing returns**; once the metric is **good enough**, further tuning is **its own kind of friction**.

3.6 Worked Example: Workflow Efficiency Audit

Before: weekly reporting takes **about 45 minutes per analyst**; **outputs vary by who runs it**.

Friction found	Type	Home	Fix
Each analyst **re-pastes the same background**	**Reference**	**Knowledge base**	Move background into a **shared Project knowledge base**
Each **reformats output by hand**	**Procedure**	**Skill**	Add a **Skill for the report format**
Each **catches different things**	**Rule**	**Standing instruction**	Set a **standing instruction for the verification step**

After: about 25 minutes per analyst, consistent format across the team, one fewer revision round.

- Friction was **found by instrumenting the workflow**; the gain came from **promoting the fixes into configuration**.
- Sorting each friction into **rule, reference, or procedure** turns **a list of annoyances into a concrete set of changes**.

Note on the Lesson's Disclaimer

The lesson includes an educational-content disclaimer: the course is **not legal, financial, or professional advice**; products **change quickly**, so content may be **outdated**; examples are **illustrative and often fictitious**; company mentions **don't imply endorsement or affiliation**; and Anthropic's **terms, policies and documentation take precedence** over course content. This is not an exam objective, but it reinforces "verify current product behaviour".

Links to Other Lessons (Tested Under Domain 7)

Earlier point	Lesson	How it shows up here
Five prompt components (Role, Context, Task, Constraints, Output format; C·T·R·O·C)	02	Diagnostic step 1 checks the prompt against them
Symptom → cause → fix table: generic = thin context; wrong question = ambiguous verb; wrong length/tone/shape = missing constraint/format; one section off = iterate that section only	02	Under-specification diagnosis in detail
Change one component at a time; stop when rounds give marginal change	02	Targeted adjustment, not thrashing
Context warning signs and **Restart · Summarize · Persist** (Project instructions and knowledge base carry into a new chat; the thread does not)	01	Fix for context overload (step 2)
Model tiers: Haiku = speed/volume; Sonnet = default; Opus = complex, nuanced, high-stakes	01	Fix for "shallow analysis" (step 3)
Code execution for numbers that must be right (still read and verify the code)	01, 03	Fix for "subtly wrong numbers" (step 3)

Earlier point	Lesson	How it shows up here
Maintenance checklist: instructions current, no superseded docs, right Skills enabled; custom Skills change only on re-upload	05	Fix for stale configuration (step 4)
Instruction / knowledge / Skill slot choice	05	Same as the rule · reference · procedure test
Iteration vs escalation: flat improvement on high-stakes work → human expert	03	When troubleshooting stalls on a high-stakes output, escalate

---

Exam Traps

1. **Jumping straight to "use the most capable model".** That's step 3 at best, and usually unnecessary. Start with the **prompt** (step 1, cheapest).
2. **Concluding "Claude can't do this" early.** Task fit is **step 5**, the most expensive conclusion. Rule out the cheaper causes first.
3. **Writing a better prompt for context overload.** If a good session degraded over time, the fix is **restart or summarize**, not a better prompt.
4. **Prompting harder for subtly-wrong numbers.** Wrong feature: move the calculation to **code execution**.
5. **Fixing a "used to work" problem with prompt tweaks.** It's **stale configuration**: run maintenance.
6. **Misreading symptom timing.** First response wrong = **under-specification**. Degraded over time = **context overload**. Specific repeatable error = **wrong feature/model**. Used to work = **stale configuration**.
7. **Tuning prompts for an impossible output** (exact future sales). That's **expectation mismatch**: **reshape the task** (range plus assumptions, or a driver model).
8. **Vague critique as feedback** ("not quite right"). Convert the reaction into an **instruction tied to a lever** (prompt, context, configuration).
9. **Leaving a working fix in the chat.** Promote it: phrasing → **standing instruction**; multi-step fix → **Skill**.
10. **Relying on Memory to remember team fixes.** Memory is **per-user and best-effort**; configuration is **deliberate, shared, reliable**.
11. **Putting a procedure in a one-line instruction** (loses its steps) or **reference material in an instruction** (bloats every prompt). Use **rule → instruction, reference → knowledge base, procedure → Skill**.
12. **Committing to an optimization immediately.** Run a few cycles with the old approach still available, then commit.
13. **Measuring only time saved.** For customer-facing work, **consistency and accuracy** may matter more.
14. **Optimizing forever.** Once the chosen metric is **good enough**, stop; more tuning is friction.

15. **Variance across people fixed by "everyone write better prompts".** Fix with a **shared Skill or knowledge base**.
16. **Blaming the tool before locating the failure.** Locate it first: prompt, context, feature choice, or expectation mismatch.

Most Likely Questions

Q1. [Easy] What is the most common cause of underperforming output, and the cheapest to fix?

- A. Context overload
- B. Under-specification
- C. Stale configuration
- D. Wrong model tier

Answer: B
Why: Prompts that leave out context, constraints or format are the most common failure. Adding what's missing costs seconds.

Q2. [Medium] An output was wrong from the very first response. Which pattern is most likely?

- A. Context overload
- B. Stale configuration
- C. Under-specification
- D. Expectation mismatch

Answer: C
Why: First-response-wrong means the prompt never carried what it needed. Context overload shows up after a good start. Stale configuration shows up as "it used to work".

Q3. [Medium] A session followed your format perfectly for an hour, then drifted. What is the best fix?

- A. Rewrite the prompt with more detail.
- B. Restart from a summary, or persist the format to a standing instruction.
- C. Switch to Haiku.
- D. Conclude the task doesn't fit.

Answer: B
Why: Degraded-over-time points to context overload. Earlier context was summarized and detail compressed. The fix is a restart or summary (or persisting the rule), not a better prompt.

Q4. [Easy] Financial totals in Claude's prose output are subtly off every time. What is the fix?

- A. Ask Claude to be more careful.
- B. Move the calculation to code execution.
- C. Restart the session.

- D. Add a tone constraint.

Answer: B
Why: Subtly-off numbers are a specific, repeatable error type pointing to the wrong feature. Calculations belong in code execution, not prose.

Q5. [Medium] A deep strategic analysis comes back shallow on every attempt, even with a well-specified prompt in a fresh chat. What should you check next in the sequence?

- A. Whether the task is impossible
- B. The feature and model: a complex task may be running on a speed-tier model
- C. Memory settings
- D. Incognito mode

Answer: B
Why: With the prompt and context ruled out, step 3 checks feature and model. Shallow analysis on a task needing depth points to the wrong model tier. A is step 5, the last resort.

Q6. [Medium] A monthly report Project produced good output last month but is now off, with no change to how anyone prompts. What is the diagnosis?

- A. Under-specification
- B. Stale configuration; run the maintenance checklist
- C. Context overload
- D. Expectation mismatch

Answer: B
Why: "It used to work" signals stale configuration. Something it depends on (instruction, knowledge, Skill) has drifted.

Q7. [Easy] Put the diagnostic sequence in order.

- A. Task fit → Model → Prompt → Context → Configuration
- B. Prompt → Conversation length → Feature & model → Configuration → Task fit
- C. Configuration → Prompt → Task fit → Model → Context
- D. Model → Prompt → Context → Task fit → Configuration

Answer: B
Why: The sequence runs cheapest fix first: re-read the prompt (seconds), check context length, check feature/model, check configuration, and only then question task fit.

Q8. [Medium] Why is "question whether the task is a fit at all" placed last?

- A. It's the least important step.
- B. It's the most expensive conclusion: it ends the attempt, so cheaper causes must be ruled out first.
- C. Claude can't assess task fit.

- D. It requires admin approval.

Answer: B
Why: Concluding the task doesn't fit ends the attempt. Most people invert the sequence; discipline means resisting the urge to skip to step 5.

Q9. [Hard] A sales lead asks Claude to predict next quarter's exact revenue. The number turns out wrong, and prompt tweaks don't help. What is the correct response?

- A. Switch to the most capable model and retry.
- B. Add more historical data and demand an exact figure.
- C. Recognize an expectation mismatch and reshape the task: ask for a range with stated assumptions, or a model of the drivers.
- D. Restart the conversation.

Answer: C
Why: No prompt, restart, feature or configuration change fixes a request for something the tool can't do (predict a future number precisely). Reshape it into a task that fits.

Q10. [Hard] Which reflect the "isolating the cause" step? (Select 3)

- A. Check whether the prompt is under-specified
- B. Check whether the context window is full or the wrong material is loaded
- C. Check whether the wrong entry point, model, or no code execution was used
- D. Blame the tool first, then investigate

Answer: A, B, C
Why: Before blaming the tool, locate the failure: prompt, context, feature choice, or expectation mismatch. D is exactly what the lesson warns against.

Q11. [Medium] A manager's feedback on a draft is "It's too generic." Which converts this reaction into an instruction?

- A. "Make it better."
- B. "Name the audience and the one action I want them to take."
- C. "Try again."
- D. "Use a more capable model."

Answer: B
Why: An instruction names what to change and points to a lever (here, context). If you can't name the lever, the critique is still a reaction and the next attempt is a guess.

Q12. [Medium] Which question best helps turn a critique into a specific adjustment?

- A. "Whose fault is this?"

- B. "What specifically would have to be present for this to be right, and which part of the setup controls that?"
- C. "Which model is the newest?"
- D. "How long is the output?"

Answer: B
Why: This finds the missing element and the lever (prompt, context loaded, or configuration setting) where the fix goes.

Q13. [Medium] An analyst found the right instruction to exclude internal test accounts in month one but retyped it every month. A colleague covering for them didn't know, and a report shipped with test data. What was the real failure?

- A. Claude ignored the instruction.
- B. The fix was never captured into configuration (a standing instruction), so it was lost.
- C. The model was too small.
- D. Memory was off.

Answer: B
Why: The fix existed the whole time; the failure was leaving it in conversation. A standing instruction would have applied for anyone running the task.

Q14. [Hard] Why is configuration preferred over Memory as the home for a recurring team fix? (Select 2)

- A. Memory is per-user and best-effort.
- B. Configuration is deliberate, shared and reliable.
- C. Memory can't store text.
- D. Configuration updates itself automatically.

Answer: A, B
Why: Memory may pick up patterns you repeat, but only for you and not reliably. Configuration (standing instructions, Skills) applies to everyone who runs the task. D is false; configuration must be maintained.

Q15. [Medium] Match each friction signal to its fix. "Different analysts running the same task get different results."

- A. Saved context or a standing instruction
- B. A configuration change so the flaw stops appearing
- C. A shared Skill or knowledge base so everyone runs the same setup
- D. Switch everyone to Opus

Answer: C
Why: That's variance. A fits repetition. B fits correction (the same flaw every output).

Q16. [Medium] Using the rule · reference · procedure test, where does "our current product list" belong?

- A. Standing instruction
- B. Project knowledge base
- C. Skill
- D. Memory

Answer: B
Why: It's a reference every run needs. Rules go to standing instructions; procedures with steps go to Skills.

Q17. [Hard] A team pasted its eight-step weekly report procedure into one line of the standing instructions, and outputs keep skipping steps. What went wrong?

- A. The model tier is too low.
- B. The fix landed in the wrong home: a procedure pasted as a one-line instruction loses its steps; it belongs in a Skill.
- C. The knowledge base is too large.
- D. Context overload.

Answer: B
Why: Wrong home is why some optimizations don't stick. Procedures go to Skills.

Q18. [Medium] You've just moved a fix into a shared Project's configuration. What is the recommended next step?

- A. Remove the old approach immediately.
- B. Run the workflow a few cycles with the old approach still available, then commit once the gain is proven.
- C. Tell the team the workflow is now fully automated.
- D. Stop measuring.

Answer: B
Why: Not every change is an improvement, and a bad change baked into a shared workflow compounds. Prove the gain on a few runs, then commit.

Q19. [Hard] A team is optimizing a customer-facing monthly statement summary. Which metric should lead?

- A. Raw speed only
- B. Consistency and accuracy
- C. Number of prompts used
- D. Length of output

Answer: B
Why: Pick the metric that matches why the workflow matters. Customer-facing reports optimize consistency and accuracy more than speed; internal drafts optimize time.

Q20. [Hard] A weekly reporting workflow takes 45 minutes per analyst and varies by person. Analysts re-paste the same background, reformat by hand, and catch different things. Which mapping of fixes is correct? (Select 3)

- A. Background → shared Project knowledge base
- B. Report format → Skill
- C. Verification step → standing instruction
- D. Background → Skill

Answer: A, B, C
Why: Reference → knowledge base, procedure → Skill, rule → standing instruction. The result was about 25 minutes, a consistent format, and one fewer revision round. D puts reference material in the wrong home.

Key Takeaways

1. **Don't give up or thrash; diagnose.** Underperformance has findable causes, and the sequence usually yields a quick fix.
2. **Read the symptom timing:** first-response-wrong = **under-specification**; degraded-over-time = **context overload**; specific repeatable error = **wrong feature or model**; used-to-work = **stale configuration**. A task that was never a fit = **expectation mismatch**: reshape it.
3. **Run the sequence cheapest-first:** prompt → conversation length → feature & model → configuration → task fit. Don't jump to "bigger model" or "impossible".
4. **Turn reactions into instructions** by naming the lever (prompt, context, configuration).
5. **Capture fixes:** phrasing → standing instruction, multi-step fix → Skill. Memory is per-user and best-effort; configuration is shared and reliable. Uncaptured fixes get rediscovered at full cost by everyone.
6. **Optimize deliberately:** look for repetition, correction and variance; consolidate and promote. Use **rule → instruction, reference → knowledge base, procedure → Skill**.
7. **Promote with caution and measure:** trial a few cycles, then commit. Track time, revision rounds, consistency or quality, depending on why the workflow matters, and stop when the metric is good enough.